Myfirsthack Networking Foundation Notes

DAY-01/02 Intro to Cybersecurity and Roles

Cybersecurity: protecting computers, network, data, and people from digital attacks

- Around 82% of cyberattacks involve human error or compromised credentials.

SOC analyst: security operations center.

- a log-in into your dashboard called SIEM
- looks for suspicious activity, investigate, pull the logs and write a report

GRC Analyst: Governance, Risk and Compliance.

- accessing wether a new software vendor is a security risk
- writing a security policy that thousands of employees would follow.

DAY-03 Audit

Audit: A structured look at a system to find the gaps between how secure it is and how secure it should be.

This happens in 5 steps- and the **5 step Audit methodology** is

1. **Scope**: Define what you're auditing
2. **Intelligence**: gather information on your target, what's publicly known and what has been exposed.
3. **Exposure**: Identify current weakness, where are the gaps now?
4. **Severity**: Rate what you found.
5. **Remediation**: Fix at least one thing today and document the rest.

IT

DAY-04 How Computers Actually Work?

1.) **CPU (Central Processing Unit)**: A part that does everything. It carries billions of operations per second.
    a.) Spectre and meltdown vulnerabilities in 2018 exploited the CPU
2.) **RAM (Random Access Memory)**: Where the computer keeps whatever it's currently working on like the half written emails, tabs, the passwords that the browser saved 2 seconds ago.

a.) RAM is everything that a machine is thinking at that moment forensic investigators firstly do in an incident is memory dumping i.e, capture the RAM before it disappears

3.) **Storage**: where your files live when the computer is off. The hard drive, SSD. IF RAM is your desk, storage is the filing cabinet. These are long term prizes for an attacker like your personal documents, database and photos

4.) **Network interface:** The part that lets the computers talk to other computers. WiFi card, ethernet port, cellular modem. Attackers use this to get in and out of a system.

- Every program that's actively doing something is called a process. Every running process is a door for the attacker.
- Attackers can plant something via getting a malicious process running.
- The duty of a SOC analyst is to check if there are any malicious programs running that are not supposed to. The trick here is to learn what's normal and spot the abnormal.
- Abnormal process appears out of nowhere and looks legitimate but slightly off like(sch0vs.exe instead of schovs.exe), making unexpected network connections, it's running from an unusual file location
- **Homolyph attack**: using characters that look almost identical to trick the human brain into thinking a malicious process is legitimate. (Using "0" instead of "O", and "-" instead of "_").
- Every file has an address and everything in your computer is a file. Address is that path that tells the computer exactly where to find it. A file's location is part of its identity.
- c:/user/Admin/AppData/Local/Temp/update.exe: this is an address path for a file. The file name is "update.exe". A program running in Temp/downloads is enormously more suspicious than one running from program files, because attackers cannot write in protected system folders, so they drop their malware wherever they can write.

- Security Investigations are often just a defender looking at the full path of a suspicious file and asking does this belong here?
- Network connection is where the crime happens. A computer always communicates through the network. Security exists for this reason: watch every outbound connection and flag that ones looks wrong
- They are called **EDR(Endpoint Detection and Response)**, because a malicious process on some internal machine opens a connection to the server controlled by the attackers, and slowly sends data over a period of time, and only these tools can prevent this from happening.

DAY-05 7 Step approach of a search made on the web.

What actually happens when you type google.com into your browser and hit enter.

- Signals from the laptop roughly makes 7 stops on its way to google and back
- Every stop is an attacker's surface. Learning the stops will make you understand the map of the entire field
- Computer only understand numbers, every device connected to the internet has its own IP address, and its real identity
- Original IP system (IPv4 only had 4 billion combination and it ran out a few years ago so they patched the internet with IPv6 a new system)
- DNS (Domain Name Server) A phonebook that maps out human names to machine numbers
- The Browser asks the OS(Operating System) what's the IP address of google? OS asks the router, router asks ISP's(Internet Service provider) DNS server, and it asks the root server which points to a ".com" server, which points to google's DNS server that gives "142.250.191.46" and it travels back to the browser.
- **DNS Spoofing:** Intercepting the above search process and returning a server number that attackers control they can then get your personal information.
- DNS is the translation Layer

The 7 step Journey of a search made:

1. **Browser phrases the URL**: It figures that you want google.com on port 443 (HTTPS)
2. **DNS lookup**: translates google.com to IP address like "142.250.191.46"
3. **Computer opens a TCP connection**: Connection is made with the server.
4. **TLS Handshake**: The conversation that is happening between the computer and server is encrypted so that there are no interceptions.
5. **HTTP request sent**: Browser sends the actual GET/ request to send the page.
6. **Server process and responds**: google server then process the things and sends backs HTML, images and scripts
7. **Browser renders the page**: browser then takes all the blob and into the pictures that you can see.

- Every security control you'll ever learn will either block one of those seven steps, monitor one of them, or cleans up after one of them goes wrong.
- Firewalls intercept at step-3, TLS certificate protects steps step-4, web application firewalls intercept step-5, Antivirus catches from Step-6. And EDR watches all over these steps.
- **Traceroute command:** Network diagnostic tool that tarcks the path your internet traffic takes. Hop to Hop to reach a particular destination.
    - This shows you the physical skeleton of the internet, every router through which the traffic had passed through b/w you and destination usually 10-15 hops for a major site.

DAY-06 IP Address

- IP address- four numbers b/w 0 to 255 separated by dots.
- Your laptop has 2 IP address at the same time
    - Private: The one the home router gave to the laptop. It is invisible to the outside world. This helps the router to tell the devices at home apart from each other.

    - Public: A signal address that ISP gave to the router.Every device in the house shares it when talking to the outside world to google. The laptop, TV and smart fridge all are the same.
- The router does the translation in between. It is called NAT (Network Address Translation)
- “127.0.0.1” is called “localhost” and it is used for testing. It tells the computer to talk to itself.
- IPv4 – ran out in 2011, two fixes emerged
    - NAT - only one IP address to all the devices connected to the same network.
    - IPv6 - eight groups of hexadecimals. A trillion combinations
- Every moment on the internet leaves traces on your IP address on log files
- With an IP address you can know your rough location, your ISP, what kinda connection you are on, your country , your approximate speed tier.
- Attacks that can happen:
    - **Port Scanning**: Probe every one of the 65,535 ports on an IP to find vulnerabilities
    - **Reputation Lookup**: was it part of any data breach
    - **Cross site correlation**: Same IP for different account and different services
    - **Geo-targeted Social Engineering**: Attackers know your location.
    - **DDoS Attack**: Attackers know your location
- 192.168.x.x a private IP address never to be used on the public internet.

DAY-07 DNS

DNS- A whole global network that translates website names to IP address

- When you type google.com it does not ask the internet directly for an IP address. It runs a specific, ordered conversation across 8 different systems.
    1. Browser Cache: Chrome/FireFox store a list of recent lookups. The browser remembers IP and skips everything below.

2. Operating System Cache: If browser doesn't know, your OS check in it cache
3. Router Cache: Routers also cache DNS answers for every single device on the same network
4. ISP recursive resolver: IF no cache is available the router forwards the question to your ISP, Their server is called recursive resolver and it does the job of finding on your behalf.
5. Root Server: The recursive resolver asks one of the 13 root servers that sit at the top of the DNS hierarchy.
    a. Even if the root server does not know the IP address all it knows is anything that ends in ".com", ask these servers.
6. TLD(Top level Domain) Server: Google's own DNS server, the one that actually holds the definitive answer
7. Authoritative server: Google's own DNS server, the one that actually holds the definitive answer.
8. Back to you: The recursive resolver cache's the answer, hands it back through your router, which hands it to your OS which hands it to your browser which finally opens google.com

- All eight steps happen in 100 milliseconds for the first time and zero milliseconds every time after that.
- **TTL(Time to live)**: Every DNS answer comes with an expiry timer, it could be 5 min for some websites that change fast and 24hr for the stable ones.
- Caching is the load bearing wall of the modern internet.DNS works not because it is fast, it works because it almost never has to actually run.
- Every layer caches aggressively, so that the authoritative servers only hear from resolvers whose caches expired i.e, a tiny fraction of real world traffic.
- This level of caching is great for performance, but catastrophic for security. Because if a resolver's cache is poisoned then all the user downstream will get those results until TTL expires

    - Shorter TTL: Faster propagation and more load on the server
    - Longer TTL: faster, cheaper but poisoned entries stay longer
- You are not attacked, you are just handed the wrong information
- Where does DNS break?
    - DNS is the most attacked, as it's not fully encrypted, most networks trust DNS traffic implicitly.
- There are four kinds of **DNS attacks**:

    1. **DNS spoofing or Cache poisoning**: Attackers trick a recursive resolver into caching a wrong answer, so every user who queries that resolver gets sent to the attacker's server instead of the real one.

    2. **DNS Hijacking**: Attackers direct a heavy traffic towards cache and they compromise the DNS setting themselves. The router's malware rewrites your router's DNS server from an original one to attacker controlled IP. From then every domain you try to visit goes through the attackers resolver, and they can send you wherever they want. Very common in home routers with default passwords.

    3. **DNS Tunneling**: Attackers abuse DNS to smuggle data in and out of the networks. Because DNS queries are allowed through almost every firewall. The DNSMessenger malware family used this technique to run entire remote-control sessions over DNS.

    4. **Typo squatting:** It's an attack that lives on DNS behavior. Like “gooogle.com" instead of “google.com”. Build a convincing fake one and wait for people to mistype. DNS does exactly what it is supposed to do, it resolves the typo domain to the attackers server, because that's a real domain they legitimately own.
- Every defender watches over the DNS logs. Unusual query patterns— a workstation suddenly sending thousands of DNS requests per minute, or a domain nobody should recognize appearing in traffic. This is one of the loudest and earliest signals that a network is compromised.

DAY-08 Passwords

- There are four ways to break a password:
    1. **Credential Stuffing**: An attacker downloads a database of usernames and passwords pairs from the last breach, and they then try each single pair against the other sites, it mostly works because people reuse passwords. This is the most common attack on the internet somewhere between 70% - 80% of account takeovers come from credential stuffing.
    2. **Phishing** :Attackers send you a fake email from your bank and ask you to login into a page that looks real but belongs to them. You type your password, and they have it, the length of the password does not matter the moment you type into a wrong website.
    3. **Dictionary attacks:** Real brute force attack but targeted. The attackers use a list of the most common passwords and predictable variations. Modern hardware can test billions of these guesses per second against a stolen password database.
    4. **Password spraying**: The inverse of a dictionary attack. Instead of trying many passwords against one account, the attackers try one common password against thousands of accounts.
- Length beats complexity, cause a complex 8 characters can be broken by modern gpu in a few hours, but when it comes to a 16 characters the possibilities raises a lot and it would take the same gpu a decade to break
- A 25 characters passphrase you can actually remember beats an 8-character random string you'll forget and reuse. The point of a password is not to look complicated. It is to be uniquely yours and long enough that searching for it isn’t worth an attackers time
- 3 things that would make a difference: have different passwords for different accounts, use a passwords manager and use 2FA for all the accounts.

DAY-09 Phishing

- Phishing is the most common attack in the world, around 70%-80% of the attacks happen because someone clicked something that they were not supposed to.
- Things that you can spot in a phishing email are the sender's domain: they usually have a typo and resemblance to the original one. Link description: when hovered the link shows that it is directing you to another page, which is often ignored. The ask: there is always an ask to do something in the email.
- Every phishing email pulls one or more of three psychological levers.
    1. **Urgency:** it bypasses you from slow and deliberate to fast and instinctive. Here you first click and evaluate later.
    2. **Authority**: It triggers a compliance response built into every human since childhood. The more senior the apparent sender, the more likely you are to act before questioning.
    3. **Curiosity:** It is the weakest lever on the paper because it gets you to the link before you've even started evaluating legitimacy.
- A 5 habit check list that will keep you safe:
    1. **Hover before clicking every link every time:** Always rest your cursor on the link, and if the visible link and the actual link do not match then it's a phishing email.
    2. **Never authenticate from an email link ever:** Never click the links to log-in into your emails, banks or any accounts. Always open a new tab and then login through the direct official website.
    3. **2FA is Important**: The phishing password is just a half key, you still need to authenticate it using your phone for a hardware device.
    4. **Treat Urgency as a red flag, not a reason to hurry**: when an email makes you feel like you need to act right now, that's a feeling that the attacker is going for. Real emergencies rarely arrive by email.

5. **Verify suspicious requests**: email from your boss asking you to send a wire transfer? Text or call them before doing so, call the bank from the number mentioned behind the card rather than trusting the email.

- AI has made things believable, the voice generation, emails with official tone and video deepfakes make it more believable, but stick to the 5 step process.

DAY-10 DATA

- In the eyes of a security professional, you should not be thinking about how this works technically, instead you should be thinking about what data lives in here, and why would someone want it.
- There are three categories of data:
    1. **Data about people**:Industry calls it PII: personally identifiable information. Names, DOB, addresses, phone numbers, emails, insurance, SSN, passport numbers, DL ids, medical reports, biometric data, anything that identifies as a real human.
        a. PII- enables to commit identity theft, fraud, targeted scams, and resale on underground markets. Medical records specifically are valuable because they include enough data points to open bank accounts or commit insurance fraud in someone else's name.
    2. **Data that proves that you are you**: Credentials, passwords, password hashes, session tokens, API keys, SSH keys, 2FA recovery codes, anything a system uses to verify identity.
        a. The twitter bitcoin hack, a phishing email got the credentials of an employee and then they hacked into Obama, apple and other accounts and posted about the Bitcoin and made 120k in a few hours.
    3. **Data that belongs to an organization**: Proprietary data. Source code, internal documents, financial records, customer databases, trade secrets, product designs, etc.

    a. The 2014 Sony Pictures hack- attackers got 100 terabytes of data including unreleased films, executive emails, employee personal records, salary information, and scripts for future projects.

- Not every data is of the same value, the more permanent the data, the more valuable it is to steal. Like a stolen credit card can be replaced but not the info in medical records.
- Hospitals are the most attacked by ransomware, because it works against organizations that cannot tolerate downtime, when a hospital cannot operate, patients literally die, so they pay.
- **Permanence+leverage= value**.
- Three questions that every security pro asks is:
    1. **What data does this hold?** Answers are obvious, a customer database holds customer records, a log system holds the IP addresses, session identifiers.
    2. **Which of the three categories?** PII? Credentials? Proprietary? Often a mix. Being able to name the categories tells you immediately what attack patterns apply, what regulations apply.
    3. **Who would want it and what would they do with it?** How hard will the attackers try, what kind of attacks. What the worst case scenario looks like.

DAY-11 WEB

- The internet moves the data. The web is what you see when that data is shaped into pages a browser can render. What actually happens when you load a page?
- A browser is a complex piece of software which runs the following:
    - **Renderer**: runs the code into page you can see
    - **Script engine**: Javascript that makes pages interactive
    - **A network client**: making hundreds of requests in the background
    - **A storage system**: cookies, cache, and local data
    - **A security boundary**: deciding what one website should be allowed to do.
- What actually happens when you open a browser

    - **DNS lookup:** asks what's the IP for a particular website
    - **TCP connection:** opening a network connection to that IP
    - **TLS handshake:** if the URL starts with ‘https://', the browser and the server negotiate encryption. IF there is something wrong with the page, then it will not load.
    - **HTTP reques**t: browser sends a polite text based request asking for the homepage
    - **HTTP response:** the server replies with HTML of the page, plus instructions on what else to fetch
    - **Resource fetching:** after the browser reads the HTML and discovers it needs more stuff like images and fonts, so it makes another request for those.
    - **Rendering:** The browser parses the HTML, applies CSS and JS and paints the page on screen
    - **Ongoing chatter:** after the page is loaded the scripts still keep talking to the server to keep it live.
- Loading a webpage is not one event.It's hundreds of tiny events, each with its own trust assumptions, each potentially exploitable.
- There are two halves of the web, one is front end- what a customer can see, the HTML, CSS and JS, and the backend- the servers, databases, and code running on the company's infrastructure. The backend is private
- The conversation between frontend and backend happens through an **API, it's a predictable little doorway** the front-end uses to ask the backend questions like get me this user's orders, Add the item to the cart.
- Attacks in frontend are Cross site scripting, malicious extensions, and fake logins pages
- Backend-end attacks target the server. SQL injection, broken authentication, leaked database.

- A weak frontend means data gets stolen when its passed, but a weak backend means data get the entire customer list at once
- A webpage is built in three layers they are HTML, CSS and JS
    - **HTML carries forms**: forms are how attackers harvest credentials on phishing pages. Every fake login page you'll analyze lates is HTML mimicking a real one.
    - **CSS can hide things**: Attackers use invisible CSS to hide malicious links inside legitimate-looking text.
    - **JS:** A malicious script can read what you type, send your cookies to an attacker, or rewrite the page in front of your eyes.

DAY-12 HTTPS

- HTTPS protects the channel, not the destination. The padlock(next to the URL) proves the conversation is private. It does not prove the person on the other end is who you think they are.
- Through HTTP you can read the conversation that is happening between the browser and the server, like anyone with the same Wi-Fi as you, the internet provider and every network device your traffic passes can intercept the conversation.
- HTTP sends your data in plain text. HTTPS encrypts it. The shift from one to the other was driven by attackers proving publicly how trivial the alternative was.
- What actually happens when you see a padlock three things specific things have happened in order:
    - **The browser and the server agreed on encryption:** They negotiate which cipher to use, exchange keys and set up an encrypted tunnel.
    - **The server presented a certificate:** A small file that says that I am bbc.co.uk , and here's my proof. Your browser checks for that.
    - **A trusted third party vouched for that certificate**: The browser doesn’t just take the server’s word for it. The certificate has to be signed by a

Certificate Authority- a company browsers have agreed to trust. Companies like Digicert, Sectigo and Global sign.

- Here are the things that can go wrong:
    - The cert only proves that the domain belongs to them, but it could be anyone, like a malicious site or the original one.
    - It does not prove the site is safe. Phishing sites use HTTPS routinely and around 71% of them have the encryption and so does the bank.
    - Anyone can get a free certification for a domain in five minutes. The bar is “I control" this domain not “I am trustworthy”
- The padlock only proves three narrow things: encryption is on, the server identifies itself, and a CA signed off. None of these three things is the same as " this site is safe"
- What does an analyst actually check for:
    - **The domain itself, character by character:** The actual domain in the URL, Phishers buy real certificates for lookalike domains all day long: netfl1x.com, amaz0n.com. The padlock will be green and proud on every single one of them
    - **The certificate's subject and issuers:** The certificate issued should be on the name of the company but not some random reseller in the country through which the company doesn't operate.
    - **Whether the certificate is recent and short lived:**Modern certificates expire fast currently 90 days for a Let's encrypt certificate and soon to be short as 7 days across the whole industry. That's on purpose short lifespan means a stolen or mis-issued certificate has a small window of damage.
    - **The full URL, including subdomain:** Attackers love subdomains. Paypal.com.security-update.net is a domain on security-update.net not on paypal.com. Read the urls right to left not left to right— the rightmost dot tells you who actually owns it.

- The padlock is a starting point for investigation, not an ending one. Real phishing analysis starts after the padlock— checking domain spelling, certificate issuer and URL structure.
- **“ssllabs.com/ssltest”** It's a free tool from Quay’s, used by SOC analysts and infrastructure engineers worldwide. When you type the domain address in it, you will get a grade report that breaks down exactly which TLS versions the site supports, which cipher, whether it has known vulnerabilities, and how its certificate chain is configured. This is the same report a professional would generate during a security audit.

DAY-13 COOKIES

- Cookies are how the web remembers you between page loads. They're also one of the most stolen things in cybersecurity, because if an attacker has your cookie, the website thinks they are you.
- Every time a browser made a request for a page, image -- the server treated it as a new conversation. The servers had no memory. It's the same problem across banking, shopping, email and social media-- all of them had to know which user is making the request, otherwise it would ask them to login again and again.
- The web is stateless— every page request stands alone. Cookies were invented to give it memory. They're the pass that the browser hands back to the server on every visit.
- The cookies made a stateless protocol to stateful by a single text file.
- What does a cookie look like?
    - `session_id=a8f3kp29d1xq7b4m; expires=Thu, 31 Dec 2026 23:59:59 GMT; secure; httponly; samesite=lax`
    - **A name and value**- The name(session_id) tells the server what kind of cookie it is, the value(a8f3kp29d1xq7b4m) is the actual data, a long random string that points to your record on the server's side.

    - **An expiry date:** when the cookie should stop working, some last 5 min some last 5 years
    - **Security flags:** small instructions that change how the browser handles the cookie.
- When you log into a site, the server creates a record on its end, and sends your browser back a cookie containing the random ID that points to the record. From then every time the browser talks to that site, it includes the cookie
- A news site typically has 50-200 cookies, only some of them are from the news site, but the rest are from ads, analytics providers, embeds and tracking pixels.
- Session cookie is the one that matters the most for a security analyst. Because it is the one that proves that you are logged in. It is the one that the server checks every time you load a new page after signing in.
- If you have it, the site treats you as the logged-in user. If you don't, you're a stranger who needs to authenticate again. Session cookies don't care who is holding them.
- Session hijacking-- stealing someone's session cookie is one of the most consequential attacks on the modern web. It bypasses password and 2FA.
- In 2010 a free Firefox extension called Firesheep showed the world that cookies traveling over plain HTTP could be stolen with one click. The industry's response was universal HTTPS, which was not voluntary. It was forced.
- The defense for the cookies lies in the security flags on the cookie itself- every analyst should learn these.
    - **"Secure"**: tells the browser to only send this cookie over HTTPS, never over HTTP. If a site sets a session cookie without this flag, an attacker who can downgrade the connection can capture it.
    - **"HttpOnly"**: Tells the browser" don't let the JS on the page read the cookie". This one is the defense against the cross-site scripting attack(XSS) where you inject malicious JS into the page(in the sites comments section)

and it can normally read every cookie in the user's browser and exfiltrate it to the attackers server.

    - **"SameSite"**: tells the browser when it's allowed to send the cookie on request originating from other sites. This is a defense against cross-site request forgery(CSRF).Without it , a malicious site could trick your browser into sending requests to your bank that include your bank's cookies— effectively making the request look it came from you
    - **"Expires"/"Max Age":** how long the cookie lives Short lives session cookies limit the window of damage if one is stolen. Some financial sites expire sessions after % minutes of inactivity for exactly this reason.
- Cookie security isn't magic- its four flags developers either set or forget. Secure, HttpOnly, SameSite, and a sensible expiry. Most cookie theft incidents trace back to one of these missing.
- Session cookie- theft: no login event in the logs at the time of breach, because the attackers never logged in.

DAY-14 WiFi

- The home wWi-Fi router is one device doing three jobs: a network gateway, a security boundary, and a piece of software that might be a decade old. All three matter
- The router is the front door, most people think that the router only gives them Wi-Fi but that is the smallest part of what it does, a modern router does the following:
    - **Connecting your home to the internet:** The only device in the house that talks directly to your ISP
    - **Issuing IP addresses**: It does this to every device in the network
    - **Resolving DNS**: for all the devices, often through your ISP's server
    - **Running a firewall**: that blocks most of the unsolicited inbound traffic from the wider internet

    - **Broadcasting Wi-F**i: to every device that is in the rage to connect to.
- The router has a different username and password, it's not the same as the WiFi password. The admin password is what you'd use to change the router's settings, including the Wi-Fi password itself
- For a startling number of home routers, the admin password is still the one printed on a sticker on the back. Sometimes it's "admin" / "admin"in or "admin" / "password".Sometimes the manufactures print a unique password but the attacker can guess that based on the model of the router
- If the attacker gets hold of the admin password, they own the router. They can change the DNS to point your traffic through their servers. They can open ports to the wider internet. They can install malware on the router itself. From that position they don't need to break HTTPS. They sit underneath it.
- The router and your phone establish an encrypted channel using a Wi-Fi security protocol. Over the last 25 years there have been four of these, and the story of how each one broke is the story of why network security keeps getting harder.
    1. **WEP (Wired Equivalent Privacy)-** released in 1997. Designed to make wireless feel as secure as wired.Within years researchers found a way to break this using the off-the self tools, anyone using this in 2026 is basically connected to the open network. Most modern routers don't even provide this option anymore.
    2. **WPA (Wi-Fi Protected Access)**: 2003 A patch on top of WEP, intended as a stopgap until something better arrived.
    3. **WPA2-** 2004: The standard for two decades. Still in widespread use today. WPA2 with AES encryption is genuinely strong when configured correctly, through the protocol had serious flaws
    4. **WPA3-** 2018. The current standard. Fixes structural problems in WPA2's handshake. Modern routers and devices support it, older ones don't. Most homes today are still on WPA2 because not every device on the network supports WPA3.

- Why does Wifi need encryption at all if HTTPS already encrypts the actual websites I visit?
    - Not all devices use HTTPS, devices like printers, smart bulbs and older IoT devices often talk in plain text.
    - An attacker who can see the unencrypted WiFi can learn which sites you visit, when, from which devices, even if they can't read your content.
    - Wifi encryption protects the metadata, not the actual data.
- In October 2017, a Belgian researcher called Mathy Vanhoef published a paper that broke the internet for a week. His team had found a flaw in WPA2's four-way handshake (the cryptographic dance two devices do when joining a network) that allowed an attacker within Wi-Fi range to decrypt traffic.
- The attack was called KRACK (Key Reinstallation Attack). It didn't need to crack the Wi-Fi password. It didn't need any inside information. It exploited a flaw in the protocol itself, which meant it worked against every WPA2 network on Earth.
- KRACK in 2017 was a reminder that Wi-Fi security can fail at the protocol level, not just at the password. Patches fixed it for modern devices. The unpatched ones (smart cameras, old routers) are still vulnerable today.
- Public Wifi what it actually is in 2026. Now that the 95% of web is HTTPS, it is secure unlike the firesheep attack from 2010 where anyone the same network could intercept the traffic, but there are new problems that arise now:
    - **Evil Twin networks:** An attacker sets up a Wi-Fi access point with the same name as a real one ("Free Airport WiFi"). Your phone joins it automatically. The attacker now sees all your traffic and can launch other attacks from a privileged position.
    - **DNS manipulation**: On a network the attacker controls, they can answer DNS queries however they like. You type paypal.com, you go to a fake site that looks like PayPal, and on the network's own machinery, the attacker can even strip HTTPS warnings.

    - **Captive portals as malware delivery**. Those "accept terms" pages on hotel and airport Wi-Fi have been used to push malware to users who think they're just clicking through to the internet.
    - **Plain old surveillance:** Even with HTTPS, the attacker sees which sites you visit, when, and for how long. That metadata alone is valuable.
- Use a VPN if you're doing anything sensitive, never enter credentials on a captive portal page, and don't let your phone auto-join unfamiliar networks.
- If a security professional walked into to audit the home network this is what he would do:
    - **The router admin password:** Has it been changed from default? If yes, is it strong? If not, fix it now.
    - **The Wi-Fi encryption mode.** Is it WPA3, WPA2-AES, or something older? "Open" or "WEP" or "WPA-TKIP" are immediate red flags.
    - **The Wi-Fi password.** Is it short, predictable, or shared with a hundred people? A 16-character random password takes attackers years to brute-force; an 8-character word with a number on the end takes minutes.
    - **The router firmware.** When was it last updated? Most routers can update automatically. Many don't have it switched on. A router running 2019 firmware in 2026 has six years of unpatched vulnerabilities sitting in it.
    - **The guest network.** Does the router have one, and is it switched on? Guest networks separate visitors from your main network so a friend's compromised laptop can't see your devices. Smart home gadgets (cameras, plugs, bulbs) ideally live on the guest network too. When one of them inevitably gets compromised, it can't pivot to your laptop.
    - **Remote management.** Can the router admin page be reached from outside your home network? It almost never needs to be. Switch this off if it's on.
    - **Connected devices**. What's on the network? Most routers list every device that's ever connected. Look for things you don't recognise: neighbours who never quite left, old phones, IoT gadgets you forgot you owned.

- Home networks aren't small versions of corporate networks. They're what corporate networks looked like in 2008, with the same risks. The difference is that Maya is the only person responsible for hers.

DAY-15 Phishing Email Analysis

- A phishing email is the most common attack on the modern web. Every SOC analyst, regardless of seniority or specialism, can analyse one.
- Five things that you'll do for analysing a phishing email
    - Source a real phishing email
    - Read the headers to find out where it came from
    - Investigate the URLs and attachments safely without clicking
    - Identify the social engineering technique the attackers used
    - Write a SOC-style incident report
- 3 safety guidelines that you need to follow:
    - Never click any link in a suspicious email: Hover over the links to see where it goes. Right-click and "copy link address" if you need to paste them into an analysis tool
    - Never download or open attachments: The safest way to investigate a file is to grab its hash(a fingerprint of the file content) and check the hash against known malware database
    - If you're not sure about something, stop: Real SOC analysts work in an isolated environment called Sandboxes for exactly this reason.

DAY-16 Mobile Devices

- The phone in your pocket is the most attacked device most people own. It is also the device most people protect the least
- 5 things that would make a mobile different than a laptop
    - **The OS is sandbox by default:** On a desktop, any program you install can usually read any file you have, but on an modern iOS/Android device every

app lives in its own sandbox. It cannot read other apps' data unless you explicitly grant permission.

    - **The users grant permission in the moment:** When the app asks “Allow access to contacts?”. Most people tap without thinking. This is the central decision point on a phone. A flashlight app that asks for your location, microphone, contacts and storage is doing what flashlight apps shouldn't need to do.
    - **Updates are out of your hands:** On laptops you choose when to update, but with phones the manufacturers gate it. An android phone stops getting updates after 2-3 years and it's the same in iPhones also but after a longer time.
    - **The attacks surface in messages not the browser:** On a desktop, most attacks land via emails or web. On mobile they land through messages. The user is conditioned to react fast to messages, especially from short numbers and unknown senders. Smishing(SMS phishing) and quishing(QR-code phishing) are mostly mobile specific
    - **Biometrics replace passwords most of the time:** Face ID, fingerprint, PIN. The phone is authenticating you constantly, often without noticing. That changes the threat model. An attacker who gets your unclocked phone is briefly you in a way that’s hard to replicate on a laptop.
- The story of pegasus- a piece of spyware built by the Israeli company NSO Group.What made Pegasus historically significant is that it evolved from "click this link" to zero-click(The phone receives a specially-crafted message (iMessage, WhatsApp, SMS), processes it in the background to render a preview or notification, and the exploit triggers during that processing.)
- In May 2025, a US court ordered NSO Group to pay damages to WhatsApp for around 1,400 affected users in a single 2019 campaign.The case marked the first time a spyware vendor was held legally responsible for the exploitation of a messaging platform.

- The zero click exploit is currently being sold for 2.5 million on the legitimate exploit market.
- The most common attack that happens on a mobile phone.
    - **Smishing:** Text-message phishing. “Your parcel could not be delivered”, “Your license has expired” and many more.The link goes to a credential-harvesting page that looks identical to the real brand. Mobile users have less visual real estate to inspect URLs and are conditioned to tap fast.
        - Smishing has overtaken email phishing as the primary entry point for consumer fraud in many countries.
    - **Malicious apps**: Even with App store and google play screening malicious apps slip through, In 2025 google removed 77 malicious android apps that had 19 million installs collectively. Most posed as utilities — file managers, QR scanners, photo editors — and ran banking trojans in the background that overlaid fake login screens on top of real banking apps when the user opened them.
    - **Sideloading**: Loading apps from outside the official store. Android allows it, but ios makes it harder.Sideloaded apps skip the (imperfect) screening of the official stores. Phishing campaigns often direct users to download a "legitimate-looking" APK file from a fake website. The user grants permissions, the app starts logging keystrokes or stealing SMS messages, the user has no idea anything happened.
    - **Public Wi-Fi as a phone problem:** Phone auto join networks they’ve seen before, including networks an attacker has named “Free Airport WiFi” or “Starbucks guest”. Your phone is silent.
    - **Lost or stolen phones:** The most prosaic threat, and the one most likely to actually happen to you. An unlocked phone in the wrong hands gives the attacker minutes of unrestricted access to your email, banking, photos, and password manager.

- What an analyst would actually check on your phone: This is the Six step phone analysis.
    - **OS version**: Are you on the latest available OS. Are you the latest available OS. Some Android phones simply cannot upgrade past a certain version because the manufacturer stopped supporting them. If that's your situation, the honest answer is the phone needs replacing.— running a 3-year-old Android is a real risk.
    - **App permissions**: Review which apps have access to your camera, microphone, location, contacts, photos. The bar should be: if I cannot remember why this app needs that permission, revoke it. You can always grant it again when the app actually needs it.
    - **Lock screen security**. A six-digit PIN is the absolute minimum. A long password or biometric is better.
    - **SMS-based 2FA**. Most banking and account-recovery flows still use SMS as a second factor. SMS is the weakest form of 2FA because of:
        - **SIM swapping** (an attacker convinces your carrier to port your number to their SIM). Where a service offers an authenticator app or hardware key alternative, use that instead.
        - The
    - **App inventory.** When did you last go through Settings → Apps and uninstall everything you no longer use? Each app you don't use is a permission grant you forgot you made and a piece of code that may not be getting updates.
    - **Find My / Find My Device**. Is it switched on? If your phone is lost, this is the difference between "remote-wipe in 30 seconds" and "everything is gone."
    - **Lockdown mode (iPhone)**. iOS 16+ has Lockdown Mode — a setting that disables many of the features attackers exploit (link previews, certain message types, complex web font rendering).

- Mobile is one of the four layers a defender has to think about: web, network, device, and the cloud underneath.

DAY-17 Cloud

- The cloud is just someone else's computer in someone else's data centre.
- The companies that dominate the cloud infrastructure:
    - AWS(Amazon Web Services): 30% of the global market. The original cloud launched in 2006. Powers Netflix, Reddit, Airbnb and much of the UK government and a long tail of startups
    - Microsoft Azure- 20% Tightly integrated with Microsoft 365, Windows, and the enterprise software that most large companies already run.
    - Google Cloud Platform(GCP)- 13% strong in data,AI and analytics workloads.
- For a SOC analyst, this matters because most of the systems you'll defend in your career will run on one of these three platforms.
- Knowing AWS in particular is close to a default expectation in modern security roles. You need to understand the basic vocabulary — EC2, S3, IAM, VPC — and what each one is, because they show up in every cloud security incident report you'll ever read.
- A typical web application uses several different cloud services that fit together. Here are the ones that show up most often in security incidents.
    - **Compute** (EC2 in AWS, Virtual Machines in Azure, Compute Engine in GCP). These are virtual computers — full operating systems running on shared hardware. When a developer says "spin up a server," they're creating an EC2 instance.
    - **Storage** (S3 in AWS, Blob Storage in Azure, Cloud Storage in GCP). Object storage — like a giant filesystem in the cloud. Your iCloud photos, your Netflix shows, your company's document backups. Most cloud

breaches involve storage that was misconfigured to be public when it should have been private.
        - The phrase "leaky S3 bucket" is so common in the industry that there's a famous database of them.
    - **Identity and Access Management** (IAM). The rules engine. Who is allowed to do what, on which resource. Every cloud action (read this file, start that server, send this email) runs through IAM. Most cloud breaches are not zero-days in the cloud platform itself. They're bad IAM configurations giving the wrong people, or the wrong code, too much access.
    - **Networks** (VPC, Virtual Networks, Subnets). Cloud-native versions of the network concepts.Day 14. Same ideas (public, private, firewalled) implemented in software inside the cloud provider.
    - **Databases** (RDS, DynamoDB, Cosmos DB, Cloud SQL). Managed databases. The provider runs the database software, you just use it.
    - **Serverless (**Lambda, Azure Functions, Cloud Run). Code that runs only when triggered. No server to manage. Pay per execution. Common in modern web architectures.
- When something goes wrong in the cloud, who is responsible — you, or the cloud provider?
    - Cloud providers operate under what's called a **Shared Responsibility Model**.
    - The provider is responsible for security of the cloud — the physical buildings, the hardware, the underlying virtualisation, the network backbone
    - You are responsible for security in the cloud — your data, your access controls, your application code, your configurations.
    - **The Capital One Breach(2019):** the customer (Capital One) had stacked together small misconfigurations: a WAF that allowed an SSRF attack, an

IAM role with far too many permissions, no monitoring on the metadata service, and no alerts when 30 GB of data was copied. The chain of small configuration errors created a path the attacker could walk. Paid $270million in fines and settlement.

- What does an analyst actually check on the cloud
    - **Public storage:** Are any S3 buckets, Azure Blob Containers, or Cloud Storage buckets accessible to the public internet that shouldn't be?Tools like AWS Trusted Advisor and Azure Security Center flag these automatically. They are still missed all the time.
    - **IAM(Identity and Access Management)permissions**: Does any user, role, or service have more permissions than it needs? The principle the industry calls **least privilege** is the single most important defensive concept in cloud security. A WAF doesn't need to read S3. A logging service doesn't need to delete users. A backup script doesn't need admin access.
    - **Logging and monitoring**. Is CloudTrail (AWS), Activity Log (Azure), or Cloud Audit Logs (GCP) switched on? These record every action taken in the account. Without them, you have no way to know what an attacker did.
    - **Multi-factor authentication on the root account**. The root account is the master key to the entire cloud setup. If it doesn't have MFA, the entire account is one phished password away from compromise.
    - **Default encryption.** Is data encrypted at rest by default? Modern cloud platforms make this easy, but it has to be switched on.
    - **Network exposure.** What ports are open to the public internet? An accidentally-exposed database (no password, port 3306 open to the world) is one of the most common embarrassing breaches in the industry.
    - **Secrets management**. Are passwords, API keys, and database credentials stored in proper secrets services (AWS Secrets Manager, Azure Key Vault), or hardcoded in the application code,

- Two things that an analyst checks all the time while accessing a cloud account is whether the root account has multifactor authentication enabled and whether any S3 buckets are publicly accessible.

DAY-18 How Attackers Actually Work

- Defending without understanding the attacker's process is shadowboxing. The frameworks today give you the structure that turns guesswork into method.
- The Cyberkill chain- seven stages of an attack
    1. **Reconnaissance**: The attackers research the target. LinkedIn, company website, job listings, public databases, leaked data. Who works there? What technology do they use? What email format does the company use? This stage is silent. The target has no idea it’s happening. Mostly there are no traces left.
    2. **Weaponisation:** The attacker builds the attack. A phishing email crafted for a specific person. A malicious document with an exploit embedded. A fake login page that mimics the target's real one. The weapon is custom-made for the target identified in stage 1.
    3. **Delivery.** The attacker delivers the weapon to the target. Email, USB drop, malicious website, SMS, supply chain. The most common delivery method by far is email, but cloud apps, social media, and physical access all count. This is where the target first encounters the attack.
    4. **Exploitation**. The weapon fires. The user clicks the link, opens the attachment, plugs in the USB. The exploit runs. The vulnerability triggers. The attacker now has a foothold inside the system. This is the moment of compromise.
    5. **Installation**. The attacker establishes persistence. They install malware, create a backdoor, schedule a task that re-launches their access on every reboot. The goal: survive a system restart. Without persistence, the attacker

loses access the moment the user closes their browser. With persistence, they are inside for weeks or months.

6. **Command and Control (C2)**. The compromised system phones home. It establishes a quiet, ongoing connection to the attacker's infrastructure, used to receive commands and send back stolen data. This is the puppet string between attacker and victim. C2 traffic is one of the things SOC analysts watch for most carefully.
7. **Actions on Objectives.** The attacker does what they came for. Steal data. Encrypt files for ransom. Move laterally to higher-value systems. Sabotage infrastructure. Drain a bank account. Exfiltrate credit card numbers. This is the only stage the public usually hears about, because it's the stage that produces visible damage

- The kill chain is the simple narrative model. Real attacks bend it, loop it, and run stages in parallel. The model is still useful as a thinking tool. Just don't expect the bad guys to obey it.
- The non-profit MITRE Corporation maintains a much more detailed framework called ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge).This is now the most dominant framework in cyberspace.
- ATT&CK organises attack behaviour at two levels
    - **Tactics- the why:** The attacker's goals at any given stage. ATT&CK has 14 tactics, including Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, and Impact
    - **Technique– the how.** The specific methods used to achieve each tactic. Over 200 documented techniques and counting. Each has a unique ID. Phishing is T1566. Exploiting a public-facing application is T1190. Lateral tool transfer is T1570.
- The framework is publicly available at `attack.mitre.org`. Bookmark it. You will return to it for the rest of your career.

- The frameworks teach defenders where to invest. Detection at every stage. The earlier you catch the chain, the less damage gets done.
- The kill chain isn't corporate. It runs at every scale, against every target. Maya is a target. You are a target. The defence is the same: break the chain at every link you can.

DAY-19 Social Engineering

- This is social engineering. It is the manipulation of people, not systems. And it is how the majority of successful attacks now begin.
- Social engineering is the single most common entry point for modern attacks. The 2024 Verizon Data Breach Investigations Report found the human element involved in 68% of all breaches. The technology can be perfect. The people are still human.
- This is the most common part of the stage 3(Delivery) in kill chain.
- Any attack technique that manipulates a person into doing something the attacker wants.
    - Reveal Information(a password, a verification code, the name of internal system)
    - Take an action(click on the link, open an attachment, transfer money, reset password)
    - Grant access(let someone in the building, allow remote access and change the billing detail)
- Social engineering is not a single technique. It is a category that describes any attack on the human, rather than the system. Modern security succeeds technically and fails socially. The category is dominant for that reason.
- The six universal triggers that are followed in every social engineering playbook
    1. **Authority:** People obey perceived authority figures with surprisingly little verification. The "IT department," the "head office," "head of finance," "the CEO," "a police officer," "your bank's fraud team."

2. **Urgency:** Time pressure breaks careful thinking. "Your account will be suspended in 24 hours unless you verify."The urgency does not have to be real. It only has to feel real long enough to make the target act before they think.
3. **Scarcity:** Limited availability triggers the fear of missing out. "Only three of these are in stock." "This offer expires today." Scarcity makes people commit before they evaluate.
4. **Reciprocity:** People feel obligated to return favors, even small ones. An attacker who offers help first, gives something away or appears to do you a service that creates a debt.
5. Social Proof: People follow the crowd. "Everyone in your team has already done this." "Three of your colleagues have already approved."If others appear to have complied, the target's resistance drops sharply.
6. Liking: People comply more readily with people they like. Attackers build rapport before the ask: small talk, shared backgrounds, friendly tone, names dropped that should be familiar.

- Six principles drive almost every social engineering attack: authority, urgency, scarcity, reciprocity, social proof, liking. Recognising the principles is the first defence. When two or three of them appear at once, slow down deliberately.
- The category has its own vocabulary. Each term names a different delivery channel for the same fundamental attack: manipulating a human.
    - **Phishing.** Email-based social engineering. The most common form.The attacker sends a message designed to manipulate the recipient into clicking, replying, or revealing.
    - **Spear phishing**. Targeted phishing. Crafted for a specific person, using their name, role, and context. Far higher success rate than generic phishing.
    - **Whaling.** Spear phishing aimed at executives. CFOs, CEOs, senior leaders. The reward of compromising a single executive justifies hours of preparation.

- **Vishing**. Voice phishing. Phone-based social engineering. The MGM attack was vishing. Live conversation amplifies authority and urgency in a way email cannot.
- **Smishing.** SMS phishing.The phone screen leaves less room to inspect a URL, and the urgency of an SMS feels different from email.
- **Baiting.** Leaving a tempting object for a target to pick up. The classic example: USB sticks dropped in a company car park. Curious employees plug them in. Studies have repeatedly shown success rates over 50% for this technique.
- **Pretexting.** Building a false scenario to justify a request. The MGM attacker's pretext was "I am Employee X, locked out, needing a reset." Pretexting is the connective tissue of most social engineering — it's the story that makes the request feel legitimate.
- **Tailgating.** Physical social engineering. Following an authorised person into a secure building, often by carrying a heavy box and looking apologetic. Buildings stay open more often than they should.
- **Business Email Compromise (BEC).** A specific high-value variant. The attacker compromises (or convincingly impersonates) a senior executive's email and emails finance to redirect a wire transfer.

DAY-20 Build Your Threat Model

- A threat Model is the working document of every serious security professional.
- A threat model is a structured document that answers four questions about a specific person, system or organization.
    - What am I trying to protect? (Assets)
    - Who would want to attack it, and why? (Threat actors)
    - How would they actually attack? (Attack scenarios)
    - What can I do about it? (defence and priorities)

- The reason threat models work is they force prioritisation.A threat model converts generic advice into a personal priority list.
- Checklists treat all users the same. Threat models start from the specific user and produce a tailored priority list. The discipline is what makes professional security work.
- A single document with five sections:
    - Asset inventory: What you're protecting, ranked by importance
    - Threat actor profile: who realistically targets people like you, and why
    - Attack scenarios: three concrete scenarios for your top three assets, mapped to the kill chain
    - Defence map: for each scenario, what currently protects you and what doesn't
    - Priority matrix and 90-day plan: likelihood vs impact grid, with three actions you'll take in the next 90 days

Networking

DAY-21 Networks are just Conversations

- A network is the conversation between two computers, and there are four parts in the conversation. A sender, receiver, common language, route between them.
- If any of the above four is broken or compromised, the conversion fails. Every network needs the exact same four things, just technical names:
    - **The sender**: has an address (IP address)
    - **The receiver:** has an address too(and a port,which says which application is listening.
    - **The common language** is a protocol(HTTP,HTTPS, SMTP, DNS, TCP, UDP)
    - **The route** is the path through the internet from sender to receiver, hopping through routers, ISPs, undersea cables, and finally to the destination

- An easy example would be, when you load a website, your browser becomes the sender. The web server is the receiver. HTTPS is the common language. The route runs through your router, your ISP, possibly several backbone networks, and into the data center where the server lives.
- Every single network attack you will ever encounter is an attack on one of those four parts.
    - **Attacks on the sender:** the attackers pretend to be someone they're not. They spoof an IP address. They impersonate a trusted device. ARP spoofing, IP spoofing, MAC spoofing.
    - **Attacks on the receiver:** the attackers send huge amounts of traffic to overwhelm the destination so legitimate sender can't get through. Denial of service. Distributed denial of service.
    - **Attacks on the language:** the attacker exploits flaws or insecure defaults in the protocol itself. DNS poisoning, TLS downgrade, HTTP request smuggling. The protocol behaves correctly according to its rules, but the rules themselves let bad things happen.
    - **Attacks on the route:** the attacker sits between sender and receiver and intercepts the conversation. Man-in-the-middle, evil twin Wi-Fi, BGP hijacking, malicious VPN.
- Four attack categories: spoof the sender, flood the receiver, exploit the language, intercept the route.

DAY-22 IP Addresses

- An IP address is a unique identifier assigned to a device on a network. The format you've seen most often looks like this: `192.168.1.5` or `8.8.8.8` or `185.27.134.42`. Four numbers, separated by dots, each between 0 and 255.
- That's called IPv4 (Internet Protocol version 4). Each of those four numbers is 8 bits, so the whole address is 32 bits. Total possible IPv4 addresses: about 4.3 billion.

- Every IPv4 address has two halves: the network part (which network the device is on) and the host part (which specific device on that network).
- **Public IP** addresses are unique on the entire internet. Every website, every cloud server, every device that wants to be directly reachable from outside has one.
- **Private IP** addresses are reused inside private networks (your home, your office, your phone hotspot). They're drawn from three reserved ranges, called RFC1918 ranges after the document that defined them in 1996:
    - `10.0.0.0` to `10.255.255.255` (used by larger networks and many corporate offices)
    - `172.16.0.0` to `172.31.255.255` (used by some routers and Docker containers)
    - `192.168.0.0` to `192.168.255.255` (the most common home-network range)
- Private addresses don't exist on the public internet. Routers won't carry packets addressed to private IPs, because they're not unique.
- **NAT (Network Address Translation)** is the trick that makes the modern internet work. When your laptop sends a packet to Google, your router does the following
    - Note that your laptop (`192.168.1.5`) wants to talk to Google.
    - Replaces your laptop's private address with the router's own public address before sending the packet out.
    - Remember the mapping so when Google replies, the router can rewrite the destination back to `192.168.1.5` and forward it to your laptop.
- NAT is why a single household with one public IP can have twenty devices on private IPs all using the internet at the same time.
- The proper fix to **IPv4 exhaustion is IPv6,** the next-generation Internet Protocol. IPv6 addresses are 128 bits long instead of 32.
- An IPv6 address looks like this: `2001:0db8:85a3:0000:0000:8a2e:0370:7334`. Eight groups of four hexadecimal digits, separated by colons.

- As of 2026, only about 45% of internet traffic uses IPv6. The rest is still IPv4. The reason is straightforward: NAT works well enough that there's no urgent reason to switch.
- You may have an IPv6 address right now and not know it. Most modern phones and laptops support both protocols (called dual-stack).
- What does the public IP usually reveals:
    - **Your country and region:** Always. IP-to-location is accurate to country level for almost every public IP.
    - **Your city or metropolitan area.** Usually. Accuracy varies by ISP and country, but most lookups place you within 50 km of your actual location.
    - **Your ISP.** Always. Whoever provides your internet connection (your home broadband provider, your mobile carrier, your office network, your VPN provider) is visible.
    - **Whether you're on a known VPN, proxy, or Tor exit node**. Usually. Many security services maintain databases of these.
- What does it not reveal
    - **Your name.** Never directly. Your name is only linkable to your IP via ISP's records, which requires a legal process to obtain.
    - **Your exact street address.** Almost never. The "city-level" geolocation is approximate at best.
    - **Your device** or what you're doing. The IP alone says nothing about you specifically.
- When you use a VPN, your traffic exits the VPN provider's server, so the destination sees the VPN's public IP, not yours.Your real ISP and approximate location become invisible to that server.
- Public IPs are how services build behavioural baselines for users. Logins from new IPs, new countries, or known anonymisation networks trigger alerts. Your IP is a routing identifier and a security signal at the same time.

    - A normal login: `2026-04-12 14:23:01 | login | maya@hershop.com | 81.139.42.18 | London, UK | BT Broadband`
        - Maya logs in from London, on BT Broadband, between 9am and 6pm UK time. That's baseline normal.
    - A malicious login: `2026-04-13 03:47:12 | login | maya@hershop.com | 185.220.101.4 | unknown | Tor exit node`
        - That's an alert. Login at 3:47am. Tor exit node. Different country signals. Either Maya is travelling and using Tor for some reason, or someone else is logging in with her credentials. The supplier's system flags it for verification.

DAY-23 Ports

- If the IP address is the building then ports are like the apartment numbers in that. The full address for a network conversation is always both: the IP gets you to the right building, the port gets you to the right door.
- A port is a 16-bit number (0 to 65535) that identifies which application on a device should receive a network packet.That’s about 65,000 possible doors per device. Most devices use far fewer than that at any given moment, but the range is large because every conversation needs its own port pair: one on the source side, one on the destination side.
- When your browser loads a webpage, the conversation looks like this:
    - Source: your laptop's IP, port 53219 (a random high number your OS picked)
    - Destination: the server's IP, port 443 (the standard port for HTTPS)
- The server replies back to the laptop’s IP at port 53219. Your laptop sees the incoming packets, looks at the destination port(53219), and packets are then handed.
- A handful of ports are so commonly used for specific services that the entire internet treats them as standard. They are called well-known ports.

    - 22 — SSH (secure shell, the standard remote-login protocol for servers)
    - 25 — SMTP (sending email between mail servers)
    - 53 — DNS (domain-name lookups, tomorrow's lesson)
    - 80 — HTTP (the unencrypted web)
    - 110 — POP3 (older email retrieval)
    - 143 — IMAP (modern email retrieval)
    - 443 — HTTPS (the encrypted web — most of what you use)
    - 587 — SMTP submission (sending email from a mail client)
    - 993 — IMAPS (encrypted IMAP)
    - 3306 — MySQL (database)
    - 3389 — RDP (Remote Desktop Protocol — Windows remote access)
    - 5432 — PostgreSQL (database)
    - 5900 — VNC (Virtual Network Computing — remote desktop).
- Ports 0-1023 are the well-known ones (HTTP, HTTPS, SSH, DNS, etc.). Ports 1024-49151 are registered for specific applications. Ports 49152-65535 are dynamic, used by your OS as random source ports for outgoing connections.
- Every port your device has open and listening is a potential entry point. An open port means an application is sitting there waiting for incoming connections.
- What ports are exposed? Is the first question that a security analyst asks about any system. The fewer open ports, the smaller the attack surface. Every additional open port adds risk that has to be justified.
- Three categories of open port matter:
    - **Listening on localhost only**(127.0.0.1). The service is running but only reachable from the same machine. No external risk
    - **Listening on your private network** (192.168.x.x). The service is reachable from other devices on your home or office network. Risk depends on who else is on that network. A printer with port 9100 open to home wifi is fine but not to a hotel Wifi.

    - **Listening on the public internet:** The service is reachable from anywhere on earth.This is where almost every “exposed service” data breach starts. Every open public port is announcing your existence to every scanner on the internet.
- Defensive principle: Every open port should answer four questions: why open?, what service?, is it patched?, who needs access?.
- Tools like Shodan continuously scan the entire internet for open ports and index what they find.
- Port scanning is one of the most common reconnaissance techniques. Before an attacker decides how to attack, they figure out what doors are open.
- A port scan systematically checks each port on a target device to see which ones respond.
- Nmap(**NetworkMapper**), free and open source, and used by every working pentester and SOC analyst.
- This is why port scanning is the very first step in offensive security.
- Almost all consumer internet traffic runs on port 443, The ports that matter for security are the ones exposed unintentionally- usually by a misconfigured home router or an internet-facing service nobody documented.

DAY-24 DNS

- It is the layer that, every time you type a website name, asks about fourteen different servers to send you to the right place.
- It is called DNS, which translates the well known names into IP addresses.
- The question asked is the DNS query. The answer is a DNS response. The whole system that handles that conversation, all over the world, is the Domain Name System.
- A typical DNS query involves the following steps:
    - Browser checks its cache
    - O.S checks its cache

    - If neither cache has the address, it will ask the recursive resolver, usually run by the ISP.
    - The recursive resolver also checks its own cache, and if it doesn't know it does the actual work of resolution: asking other servers in turn until it finds the answer.
- The hierarchy tree of the servers are as follows. With root at top, TLDs(top level domain: .com, .org, .uk, .io) and one step down is the authoritative servers for individual domains at bottom.
- The resolver roughly does this when its querying: "myfirsthack.com"
    - Asks the root server, and it replies with the IP address of the .com TLD server.
        - There are 13 root servers in the world, managed by 12 organisations
    - The .com TLD replies with the IP addresses of the servers
    - The authoritative server replies with the actual IP address.
    - The resolver then caches the IP address with TTL(time to live, saying how long the cache is valid for) and sends it back to the computer.
    - Your browser finally has an IP address.
- Every hop caches its answer. Every hop is also a trust point an attacker could target.
- DNS was designed in 1983, The protocol was designed for correctness and speed not for security. Three properties of DNS make it permanently attractive to attackers.
    - **Most queries are still unencrypted.** Standard DNS runs over UDP on port 53 in plain text. Anyone on the network path between your device and the resolver can read your queries (revealing every site you visit) and can modify the responses (sending you to a different IP).DNS-over-HTTPS (DoH) and DNS-over-TLS (DoT) fix this, but adoption is still partial.

    - **Caching makes attacks sticky.** A poisoned cache entry stays poisoned until the TTL expires, sometimes hours. One successful poisoning can affect every user of that resolver until the cached answer times out.
    - **Authentication is rare in practice.** DNSSEC (DNS Security Extensions) was designed to cryptographically sign DNS responses so attackers can't forge them. It's been around since 1997. It's still used by less than half of the internet's domains. The standard you're relying on does not, by default, prove that the answer you got is real.
- The result: DNS attacks are common, varied, and consistently effective. Three of the most important categories:
    - **DNS hijacking:** The attacker takes over the DNS records for a domain and points the name to an IP they control. Often done by compromising the registrar account, the authoritative server, or the resolver's upstream.
    - **DNS cache poisoning**: The attacker injects a malicious answer into a recursive resolver's cache, so every user of that resolver gets the wrong IP for the targeted domain until the cache expires.
    - **DNS rebinding:** A subtler attack where a malicious site exploits the fact that DNS answers can change quickly. After the initial page loads, the DNS record is updated to point to an internal IP (like a router admin panel), letting the malicious page reach inside the user's network.
- Domain hijacking via the registrar is one of the most common attack paths against small businesses, and it bypasses every technical defence at the website level. The attacker calls the registrar's support, claims to be the domain owner who has lost access, and uses social engineering to pass identity verification.
- Once the attacker has the registrar account, they can change DNS records to redirect the website to a fake copy, redirect email to themselves, and capture every customer payment and supplier conversation

DAY-25 TCP vs UDP

- TCP and UDP are the two ways a network conversation can be carried out. T The choice between them shapes performance, user experience, and which attacks each conversation is vulnerable to.
- TCP(Transmission Control Protocol): Careful, ordered, and reliable. It checks that every word arrived in order, and asks for a re-send when something goes missing. This is used in emails, web, banking and almost everything where correctness matters
- UDP(User Datagram Protocol): Fast, lightweight, and forgetful. It sends each packet without checking and if something gets lost, too bad, the conversation has already moved on. This is used in voice calls, video streaming, and DNS.
- TCP is for conversations where every byte must arrive correctly. UDP is for conversations where speed and recency matter more than perfection.
- Every TCP connection begins with a three way handshake:
    - The client side sends a **SYN(** synchronise) packet: saying that I want to start the conversation
    - The server replies with **SYN-ACK** packet: saying that I'm ready and I acknowledge the conversation.
    - The client replies with an **ACK** packet: saying, acknowledged, We're connected.
- Once the connection is open, every **TCP packet has a sequence number.** The receiver tracks which sequence number has arrived.
- When a packet is missing, the receiver sends a request for retransmission. The sender re-sends. Order is preserved, loss is detected and duplicates are removed.
- TCP also handles **flow control** (slowing the sender down when the receiver can't keep up) and **congestion control** (slowing both sides down when the network is overloaded).
- TCP stands for Transmission Control Protocol. It was designed in the 1970s by Vint Cerf and Bob Kahn at DARPA, alongside IP itself.

- What runs on TCP:
    - HTTP and HTTPS (port 80 and 443) — the web
    - SSH (port 22) — secure remote login
    - SMTP, IMAP, IMAPS — email
    - FTP and SFTP — file transfer
    - MySQL, PostgreSQL, MS SQL — most databases
    - TLS in general — encryption requires reliable delivery
- **UDP stands for User Datagram Protocol**. It was designed in 1980 — two years after TCP — specifically for cases where **TCP's reliability is too expensive.**
- There is no handshake. There are no sequence numbers. There are no acknowledgements. A UDP packet is sent, and either it arrives or it does not. The sender does not know which, and does not care.
    - **DNS (port 53)** — the lookup is one quick question, one quick answer. Setting up a TCP handshake first would double the time. If the answer is lost, your computer just asks again.
    - **NTP (port 123)** — clock synchronisation. Same logic.
    - **Voice calls (SIP, RTP)** — losing the occasional packet is fine. Pausing for retransmission is not.
    - **Live video streaming** — same logic. A small visual glitch is better than a stutter.
    - **Online games** — current state matters more than past state. If a player's position update is lost, the next update is already coming. Replaying the lost one would be useless.
    - **DHCP** — broadcasting "is anyone there?" on a fresh network. Doesn't need a handshake.

- UDP wins when the data has a short useful lifespan, when one missing packet is recoverable from the next packet, or when the cost of reliability would dominate the cost of the conversation itself.
- It is simpler because, there's no per-connection state to track, a UDP server can handle far more simultaneous "conversations" than a TCP server with the same hardware.
- UDP is the right choice for a high volume one-short interaction- exactly what DNS do all day.
- The security vulnerabilities:
    - **SYN Floods(TCP)**: The TCP handshake requires the server to remember every half-open connection until the client sends the final ACK. An attacker can exploit this by sending thousands of SYN packets and never sending the final ACK. The server holds open thousands of half-completed connections, each consuming memory and a slot in the connection table. Eventually the server runs out of slots and refuses new legitimate connections.
    - **UDP amplification (UDP).** Because UDP has no handshake, the source IP on a UDP packet is trivially spoofable. An attacker sends a small UDP query to a public server, but with the source IP forged to point at the victim. The server replies — to the victim. If the reply is much larger than the query, the attacker has used the public server as an amplifier.
    - **UDP floods (UDP)**. Simpler than amplification: just blast UDP packets at every port of a target. The target's OS spends CPU cycles checking whether any service is listening. No handshake means the attacker doesn't need to negotiate anything before starting the flood.
    - **Connection hijacking (TCP)**. If an attacker can predict a TCP sequence number and is on the network path, they can inject packets that look legitimate to both ends and steal the connection. Modern TCP defends

against this with randomised sequence numbers, but legacy systems and weak implementations are still vulnerable.

- In small business, use of DDoS protection for hosting (Shopify, AWS, Cloudflare, and similar) and a DNS provider with DDoS protection for her own domain (Cloudflare, Route 53, and Google Cloud DNS all qualify.
- **UDP amplification attack:** This happened in 2018, The attackers sent small queries to publicly exposed Memcached servers (a database caching system that runs on UDP port 11211) but forged the source IP to point at GitHub. Memcached responded — to GitHub — with payloads tens of thousands of times larger than the queries (the protocol's response was unbounded and mostly attacker-controlled).
    - With enough Memcached servers being exploited as amplifiers, the attackers turned a few megabits of attack bandwidth into 1.35 terabits of traffic at GitHub.
    - This is why UDP-based services that should not be reachable from the public internet (Memcached, NTP, DNS resolvers) keep showing up on Shodan as the source of new amplification incidents.
- This is the common list of protocols in wireshark.
    - TCP — most of your web traffic
    - TLSv1.3 or TLSv1.2 — encrypted web traffic running on top of TCP
    - UDP — DNS, video, voice
    - DNS — DNS queries (which run on UDP)
    - ARP — local network device discovery
    - HTTP — unencrypted web traffic (you should see very little)
    -

DAY-26 Mapping a Website's Footprint

- Reconnaissance is the first phase of every security engagement- They all start with the same question: what does the public internet already say about this target?

- The mapping is divided into 5 substeps:
    - Pick a target- a real website to investigate
    - DNS reconnaissance- what records does the domain publish?
    - WHOIS lookup- who owns the domain, when was it registered?
    - Certificate Transparency- what subdomains exist, what certificates have been issued?
    - Technology profiling- what's the site actually built on?
- Types of reconnaissance:
    - **Passive reconnaissance:** uses only publicly available information. DNS records, WHOIS data, certificate transparency logs, public archive sites, technology profilers that already crawled the target. Nothing you do touches the target's actual servers in a way they'd notice. You're reading public information about them — not probing them.
    - **Active reconnaissance:** sends traffic directly to the target. Port scans. Vulnerability scans. Aggressive crawling. Login attempts. Anything that the target's logs would show as coming from you.
- There are three different rules for reconnaissance:
    - Never run port scans or vulnerability scans against a target you don't own. Tools like "namp" and "nessus" are explicitly forbidden unless you have a written permission. In most countries, scanning a website you don't win is a criminal offence under computer misuse laws.
    - Never attempt to log in, submit forms, or interact with the site's authentication. Even if it's " just to see what happens".
    - If you are unsure about whether something is passive, it's active: stop! And ask. The cost of overcaution is zero.

DAY-27 Firewalls

- A firewall is a control that allows or blocks network traffic based on rules.
- There are four kinds of firewalls:

- **Packet Filtering Firewalls**: are the original, They look at each packet in isolation and decide based on a rule list. The fields that they check are the basic header information you learned across
    - Source IP- where it is coming from
    - Destination IP- where it's going
    - Destination Ports- which service it is trying to reach (Port 443 for HTTPS and port 22 for SSH)
    - Protocol: TCP, UDP, or ICMP
- Fast, cheap and simple, but because they don't track the state, they can be tricked- an attacker can craft packets that look like responses to connections your network never made, and a pure packet filter will let them through. These are found in home routers.
- **Stateful Firewalls**: They track the connection, which is a missing piece from above the firewall. They remember which conversations are active. Your laptop opens a connection to a web server on port 443, the firewall notes it, and when reply packets come back from that server they're allowed through because they're part of the conversation. The unsolicited inbound packets, on the other hand, get blocked. This solves the spoofed-packet problem. Cisco ASA, FortiGate, and Palo Alto's classic firewalls all sit in this category.
- **Proxy Firewalls (**application level gateways): Instead of forwarding traffic, they accept the connection themselves, then make a separate connection to the destination on behalf of the client. From the outside world, attackers only see the proxy- never the internal system. This adds security but introduces latency. Squid is a well-known web proxy that can act as a firewall. Cloud-based secure web gateways like Zscaler and Netskope work on this model.
- **Next-generation firewalls (NGFW):** They combine stateful inspection with deep packet inspection — looking at the actual data inside packets, not

just headers — and application awareness. An NGFW can distinguish "HTTPS traffic to Microsoft 365" from "HTTPS traffic to an unknown server" even though both use port 443, and apply different rules to each. Palo Alto's NGFW line, Fortinet's FortiGate, and Check Point are the major players. They cost more, consume more resources, and require more skilled administration, but they handle modern web traffic far better than older firewalls.

- Four firewall categories from simplest to most capable: packet-filtering (each packet alone), stateful (tracks conversations), proxy (acts as middleman), next-generation (inspects application content).
- **Firewall rules** are processed in an order from top to bottom. The first rule that matches a packet decides what happens to it, this is how a firewall rule might look:

  ```

  Rule 1: ALLOW from any to webserver on port 443/TCP

  Rule 2: ALLOW from any to webserver on port 80/TCP

  Rule 3: ALLOW from office_ip to webserver on port 22/TCP

  Rule 4: DENY from any to any on any port

  ```

- Rule 1 allows HTTPS to the webserver from anywhere. Rule 2 allows HTTP. Rule 3 allows SSH only from the office IP, Rule 4 is the catch-all deny -anything that didn’t match rules 1-3 gets blocked.

- The rule 4 is non-negotiable in mature configurations. It's called **default deny or implicit deny**, and it implements a fundamental security principle, only explicitly allowed traffic is permitted, everything else is blocked.
- The order of the rules is also very important, because if rule 3 is after rule 4 it is not gonna work, cause rule 4 will block everything and will not allow rule 3 to pass through.
- Two specific concepts, that are tested on cybersecurity exams:
    - **Whitelist:** Explicitly permit what's allowed, dent everything else. High security, and more administrative effort.
    - **Blacklist:** is the inverse, explicitly block known bad, allow everything else. Lower security, less administrative effort. Used for some specific cases (like blocking know malicious IP)
- Where does the firewalls sit in a network:
    - **Perimeter Firewall**: The classic placement: between the internal network and the internet. Every connection to or from the outside world passes through it.
    - **Internal Segmentation Firewall**: Sensitive systems — payment processing, database servers, executive workstations — sit behind their own internal firewalls, separated from the general office network. If an attacker compromises a marketing laptop, they shouldn't have unrestricted access to the customer database two hops away. Internal segmentation is the architectural choice that prevents lateral movement after initial compromise.
    - **Host-based firewalls.** Every modern operating system includes one. Windows Defender Firewall on Windows, pf or iptables/nftables on Linux and Mac, the built-in protections on iOS and Android. Host-based firewalls protect individual machines regardless of where they are on the network. Useful for laptops that leave the office (no perimeter firewall protects them

at a coffee shop) and for defence in depth (even if the perimeter is breached, host firewalls add another layer).

    - **Cloud firewalls.** AWS Security Groups, Azure Network Security Groups, GCP Firewall Rules. These operate at the cloud provider level, deciding what traffic can reach your virtual machines and services. They're stateful by default and rule-based like any other firewall, but they're managed through the cloud provider's interface rather than as a separate appliance.
    - **Web Application Firewalls (WAF)**: Where regular firewalls work at the network layer, WAFs work at the application layer, inspecting HTTP and HTTPS traffic for attack patterns like SQL injection, cross-site scripting, and OWASP Top 10 exploits. Cloudflare WAF, AWS WAF, and Imperva are the major players. Often deployed in front of web applications as an additional layer.
- Firewalls sit at multiple points: perimeter (network edge), internal segmentation (between zones), host-based (on each device), cloud (provider-managed), and WAF (application layer).
- **What firewalls don’t do:**
    - It doesn't stop traffic that’s been told to allow. If you allow HTTPS to your web server, and an attacker exploits a vulnerability in your web application by sending HTTPS requests, the firewall sees normal allowed traffic. This is the reason why web application firewalls exist, and WAF can’t stop all application layer attacks.
    - It also doesn’t stop the traffic that is allowed to leave.If an attacker is already inside your network (because they phished an employee, exploited an internal service, or were a malicious insider), the firewall happily allows their outbound connections to command-and-control servers.
    - A firewall doesn't inspect what's inside encrypted traffic, unless explicitly configured to do so (and even then, with significant cost and complexity). Most modern traffic is HTTPS-encrypted. A firewall can see the source,

destination, and port, but not the content. NGFWs and modern WAFs can do TLS inspection by terminating the connection at the firewall, decrypting, inspecting, and re-encrypting — but this requires installing trust certificates on every device and adds latency and complexity.

    - A firewall doesn't stop social engineering. The most successful attacks against companies in the past decade have largely come from phishing, not from network exploitation. The firewall was working perfectly.
    - A firewall doesn't stop insider threats. Someone who's already authorised to access systems can do harm regardless of firewall rules. The control that stops insider threats is access control and monitoring, not network filtering.
- **Defence in depth** (principle) uses multiple security controls at different layers, so that the failure of any single control does not result in compromise. Multiple firewalls at different points (perimeter, internal, host, application) is a textbook defence-in-depth pattern.

DAY-28 NAT (Network Address Translation)

- NAT lets many devices on a private network share one public IP address.The router keeps track of which device sent which request and routes the replies back to the right one.
- This is the duct tape that has been holding the internet for the last 30 years
- The IP address that the public internet sees is your router’s, not your device’s. Behind every home router, NAT is silently translating between many private addresses and one public address.
- IPv4 is 32 bits long, that gives roughly 4.3 billion possible addresses, and they all got allotted by 2011, and now each IPv4 address is a commodity for around $40-$60.
- Three ranges of IP addresses were reserved as "private" addresses, never to be used on the public internet:
    - `10.0.0.0` to `10.255.255.255` — about 16 million addresses

    - `172.16.0.0` to `172.31.255.255` — about 1 million addresses
    - `192.168.0.0` to `192.168.255.255` — about 65,000 addresses
- Any organisation can use these addresses internally, in unlimited combinations, because they never leak onto the public internet. Your home router's network is almost certainly '192.168.0.0/24' or 192.168.1.0/24'. The coffee shop down the road will also use the same addresses, and so as the office building across town.
- The router sits in the boundary, internally it has the same internal address from the range, but for outside it has a single public address allocated by the ISP. Every address on the inside network points to the router as its default route.
- How does the NAT actually translate:
    - Every time a device starts a new outbound connection, the router records an entry. It includes
        - The private source IP and port the device used
        - The destination IP and port (where the traffic is going)
        - A new source port the router picks for itself
    - The router substitutes its own public IP and that new port number in place of the device's private IP and port, then sends the packet out. When a reply comes back addressed to the router's public IP on that specific port, the router looks up the table, finds the matching entry, and translates the destination back to the original private IP and port.
- This format is called **PAT(Port address translation)**, or sometimes **NAPT(Network Address Port Translation)**, or **NAT overload**. This is what's running on your home router right now
- **static NAT** or **1-to-1 NAT**, used in enterprise networks. Each private address maps to a dedicated public address, one-for-one. No multiplexing. Used when servers behind NAT need to be reachable from outside on consistent addresses. Rare in home networks, common in datacentres.
- The security concerns in NAT:

    - **Inbound connections to private devices are blocked by default.** A device on the public internet can't initiate a connection to your laptop, because there's no NAT table entry mapping any inbound port to your laptop. The router doesn't know which internal device to forward the traffic to, so it drops the packet.
    - **This is not real security.** A device behind NAT can still initiate outbound connections to anywhere, and if it's compromised by malware that "calls home" to a command-and-control server, NAT does nothing to stop it.
    - **NAT complicates some legitimate use cases.** Running a game server, hosting a website from home, or using certain peer-to-peer protocols requires inbound connections — which NAT blocks by default.
    - **NAT obscures source addresses for logging and forensics**. A SOC analyst looking at firewall logs sees connections coming from a single public IP that might represent twenty or two hundred devices. Attributing malicious activity to a specific internal device requires correlating the NAT table at the time of the connection.
    - **Carrier-grade NAT compounds the problem**. Many mobile networks and some ISPs now run a second layer of NAT at their own boundary — multiple customers sharing a single public IPv4. This is invisible to users but means that the public IP you see for your mobile device is shared with thousands of other customers.
- IPv6 has enough addresses that NAT becomes unnecessary. The world is roughly half-way through the IPv6 transition. When fully on IPv6, every device gets a public address; firewalls do the work people used to attribute to NAT. NAT is a legacy of IPv4 scarcity, not a permanent feature of the internet.

DAY-29 VPN's

- A VPN creates an encrypted tunnel between your device and a server somewhere else. All your traffic gets wrapped in that tunnel before it touches the public internet.
- To anyone watching your local network, your traffic is encrypted noise heading to a single server. To the websites you visit, your traffic appears to come from that server, not from you.
- The traffic that used to be visible to your ISP and local network is now invisible to them — but it's now visible to your VPN provider instead. You haven't eliminated the watcher. You've changed who watches.
- There should be three pieces in place: a VPN client on the device, a VPN server somewhere on the internet, and an encrypted tunnel between them.
- When you turn the VPN on, your device establishes an encrypted connection to the VPN server. From that moment forward, every packet your device sends gets wrapped in a layer of encryption and routed through that tunnel to the VPN server. The VPN server unwraps the packet, sends it on to its real destination, and wraps the reply on the way back to you.
- Your router and ISP both see only that you're talking to your VPN server. They can't see what's inside the tunnel.
- In the perspective of the website the request came from a VPN server's IP- not from a private device.
- The encryption is typically AES-256(same family of encryption used to protect classified government data). The protocol that wraps and routes the traffic varies– OpenVPN, WireGuard, and IKEv2/IPSec are the three most often. WireGuard is the newest and currently the favoured choice for modern VPN services because it's faster and has a smaller, more auditable codebase.
- Four things VPNs actually do:
    - **They encrypt your traffic between your device and the VPN server.**This is the core feature. On public Wi-Fi, in a hotel, on an airline network, or

anywhere you don't trust the local infrastructure, the VPN prevents anyone on that local network from seeing what you're doing online.

- **They change what your ISP can see**. Your ISP can normally see every domain you visit, and in some countries they're required to log this information for years. With a VPN, your ISP sees that you're connected to a VPN server but nothing about what's inside the tunnel.
- **They let you appear to be in a different country.** Because websites see the VPN server's IP, you can effectively choose your apparent location by connecting to a VPN server in that country.
- **They let remote workers reach internal corporate networks securely.** This was actually the original use case. Before VPNs were a consumer product, they were how employees connected to office networks from home or while travelling.

- Five things that VPNs don’t do:
  - **A VPN does not make you anonymous**. Your VPN provider can see everything you do — every site you visit, every connection you make.
  - **A VPN does not protect you from malware**. If you download a virus, the VPN faithfully encrypts and transmits the virus to your machine.If a phishing page steals your password, the VPN faithfully encrypts that password as it leaves your device.
  - **A VPN does not protect you from yourself.** If you log into Google with the same account you always use, Google knows who you are. If you log into Facebook, Facebook knows who you are.A VPN changes your network identity but does nothing about your account identity.
  - **A VPN does not protect HTTPS traffic that was already protected.** This one trips people up. A VPN adds an extra encrypted layer on top, but the credit card was already protected. The realistic threat on public Wi-Fi isn't somebody reading your HTTPS — it's somebody seeing what domains you're visiting, which the VPN does hide.

    - **A VPN does not stop the websites you visit from tracking you.** Sites use cookies, fingerprinting (screen size, fonts, browser version, GPU), and behavioural patterns to identify returning visitors. None of these go through the VPN.
- a VPN is a network-layer privacy tool. It's effective against threats at the network layer — eavesdropping, traffic analysis, geographic restrictions. It's not effective against threats at the application layer, the account layer, or the human layer.
- When you should and shouldn’t use a VPN
    - **Use a VPN when you're on a network you don't trust.** Hotel Wi-Fi, airport Wi-Fi, conference Wi-Fi, the open network at a coffee shop.
    - **Use a VPN when you need to access a corporate network remotely.** Standard practice. Your employer provides the VPN client and you connect to their server.
    - **Use a VPN when you have a specific geographic restriction to bypass.** Streaming services that don't license certain content in your country.
    - **Use a VPN when your ISP is actively hostile.** In countries with state surveillance, ISP-level content blocking, or aggressive throttling of specific services, a VPN meaningfully changes who can see your traffic and what they can do about it.
    - **Don't use a VPN as a substitute for actual security practices.** Strong passwords, MFA, software updates, careful email behaviour, decent antivirus — these matter far more than whether your traffic is wrapped in a tunnel.
    - **Don't trust free VPN services with anything important.** Running VPN infrastructure costs real money. If a service is free, the business model is almost certainly selling your data, injecting ads, or worse.
    - **Don't believe marketing claims of "military-grade encryption" by themselves.** AES-256 is genuinely strong. But the strongest encryption in

the world doesn't matter if the provider keeps logs, has poor server security, or operates in a jurisdiction where they can be compelled to hand over data.

DAY-30 Proxies

- A proxy is a server that sits between you and the internet, intercepting requests for specific services (usually web traffic) and forwarding them on your behalf.
- Where VPNs are about wrapping everything in encryption for privacy, proxies are about routing specific traffic through a checkpoint — often for inspection, filtering, caching, or access control.
- VPNs are usually consumer-facing privacy tools. Proxies are usually enterprise-facing control and inspection tools.
- The three kinds of proxy you'll meet:
    - **Forward proxy:** Sits between users and the internet, on the user's side.The classic enterprise proxy. When an employee opens a browser at work, the request goes to the company's forward proxy, which forwards it to the public website, receives the reply, and returns it to the employee. Forward proxy is a checkpoint, as it can filter content, inspect traffic for security threats, cache popular pages so repeated don't all hit the public internet,and log every url every employee visits.
    - **Reverse Proxy**: Sits between the **internet and the server,** on the server's side. When you visit a website, what you think is the web server might be a reserve proxy that's accepting your request and forwarding it to the real web server hidden behind it.The reverse proxy can do TLS termination (handling the HTTPS encryption so the web server doesn't have to), load balancing (spreading traffic across multiple web servers), caching (serving popular pages without bothering the web server), and security inspection (filtering attacks before they reach the application). Nginx is the most popular open-source reverse proxy. Cloudflare runs the world's largest reverse proxy network

    - **Transparent proxy**: A forward proxy that the user doesn't know about. The network is configured so that web traffic is automatically routed through the proxy without the user's browser being configured to use one. Common in coffee shops, airports, schools, and hotels — your traffic gets quietly intercepted, possibly filtered, possibly logged, and then passed through to the real destination.
- What does proxies actually do:
    - **Content filtering:** The proxy maintains a list of categorised destinations such as gambling, social media, adult content, and known malware sites. Requests to anything on the blocked list gets rejected. Most corporate forward proxies do this. Schools networks aggressively do this.
    - **Security inspection**. Modern proxies, particularly cloud-based "secure web gateways" (Zscaler, Netskope, Cloudflare Gateway), inspect web traffic for malware, phishing pages, command-and-control communications, and data exfiltration patterns. They can do this even for HTTPS-encrypted traffic by performing TLS interception — terminating the connection at the proxy, decrypting, inspecting, then re-encrypting.
        - This requires a trust certificate on every endpoint, which is why corporate laptops are typically managed devices with the company's root certificate already trusted.
    - **Caching**. When many users in an organisation visit the same page, the proxy can keep a local copy and serve subsequent requests from there. Reduces bandwidth costs and speeds up page loads.
    - **Anonymisation**: Open public proxies exist that anyone can route traffic through to hide their source IP. Some are legitimate (free academic proxies, Tor exit nodes), most are run by people you'd rather not trust with your traffic. The use case overlaps with VPNs, but proxies typically don't encrypt the traffic between you and the proxy — meaning anyone watching that connection can see what you're doing.

    - **Load balancing.** Reverse proxies distribute incoming traffic across multiple backend servers, so no single server gets overwhelmed. Almost every large web application has this. Nginx, HAProxy, and AWS Application Load Balancer are all doing this work whether or not their users notice.
    - **TLS termination**. Reverse proxies often handle the HTTPS encryption themselves, so the backend application servers can speak plain HTTP internally. Centralises certificate management and offloads computation from the application. Cloudflare does this at planetary scale.
- Proxies raise a sharper version of trust questions than what it is compared to VPN's:
    - **Your employer's forward proxy can see every site you visit at work**:including HTTPS traffic if they've installed TLS interception (which most large enterprises now do). The certificate warnings that would normally show up when traffic is being intercepted don't appear because the company's root certificate is trusted on the company-managed laptop.
    - **Public Wi-Fi networks often run transparent proxies.** That free coffee shop Wi-Fi is filtering or inspecting your traffic, whether or not anyone warned you. The legitimate version filters known-malicious sites and limits bandwidth-heavy applications. The illegitimate version injects ads into your HTTP pages, logs your activity for sale to advertisers, and occasionally serves malware.
    - **Open public proxies are almost always traps.** Lists of "free anonymous proxies" exist online. Most of them are run by people harvesting credentials and session cookies. The handful of legitimate ones are usually saturated and slow. Paid VPNs are way better options than these.
    - **The reverse proxy you visit may or may not be the website you think it is.** When a site is behind Cloudflare, Akamai, Fastly, or AWS CloudFront, you're talking to one of those companies' servers, not the actual website.

They see every request you make. That's not necessarily bad — they're large companies with reputations to maintain and security programmes that small site operators couldn't match alone. For example if you are visiting example.com , you are talking to the CDN(Content delivery network, often the proxies to distribute the content world wide) that’s talking to example.com on behalf.

- VPNs and proxies overlap in technique but differ in scope, encryption, deployment, and intent. VPNs wrap everything in encryption for the user's privacy. Proxies typically intercept specific application traffic for an organisation's control. You can use them together; they can also work against each other (corporate proxies versus personal VPNs is a common policy battle).

DAY-31 Wireshark

- Wireshark is a packet analyser. It captures the actual network traffic flowing through your computer's network interface and shows it to you, packet by packet, field by field.
- When wireshark runs it puts your network interface into a mode where it copies every packet that passes through it and hands a copy to you for inspection.
- Each captured packet is displayed in three panes, and understanding these three panes is most of what you need:
    - **The packet list:**at the top shows one row per packet: a number, a timestamp, the source address, the destination address, the protocol Wireshark identified, the length, and a brief summary.
    - The **packet details** pane in the middle shows the selected packet broken down by protocol layer. You can expand each layer to see its fields. Select a packet and you’ll see the ethernet frame, then the IP header, the TCP or UDP headers with ports, the sequence numbers and flags, then the application data on top.

    - The packet bytes pane at the bottom shows the raw packet as hexadecimal and ASCII. This is the actual data on the wire, every byte. You won't read this directly very often.
- The single most important skill in Wireshark is filtering, narrowing thousands of packets down to the handful you actually care about. wireshark has the display filter on the top. A few filter options that cover most early use are:
    - **Filter by protocol**: dns, http, tcp, arp(address resolution), you can filter based on the protocol name, in lowercase to see those requests/traffic logs.
    - **Filter by IP address**: Type `ip.addr == 8.8.8.8` to see only traffic to or from that address. Useful when you want to isolate the conversation with one specific server.
    - **Filter by port**. Type `tcp.port 443` to see only HTTPS traffic. Type `udp.port 53` to see only DNS over UDP.
    - **Combine them.** Type `dns && ip.addr == 192.168.1.1` to see only DNS traffic involving your router. The `&&` means "and"; `||` means "or".
- The packets on Wireshark are color-coded by default- green for HTTP, light blue for DNS, black or red for packets with problems, and so on.
- Capture your own traffic freely. Don't capture others' traffic on networks you don't control — it can violate wiretapping and computer-misuse laws. Corporate networks require authorisation. Encrypted traffic stays encrypted: Wireshark shows the envelope, not the contents, which is itself a lesson in the value of metadata.
- Things that you can recognise:
    - DNS queries and responses. A query goes out asking for the IP address of a domain, an answer comes back with the address.
    - TCP three-way handshake. When your computer opens a connection, you'll see the SYN packet, the SYN-ACK back, and the ACK that completes it
        - tcp.flags.syn == 1` to find the start of every connection.

- You'll see ports in action. Every TCP and UDP packet shows source and destination ports. You'll watch your browser connect to port 443 on web servers, your computer query port 53 for DNS.
- the layered model. Click any packet, expand the details pane, and the OSI, starting with the Ethernet, then IP, then TCP or UDP, then the application protocol on top.
- You'll also see how much your computer talks without being asked.

DAY-32 Reading Packets

- Recognising packets is knowing what a SYN packet is. Reading packets is seeing a SYN with no matching SYN-ACK and knowing that means the connection was refused or the host is down. The difference is context — understanding each packet in relation to the conversation it belongs to. That contextual reading is the actual job.
- The most useful reading skill is following one TCP conversation through its entire lifecycle. A TCP connection has a clear beginning, middle and end, and reading all these would tell the story behind all the packets.
    - **The opening: the three way handshake**: The client sends SYN ("let's talk, here's my starting sequence number"). The server replies SYN-ACK ("agreed, here's mine, and I acknowledge yours"). The client sends ACK ("acknowledged, we're connected"). Three packets, and the conversation is established. When you see this complete sequence, you know a real connection formed.
    - **After the handshake, data flows**: You'll see packets carrying payload in both directions, each one acknowledged by the other side. For an HTTPS connection, the first thing after the handshake is the TLS negotiation — the client and server agreeing on encryption — after which the actual data is scrambled. You'll see the packets, their sizes, their timing, and their direction, even though the contents are encrypted.

    - **The closing: the teardown**. TCP connections close politely. One side sends FIN ("I'm done sending"), the other ACKs it, then sends its own FIN, which gets ACKed. A four-packet goodbye. Sometimes you'll see RST (reset) instead — an abrupt termination, the network equivalent of hanging up mid-sentence. RST is normal in some cases (a service refusing a connection) and a red flag in others (an injected reset attacking a connection). Learning to notice RST is learning to notice when conversations end badly.
- Right-clicking then Follow TCP Stream isolates one whole conversation from the noise. Reading the lifecycle turns packets into a story.
- What does the normal traffic look like:
    - **DNS comes first and then the connection**: Almost every connection starts with a DNS lookup, the machine asks for the IP of a domain, gets an answer, then opens a TCP connection to that IP. If you see a connection to an IP with no preceding DNS lookup that’s worth a glance- it can be normal(a cache lookup, a hard coded IP) or it can be a sign of something talking to an address it didn’t look up in a friendly way.
    - **Connection complete and close cleanly**: Normal connection handshake fully, transfer data, and tear down with FIN. Lots of half open connections(SYN with no SYC-ACK) or lots of resets (RST) suggests something is wrong- a service down , a misconfiguration, or in some cases scanning or attacking activity.
    - **Most traffic is to recognisable destinations**. Browse normally and you'll see traffic to your ISP's DNS, to the websites you visit, to the cloud services your apps use (Apple, Google, Microsoft, your password manager, your messaging apps). These are familiar names. Traffic to an IP that resolves to nothing, or to a country your device has no reason to talk to, stands out against that backdrop.

    - **There's a lot of background chatter, and it has a rhythm.** Your device constantly checks for updates, syncs time over NTP, renews DHCP leases, and announces itself on the local network with protocols like mDNS and ARP. This background traffic is repetitive and predictable. Once you've watched it a few times, you stop noticing it — which means anything that isn't it becomes noticeable.
    - **Encrypted traffic dominates**. In 2026, most of what you capture is TLS-encrypted HTTPS on port 443. Seeing lots of 443 is normal. Seeing significant traffic on unusual ports, or plaintext protocols where you'd expect encryption, is worth a look.
- A handful of fields carry the most meaning, these are the fields that you need to look at.
    - **TCP flags** are the single most informative field for reading a conversation. SYN starts a connection. ACK acknowledges received data. FIN politely ends it. RST abruptly kills it. PSH says "deliver this data to the application now." The flags are the verbs of the TCP conversation — they tell you what each packet is trying to do.
    - **Source and destination ports tell you what service is involved.** Destination port 443 means HTTPS, 53 means DNS, 22 means SSH, 3389 means RDP. Connection to a random port can be perfectly normal(the random source ports clients use) or worth a look.
    - **Time matters more than beginners expect**. The timestamp column shows when each packet arrived. Reading the timing tells you things the contents can't — a connection that sends a packet every thirty seconds exactly looks automated, not human. A burst of hundreds of connection attempts in a second looks like scanning. Regular, precise intervals are a signature of machine behaviour, and machine behaviour at odd times is often worth investigating.

    - **Length shows the packet size.** A stream of large packets is bulk data transfer (a download, a video). A stream of tiny packets at regular intervals can be a keepalive or, in a security context, a possible command-and-control beacon.
    - **The info column** is Wireshark's plain-language summary of each packet. For many protocols it's remarkably readable: a DNS packet's info column literally says what domain was queried; an HTTP packet says the request method and path.
- The security aspect of the packet reading:
    - **Spotting a port scan.** A single source sending SYN packets to many different ports on the same target, in quick succession, most getting RST back. The pattern is unmistakable once you've seen it — a fan of connection attempts, almost all refused. Small businesses get this all the time, their ports are scanned for constantly by automated tools probing for open ports, in a capture from her server you'd see this pattern form many sources daily.
    - **Spotting suspicious DNS**. A device making DNS queries for long, random-looking domain names, or an unusually high volume of DNS traffic, can indicate malware using DNS to communicate with its operators
    - **Spotting a beacon**. Malware that's compromised a machine often "phones home" at regular intervals to receive instructions. In a capture this looks like small connections to the same external IP at suspiciously precise timing — every sixty seconds, every five minutes — far more regular than human-generated traffic. The rhythm gives it away.
    - **Spotting plaintext where there shouldn't be**. Credentials or sensitive data travelling in unencrypted protocols. If you can read the contents of a packet that should be protected, that's a finding — it means the data is exposed to anyone capturing the traffic.

DAY-33 Network Attacks

- The goal is to recognize malicious traffic.
- **Man in middle attack**: the idea is the attacker positions themselves in the middle of the conversation, so all the traffic passes through them. They can read it and depending on the situation they can alter it also.
- Once the attacker is in the middle of a conversation they can harvest the credentials as they pass, they can inject malicious content into the web pages, they can downgrade the secure connection to an insecure one.
- The defence is encryption, in HTTPS there is TLS that encrypts the connection end to end and verifies the server's identity with certification, this is the reason you should never ignore the browser warning when it is triggered.
- ARP poisoning and DNS spoofing — are both, at heart, techniques for achieving the man-in-the-middle position in the first place.
- **ARP poisoning or ARP spoofing** lying about who's on the local network. When your laptop wants to send data to another device on the same network — say, the router — it needs that device's hardware (MAC) address. It finds it using ARP.
- ARP(Address Resolution protocol) has no verification — whoever answers "who has this IP" is believed. ARP poisoning sends forged replies to redirect a victim's traffic through the attacker, achieving man-in-the-middle.
- It's visible in a capture (unrequested ARP replies, duplicate IPs, changing MACs) to an analyst who's watching. It only works from inside the local network, which is why local network security and segmentation matter. Encryption limits the damage.
- **DNS spoofing or DNS cache poisoning** falsifies the answer to "what IP does this name point to," redirecting victims to attacker-controlled servers while the address bar still shows the right name. It's often achieved from a man-in-the-middle position.
- DNSSEC adds cryptographic signatures to DNS records so forged answers can be detected, though adoption is incomplete. Encrypted DNS — DNS over HTTPS

(DoH) and DNS over TLS (DoT) — protects the DNS query itself from being read or tampered with in transit, which closes the local-network version of the attack.

- HTTPS certificate validation as a backstop. The layers reinforce each other.
- **Denial of service:** Instead of intercepting or redistricting traffic, a denial of service DoS attack overwhelms a target with so much traffic that it can’t serve legitimate users.
- The **distributed version** (DDoS) uses many compromised machines — a botnet — to generate the flood from thousands of sources at once, which makes it both more powerful and harder to block.
- a classic DoS technique floods a server with SYN packets and never completes the handshakes, exhausting the server's capacity to track half-open connections
- The defence is largely about scale and filtering: services like Cloudflare absorb DDoS traffic across enormous networks, and reverse proxies sit in front of origin servers partly for this reason.
- **Packet sniffing:** A passive attack, Simply capturing traffic that isn't yours to read.On older networks and unencrypted protocols, sniffing could read credentials and data directly off the wire.
- The widespread move to encryption is the reason sniffing is far less fruitful than it once was: a sniffer on a modern network mostly sees encrypted traffic. This is, once more, why encryption everywhere matters.
- DNS spoofing falsifies the answer to 'what IP does this name point to.' The user types or clicks the correct domain name: The domain name is unchanged; the underlying IP address that name resolves has been falsified.

DAY-34 Network Segmentation

- Network segmentation — splitting a network into separate zones so that getting into one doesn't mean getting into all of them.
- The core idea is containment, and In an unsegmented network, every device can talk to every other device.

- In a segmented network the compromised is trapped in a zone.The attacker has to break through additional barriers to move further, and every barrier is a chance to detect and stop them.
- Lateral movement, attackers do not land on the target directly, they land somewhere easy and then move their way towards the valuable things.
- Segmentation breaks lateral movement by putting barriers between zones, so an attacker who gets in can't freely travel to the crown jewels. It buys time and creates detection opportunities.
- How network segmentation actually happens:
    - **Subnets**: The most basic division. A network is split into separate IP ranges (subnets), each its own logical segment.Devices in different subnets can only talk to each other through a router, which becomes a natural control point. Putting marketing on one subnet and finance on another is the first step toward separating them.
    - **Virtual LANs** let one physical network be divided into multiple logical networks. Without VLANs, separating departments would mean running separate physical cabling for each — expensive and inflexible.VLANs achieve the separation in the switch's configuration instead: the same physical switch carries multiple isolated networks, and devices on different VLANs can't see each other directly even if plugged into the same switch. This is how most business networks segment in practice.
    - **Firewalls between zones**. You met internal segmentation firewalls before. Between each segment sits a firewall that controls what traffic may cross.
    - **The DMZ.** A specific, important pattern: the demilitarised zone. Servers that must be reachable from the public internet — a company's web server, its mail server — are placed in a separate segment between the internet and the internal network. If one of those public-facing servers is compromised (and public-facing servers are the most exposed), the attacker is stuck in the DMZ, not inside the internal network.

    - **Guest networks.** The version you have at home. Your router's guest Wi-Fi is segmentation in miniature — guests get internet access but are walled off from your main network and the devices on it.
- Segmentation matters most for insecure devices (IoT — the Mirai botnet weaponised unsegmented devices), critical systems (OT in industry, where IT/OT separation stops office breaches reaching physical machinery), and small businesses like Maya's shop (where payment systems, cameras, and customer Wi-Fi should never share one flat network). The fix is the same idea at every scale: separate what doesn't need to talk.
- The principle under all segmentation is least privilege applied to networks: every zone gets only the connectivity its function needs, and nothing more — the network-level version of default deny.
- Taken to its conclusion this becomes zero trust (trust nothing by default, verify everything),
- Segmentation is the foundational, practical expression: give each zone only the reach it needs, deny the rest, contain the breach.

DAY-35 Investigating Real Traffic

- The workflow used in an investigation as an analyst:
    - **Orient first**: Get a feel for the capture as a whole. How many packets? Over what time span? The statistics menu in wireshare is built for this reason, it gives the Protocol Hierarchy of which protocols are present, and conversations show you who talked to whom and how much.
    - **From a question**: Why is there so much DNS? Who is this one host talking to so much? What's this protocol I don't recognise? Why are there hundreds of connection attempts from one address? A good investigation is question driven not to wander with the flow.
    - **Filter the thread**: Use display filter to narrow to the traffic that answers your question. Filter one host, one protocol, one conversation.

    - **Follow and read:** Once narrowed, read the conversation. Follow a TCP stream. Read the DNS queries. Look at the timing and the sizes. Build an understanding of what actually happened.
    - **Compared to normal**: Everything you learned about baseline and attacks come in here. Is this normal or not, Does this pattern match with anything you recognise- a scan, a beacon, a flood.
    - **Document**. Write it down as you go, then pull into a report. The finding isn't real until it's written in a way someone else can verify and act on.
- The statistics menu on Wireshark:
    - **Protocol Hierarchy**: shows every protocol in the capture as a tree, with the percentage and packet count of each
        - Mostly HTTP and DNS means ordinary web traffic, a lot of ARP might mean local network activity worth examining; an unusual protocol you don't recognize is an immediate threat to pull.
    - **Conversation:** lists every pair of addresses that communicate, with how many packets and bytes flowed between them. Sort by bytes and the heavy talker rises to the top– often the most interesting hosts. A single external IP where one internal host exchanges a lot of data is worth a look.A host talking to hundreds of different addresses might be scanning or might be a busy server.
    - **Endpoints:** lists every individual address and its total traffic, and for IP addresses Wireshark can map them to countries. An endpoint in a country the organisation has no dealings with is the kind of thing that draws an analyst's eye. It's not proof of anything on its own, but it's a thread.
    - **IO Graph:** plots traffic volume over time, which makes bursts and regular beacon-like patterns visible at a glance — a flat line with a spike is a very different story from a perfectly regular sawtooth.
- Be honest about what you find and how sure you are. "Normal traffic, nothing suspicious" is a complete and correct finding — not every capture has an attack.

- When you do find something, describe it precisely (what, which packets, why it matches a pattern). State confidence honestly: "definitely," "looks like but uncertain," and "unusual and unexplained" are all valuable. Avoid the trap of seeing attacks everywhere; most unusual traffic has a boring explanation.
- Traffic investigation is foundational across security careers: SOC analysts do it every shift (alert triage), incident responders use captures as breach evidence, network engineers use the same workflow for reliability, and penetration testers read traffic both to understand targets and verify attacks.

DAY-36 IDS and IPS

- There should be something that watches over millions and billions of logs per day in an organisation of a big size, here comes the Intrusion detection system (IDS) and Intrusion prevention system (IPS).
- These are the systems that watch network traffic continuously and automatically flag - or block- the suspicious patterns.
- An IDS watches network traffic and alerts when it sees something suspicious. An IPS does the same but can also block the traffic in real time.
- The difference between them is the core understanding you can get:
    - **An IDS sits off to the side, watching a copy of the traffic:** Network traffic flows normally; the IDS receives a duplicate of it (often through a switch feature called a mirror or SPAN port, or a hardware tap) and inspects that copy.
        - Because it is actively watching a copy it cannot stop anything malicious, it can only alert, and log the event for the analyst to go over it about the incidents when they occur.
    - **An IPS sits inline, in the actual path of the traffic:** Every packet passes through the IPS on its way to the destination like a firewall. Because the traffic goes through it, IPS and act- when it detects something malicious, it

can drop the packet, reset the connection, or block the source, in real time, before the traffic reaches its target.

- IPS is the most powerful as it stops the attackers rather than reporting them, but there is also a risk associated with that. As it's inline, an IPS becomes a potential point of failure: if it goes down, it can take the network connection down with it.
- The IPS block the false positive and the wrongly identified genuine traffic is also blocked for legitimate users, real business and customers. But in IDS it only created a wasted alert.
- Many organisations run both: IPS blocking obvious bad traffic(on the perimeter), IDS (internally) feeding subtler alerts to human analysts.
- The fundamental approaches used to decide whether the traffic is real or malicious are as follows:
    - **Signature-based detection works like antivirus.** The system holds a database of known attack patterns — signatures — and compares traffic against them. A signature might describe the exact byte pattern of a known exploit, the structure of a particular malware's beacon, or the request pattern of a known scanning tool.
        - When traffic matches a signature, the system flags it. This is fast, precise, and produces few false positives, because it only fires on patterns known to be malicious.
        - Its fatal weakness: it can only catch what it already knows. A brand-new attack with no signature yet — a zero-day — sails straight through. Signature databases need constant updating, exactly like antivirus definitions, and there's always a window between a new attack appearing and a signature existing for it.
    - **Anomaly-based detection:** Instead of matching known-bad patterns, it learns what normal looks like for a given network and flags deviations from it. If a server that normally talks to ten addresses suddenly talks to ten thousand, if traffic spikes at 3am when the office is empty, if a host starts

using a protocol it has never used before — anomaly detection notices, even without a specific signature for whatever's happening.

    - Its strength is catching the unknown: novel attacks, insider threats, things no signature describes.
    - Its weakness is false positives. Networks are noisy and legitimately variable; "different from normal" is not the same as "malicious," and anomaly systems can drown analysts in alerts about benign unusual activity.
    - There should be a learning period to establish the baseline, and a baseline learned during a period that already contained an attack will treat that attack as normal.

- Modern systems combine both. No detector is perfect, which is why human analysts remain essential.
- The defining challenge of intrusion detection isn't catching attacks — it's not flagging everything else. A sensitive IDS can generate tens of thousands of mostly-benign alerts daily, burying the real attack in noise.
- This "alert fatigue" is why detection systems are carefully tuned, why alerts are prioritised, and why they feed into a SIEM to aggregate and correlate them.
- Real tools to know: Snort (the classic, defined the category), Suricata (modern, high-performance), Zeek (focused on rich logging).
- IDS/IPS capability is increasingly built into next-generation firewalls rather than being separate appliances. A modern firewall from Palo Alto, Fortinet, or Cisco includes IPS functionality alongside its filtering.
- They also run dedicated network detection and response (NDR) platforms that does the same functionality as IDS/IPS
- Host-based versions (HIDS) watch individual machines, complementing network-based detection.
- Small businesses have their own firewalls and routers protection instead of running these high end tools.

DAY-37 SIEMs (Security Information and Event Management)

- It is the central nervous system of a security operations center. Everything that generates security relevant data- the IDS and IPS, firewalls, servers, endpoints, cloud services, applications- sends its logs and alerts to the SIEM.
- The SIEM collects all of it in one place, makes it searchable, correlates events across different sources, and surfaces what matters.
- A SIEM performs a handful of distinct functions, and understanding them in order shows how raw data becomes an actionable incident.
    - **Collection:** The SIEM ingests data from across the organisation. Every system that produces security-relevant logs — firewalls, IDS/IPS, servers, workstations, cloud platforms, applications, authentication systems — forwards its logs to the SIEM.
    - **Normalisation:** every system log has its own format. A windows server, a Cisco Firewall, a linux machine, and a cloud service all describe even a simple event like a login in completely different ways- different field names, different date formats, different structures.The SIEM normalises this, translating the chaos of formats into a common structure so that a login is a login regardless of which system reported it.
    - **Correlation.** This is the heart of the SIEM and the reason it earns its place. Correlation rules look for patterns across multiple events and multiple sources that together indicate something a single event wouldn't.The SIEM's correlation engine watches for these combinations and raises a single high-value alert from many low-value events. Correlation is what turns the flood of raw data into the handful of things actually worth an analyst's attention.
    - **Alerting and dashboards**. When correlation rules fire, the SIEM alerts — and crucially, it prioritises, so the most serious correlated incidents rise to the top rather than being one more line in a list of thousands. Dashboards

give analysts a live view of the security posture: what's firing, what's trending, where to look.

    - **Search and investigation.** When an analyst investigates — or when an incident responder reconstructs an attack after the fact — the SIEM's searchable archive of normalised events is the primary tool. Being able to ask "show me everything this user account did across every system in the last 24 hours" and get an answer in seconds, drawn from dozens of sources, is the investigative superpower the SIEM provides.

- Correlation: the power and the worked example: Take a single failed login. On its own, it means almost nothing — people mistype passwords constantly. A SIEM that alerts on every failed login would be useless noise.
    - **Brute force, detected by counting.** Fifty failed logins against one account in two minutes, then a success. No single failure mattered; the pattern is an account being brute-forced and then breached. A correlation rule counting failures over time catches this.
    - **Credential stuffing, detected across accounts.** One source IP attempting logins against hundreds of different accounts, a few succeeding. Each individual attempt is unremarkable; the spread across many accounts from one source is the signature of credential stuffing using a breached password list. Correlation across accounts catches what per-account monitoring misses.
    - **Impossible travel, detected by combining location and time**. A user logs in successfully from London, then ten minutes later logs in successfully from Singapore. Each login alone is fine — valid credentials, successful authentication. Combined, they're impossible: no one travels from London to Singapore in ten minutes, so one of those logins is an attacker using stolen credentials. This correlation — combining geographic data with timing — is a classic SIEM detection and catches account compromise that no single login event could reveal.

    - **The full attack chain:** A phishing email delivered (from the mail security logs), followed by a login from a new device (from the authentication logs), followed by internal network scanning (from the IDS, yesterday's topic), followed by access to a sensitive database (from the database logs), followed by a large outbound transfer (from the firewall logs). Five systems, five logs, one story — a breach unfolding. The SIEM correlates them into a single incident an analyst can see whole, rather than five disconnected alerts in five different systems that no one ever connects.
- The limits and the cost:
    - **A SIEM is only as good as what feeds it.** If a system isn't sending its logs to the SIEM, the SIEM is blind to that system entirely. A common and dangerous gap is a critical system whose logs were never connected, or were connected once and silently stopped.
    - **A SIEM is only as good as its correlation rules**. Correlation doesn't happen by itself — someone has to write the rules that define what patterns matter. A SIEM with poor or default-only rules misses things a well-tuned one catches.
    - **A SIEM still produces false positives — and tuning them is the daily grind.** A poorly tuned SIEM floods analysts with low-value alerts, recreating the alert-fatigue problem one level up. The major part of a SOC is fine tuning the rules, reducing the noise so the real signals stand out.
    - **SIEMs are expensive and complex.** Enterprise SIEMs are among the costlier security investments an organisation makes — licensing often scales with the volume of data ingested, which can make comprehensive logging genuinely expensive, and the platforms require skilled people to run
- Major SIEM platforms to know: Splunk (dominant, powerful, expensive), Microsoft Sentinel (cloud-native), Elastic Security/ELK (affordable, learner-runnable), QRadar and Chronicle (enterprise).

- SOAR (Security Orchestration, Automation, and Response) is the evolution — automating the response to what the SIEM detects, the same detect-versus-act distinction as IDS/IPS at the incident level.
- Small businesses like Maya's get SIEM capability through their cloud providers and, as they grow, through managed detection and response (MDR) services rather than running their own.

DAY-38 Logs

- A log is simply a record of something that happened — a login, a connection, a file opened, an error thrown
- Everything in security is built on the logs, they are text records of routine events, most of which never matter.
- But when something goes wrong- a breech, an outrage, a fraud- the logs are very often the only record of what actually happened.
- The key decisions (what to log, how long to keep it, how to protect it) are made before an incident, because you can't investigate what you didn't log.
- The handful of logs that do most of the work in security are:
    - **Authentication Logs**: Record who logged in, when, from where, and whether it succeeded or failed. They are the most important because many attacks such as brute force, credential stuffing, and impossible travel patterns all live in authentication logs.
    - **Network logs** record connections — what talked to what, when, on which ports, how much data. They answer "what communicated with what," which is central to tracing how an attacker moved or where data went.
        - Eg: NAT logs which internal device has made the connection useless the NAT table was logged- you cannot investigate further.
    - **System and event logs:** record what happened on a machine — services starting and stopping, configuration changes, errors, crashes.They answer

"what happened on this host," useful for spotting malware, unauthorized changes, and signs of compromise on the endpoint itself.

- **Application logs** record what happened inside specific software — a web server's record of every request, a database's record of every query, an email system's record of every message. These answer questions specific to the application: which URLs were requested of the web server, which account queried the customer database.
- **Audit logs** record security-relevant actions specifically, often for compliance: who accessed what sensitive data, who changed permissions, who performed administrative actions. These are frequently required by regulations and are designed to answer "who did this privileged thing," which matters enormously for insider threats and compliance investigations.

- What makes a log useful:A log entry is only valuable if it actually contains what an investigation needs.
  - **Timestamps, and synchronised clocks.** Every log entry needs an accurate time, and across an organisation those times need to agree. All the servers time’s should be in sync, otherwise the impossible-travel detection breaks if the two systems disagree about what time it is. Organisations use NTP(time sync protocol). Accurate, synchronised time is the unglamorous prerequisite for correlating logs across systems at all.
  - **Enough detail**, **but not too much**. A useful log records what's needed to answer questions: who, what, when, where, outcome. An investable log should contain records of the account, source address, time, and success or failure. Good logging is deliberate about capturing what matters without drowning in what doesn't. If everything gets logs then there are volumes of them which increases the cost, and buries what matters.
  - **Retention: keeping logs long enough**.A log that's deleted before you need it might as well never have existed. The catch is that attackers often go

undetected for a long time — the average time to detect a breach has historically been measured in months, not days.Retention is a balance: long enough to investigate realistic incidents, against the cost of storing high volumes of data for long periods. Many regulations also mandate minimum retention periods.

    - **Integrity: logs the attacker can't erase.** An attacker who compromises a system can often delete or alter that system's logs to cover their tracks — clearing the evidence is a standard step in a serious intrusion.The defence is to ship logs off the system as they're generated, to a central place the attacker doesn't control — which is exactly what feeding logs to a SIEM accomplishes, beyond the correlation benefit.

- Reading a log entry: Most logs are lines of text, one event per lime, with fields for who, when what, and outcome describing the event.
- Status codes carry a lot of meaning: many 404s from one address walking many paths is reconnaissance, where someone is probing for hidden or unlinked pages, i.e walking through many paths to find what exists.
- 401/403 bursts are failed access to protected resources.
- The investigation loop is identical to packet analysis — orient, question, filter, read, conclude — whether the raw material is packets or log lines.
- Analysts routinely search and filter logs using command-line tools, and being comfortable slicing through a large log file to find the relevant lines is a core practical competency.
- Real-world logging: breach post-mortems repeatedly find that quick-containment organisations had good logging while catastrophic breaches had logging gaps.
- Logging is also a compliance requirement (PCI DSS, HIPAA, GDPR mandate it)
- The defence stretches in one line: logs are the raw material, the SIEM correlates them, detection generates alerts, analysts investigate — it all rests on the records.

DAY-39 Network Forensics

- Network forensics is the practice of reconstructing events from the evidence a network leaves behind. Primarily the logs and packets captured.
- Forensics answers “what exactly happened?” after the fact, with enough rigour to support a response, a report, and possibly a court case.
- The central output of forensics is a timeline — a precise, ordered account of what happened, when.
- The investigator gathers evidence from every relevant source (authentication, network, system, application logs, plus captures) and places each event in time order on one timeline, revealing the story: initial access, lateral movement, data access, exfiltration.
- The questions that forensics answers are:
    - **How did they get in?** The initial access. A phished credential, an exploited vulnerability, a compromised third party, a stolen laptop. Identifying the entry point matter both for understanding the incident and for closing the hole so it can’t be used again.
    - **What did they do once inside?** the actions on objectives and the lateral movement. Which systems did they reach? What did they run? Did they escalate their privileges? Did they move from the initial foothold toward more valuable targets? This is reconstructed from system logs, network logs, and captures, and it's where the bulk of forensic effort usually goes.
    - **What did they access or take?** The impact. This is often the most consequential question, because it determines the organisation's obligations. If personal data was taken, breach notification laws may require informing regulators and affected individuals with a strict timeframe.
    - **Are they still here?** Persistence. Sophisticated attackers establish ways to maintain access even after the initial entry point is closed — backdoors, created accounts, scheduled tasks.A forensic investigation has to determine

whether the attacker still has a foothold, because containing an incident while leaving a backdoor open means the attacker simply returns.
    - When did it all happen? The timeline that ties the others together. Establishing the sequence and timing answers not just "what" but "in what order," which reveals the attacker's method and often their level of sophistication.
- Forensic evidence may end up in court — supporting a prosecution, a lawsuit, an insurance claim, or a regulatory action — and that imposes requirements that casual investigation doesn't have.
    - **Preserve the original;** work on copies. A core forensic principle is that you never investigate the original evidence directly, because investigation can alter it. Instead you make an exact, verified copy and work on that, leaving the original untouched.
    - **Prove the evidence wasn't altered.** Forensic copies are verified with cryptographic hashes- a kind of digital fingerprint that proves a copy is identical to the original and that neither has been tampered with since.
    - **Chain of custody.** This is the documented record of who handled the evidence, when, and what they did with it, from the moment of collection onward. An unbroken chain of custody is what makes evidence admissible — if there's a gap where the evidence was unaccounted for, its integrity can be challenged and it may be thrown out.
    - **Why this rigour matters even when there's no court case.** Because you rarely know at the start of an investigation whether one will.An incident that looks minor can turn out to involve regulated data, insider crime, or litigation. The discipline of treating evidence properly from the beginning means you're never caught having contaminated the one thing that turns out to matter.
    - Incident response and digital forensics (DFIR) is a well-defined, well-paid security career; specialist firms investigate major breaches. The tools build

on what you know — Wireshark is core, and the SIEM's searchable archive is often the forensic starting point.

DAY-40 Zero Trust

- Zero trust is the response: stop extending trust by default. The principle is captured in a phrase you'll hear constantly in modern security-never trust, always verify.
- Because nearly every attack abuses trust extended by default — including a traditional network trusting its own inside.
- The principle is "never trust, always verify": no user, device, or connection is trusted for being in a particular place; every access request is verified every time.
- The old "castle-and-moat" model built a strong perimeter and trusted everything inside it — hard shell, soft centre. It worked when the employees and systems were contained at a single place, but the shift to cloud and remote working made it hard and dissolved the perimeter.
- Zero trust is not a not a single perimeter it is actually built from several they are:
    - **Verify explicitly, every time**: every access request is authenticated and authorized based on all available information- who the user is, what device they're on, where they're connecting from, what they are trying to access.
        - This requires strong authentication and MFA
    - **Use least privilege**: every user and device gets only the access they genuinely need, and no more. A compromised account or device can only reach what it was specifically permitted to reach, which sharply limits the damage.
    - **Assume breach**: When you assume a breach you naturally segment heavily when designing. You verify constantly, and monitor everything.
    - **Microsegmentation**: It divides the network into very small segments, potentially down to individual workloads, each with its own access controls. The goal is even inside a zone where nothing is trusted by default.

Every connection between every pair of devices is controlled which eliminates the change of lateral movement.
    - **Continuous monitoring**: even after a request is verified the systems keep watching, and if something changes, access can be revoked.
- In practice: zero trust is a gradual journey, not a switch (mature organisations say they're "on a zero-trust journey," never finished)
- Identity becomes the new perimeter — when location stops determining trust, strongly-verified identity does, making MFA and identity management central.
- There's a real user-experience tension — verify rigorously but unobtrusively, because security people route around isn't security
- It's now formalised guidance (NIST SP 800-207, Google's BeyondCorp) rather than just a buzzword.

DAY- 41 IPv6

- IPv6 is the real solution to the IPv4 address exhaustion, it is 128 bits instead of 32, which gives about **340 undecillion addresses.**
- The scarcity that forced NAT to exist vanishes. Roughly 40 to 45% global traffic has already adopted it.
- IPv6 addresses are eight groups of four hexadecimal digits separated by colons (2001:0db8:85a3:0000:0000:8a2e:0370:7334)
- Drop leading zeros in each group, and replace one run of all-zero groups with a double colon (::).The example above,2001:0db8:85a3:0000:0000:8a2e:0370:7334, shortens to 2001:db8:85a3::8a2e:370:7334.
    - You can only use double colon per address.
- **Loopback** — the "this device itself" address that's 127.0.0.1 in IPv4 — is simply ::1 in IPv6
- **Link-local** addresses start with fe80 and are used for communication on the local network segment only; every IPv6 device automatically gives itself one.

- **Global addresses** are the publicly routable ones, the IPv6 equivalent of a public IPv4 address
    - The difference is every device can have its own public address as it can be accommodated.
- Unlimited addresses change more than counting. NAT becomes unnecessary, which removes its accidental inbound-blocking side effect — so IPv6 security must be deliberate (the firewall does the job NAT used to do by accident).
- The devices configure themselves, with a mechanism called **SLAA (stateless address autoconfiguration)**. That lets devices generate their own address automatically, without needing a server to hand them out.
- Early IPv6 embedded hardware addresses like MAC, a tracking risk fixed by privacy extensions.
- The transition is itself a risk: IPv6 running unmonitored alongside secured IPv4 is an open door, and it's often enabled by default, so both protocols must be secured."
- IPv6 reinforces track-wide themes: defence must be deliberate not accidental (NAT's inbound blocking was a side effect; IPv6 forces the firewall to do it explicitly, which runs on the principle of zero trust).
- New technology brings new attacks that security must keep pace with, and the unmonitored path is the dangerous one, IPv6 alongside secured IPv4 is a blindspot, the current networking equipment should secure both protocols.
- The transition is slow because IPv4 and IPv6 aren't directly compatible (no clean switchover possible, so both run in parallel indefinitely). A device speaking only IPv6 cannot directly talk to a device speaking only IPv4; they need translation or dual-stack arrangements in between.
- NAT relieved the address-exhaustion pressure that would otherwise have forced rapid adoption — the workaround postponed the real fix.
- The practical state is dual-stack: most of the modern internet runs both, which is why you need to recognise and understand both.

DAY-42 Wireless Attacks

- Most connections in modern life are wireless, and people extend them without thinking. Wireless adds a vulnerability that wired networks lack: as the signal is broadcasted through air, receivable by anyone in range, because the medium is shared rather than requiring physical cable access.\
- The **evil twin attack,** its common, effective and exploits trust. an attacker sets up a wireless access point that impersonates a legitimate one, and victims connect to it believing it's the real thing.
- Once the victim is connected to the evil twin, **all the traffic flows through the attacker's equipmen**t- which gives the **man in the middle position**, achieved not by ARP poisoning but simply by getting the victim to connect to you in the first place.
- Meanwhile the attacker s**ees the metadata of everything** — which sites are visited — and can attempt to interfere with anything not properly encrypted.
- A common refinement is a captive portal that **mimics a real login page**, **harvesting credentials** when the victim "logs in to the Wi-Fi" with an email and password they reuse elsewhere.
- The evil twin gives the attacker the man-in-the-middle position, but H**TTPS still protects the contents** of properly encrypted connections.
- Any attempt to substitute a fake certificate triggers the warnings you've been taught never to ignore.
- The evil twin tracks where you do, but **HTTPS protects** what you do, a good encryption is what will save the trouble like a **VPN** which adds a tunnel which the evil twin can't see into.
- Other wireless attacks that are worth knowing are:
    - **Rogue Access point**: a rogue access point is an unauthorised wireless access point connected to a network, often plugged in by a well-meaning employee who wanted better Wi-Fi coverage, sometimes by an attacker.it

creates an unmonitored, often poorly secured entry point into a network that bypasses the organisation's controls.
        - Defense: the networks are scanned for unauthorized access points, and policies forbid plugging in your own.
    - **Deauthentication attacks**. Wi-Fi has a feature that lets a network tell a device to disconnect.In older Wi-Fi security, this disconnect message wasn't authenticated — meaning an attacker could forge it and force devices off a network.
        - Denial of service: repeatedly knocking off devices off a network they can’t stay connected to
        - Forcing a disconnect can push a device to reconnect, and the attackers can exploit the situation.
        - WAP3 protects against this.
    - **offline password cracking** (capturing the join exchange and guessing the password on the attacker's own hardware, why Wi-Fi password strength matters.
    - Wi-Fi security evolved to close these gaps: avoid the broken WEP, use the most modern standard your equipment supports, ideally **WPA3**."
- On networks you don't control: assume hostile — rely on HTTPS, heed certificate warnings, use a VPN, distrust network names and captive portals, and prefer cellular when possible.
- On networks you do control: use WPA3 (or WPA2 minimum, never WEP/open), set a strong long Wi-Fi password, change default router credentials, update firmware, and segment (guest and IoT networks)
- Cellular (4G/5G) is generally more secure than open Wi-Fi and encrypted to the tower, which is why it's preferable to unknown public Wi-Fi
- but IMSI catchers (Stingrays) can impersonate towers, so no wireless medium is immune to impersonation, and content encryption still protects you.

- The legal line is bright: capturing or interfering with networks you don't own is a crime — practise only on your own equipment or with written authorization.

DAY-43 DNS Tunneling

- **DNS is almost never blocked,** because blocking it would break the internet — firewalls allow **port 53 (**DNS) outbound nearly everywhere, even on tightly controlled networks.
- DNS tunneling exploits exactly this: it hides data inside DNS queries and responses, smuggling it through defenses that wave DNS straight through
- It turns the most trusted and essential protocol on the internet to a covert channel.
- The trick: a DNS query carries attacker-controllable text (the domain name), and the response carries text back.
- Malware encodes data into a subdomain (data-here.evil.com) and queries it; the query travels unblocked up the DNS chain to the attacker's authoritative server for evil.com, which reads the data and encodes a reply into its response.
- This builds a two-way channel out of normal-looking lookups, used for data exfiltration (leaking data out) and command-and-control (malware receiving instructions). Slow and inefficient, but it works where everything else is blocked."
- When an DNS is abused there are the some abnormalities that the defender can identify and spot an tunneling attack:
    - **Abnormally long and complex domain names**.DNS tunneling has to cram data into the domain name, producing long, random-looking subdomains full of encoded characters.
        - A query for x7f3k9q2bv8h4m1z.evil.com does not look like anything a human or normal application would request.
    - Abnormally high DNS volume. Because each query carries only a little data, moving any meaningful amount requires many queries — far more DNS traffic than a device would normally generate.

        - A workstation suddenly making hundreds or thousands of DNS queries to subdomains of one domain is a strong signal
    - **Unusual DNS record types and patterns**. Tunneling often uses DNS record types (like TXT records, which hold arbitrary text) more heavily than normal traffic does, because they can carry more data. Heavy use of unusual record types to one domain is another flag.
    - **Queries to one strange domain**. Above all, the traffic concentrates on the attacker's domain — all those weird, high-volume queries go to subdomains of the one domain the attacker controls
        - A single domain receiving a flood of long, random subdomain queries from inside your network is the signature.
- DNS tunneling teaches lessons beyond itself: trust and necessity create an attack surface (the things you can't block are what attackers target — abusing the trust firewalls must extend to DNS).
- you can't block your way to security (necessary channels can't be eliminated, so you must monitor and detect)
- Covert channels are a category (hiding malice in trusted traffic — HTTPS, cloud, social media APIs — not just DNS).

DAY- 44 A SOC Analyst's Day

- Security Operations Center (SOC) monitors an organisation's security around the clock; its analysts watch alerts, investigate the suspicions, and respond to the real incidents.
- They work in tiers- Tier 1 triages alerts, Tier 2 investigates escalations, Tier 3 are senior responders and threat hunters. Mostly everyone enters through tier 1 and its the foundation.
- A realistic Tier 1 shift: open the SIEM, take the handover, work the alert queue. Most alerts are routine triage — an impossible-travel alert turns out to be a VPN on a work trip (a false positive, the everyday reality).

- The interesting one — high-volume DNS to a strange domain — is recognised as possible DNS tunneling, evidence gathered, escalated to Tier 2.
- Afternoons bring tuning rules, updating blocklists, moving investigations forward. The shift ends with a handover note.
- skills this track has built — reading traffic, recognising attacks, using a SIEM, investigating logs, documenting findings — are its actual foundation.
- A SOC analyst who finds something but can't clearly document it and escalate it has not done the job.
- The analysts who advance are the ones who communicate clearly, because security is a team activity and findings have to travel.
- The shift work and the pressure of not missing the real alert are real, with alert fatigue an occupational hazard; and it's a launchpad — a common entry point that opens doors to forensics, threat hunting, pentesting, engineering, and beyond.
- The technical foundation is what this track built (networking, attacks, defensive tools, reading traffic and logs, investigation), rounded out by broad cybersecurity fundamentals.
- Hands-on proof (the portfolio artifacts this track builds, plus practice platforms) turns a cert into a credible candidate.

DAY- 45 Defending a Small Business's network.

- Network phase capestone project, defending maya's network by using everything learned.
- It mirrors the actual work of a security consultant assessing a client's environment: a messy real-world setup, made defensible. You'll produce a Network Defense Plan, your third and most comprehensive Networks portfolio artifact.
- The current situation of Maya's shop:
    - Maya's grown network is one flat, unsegmented network containing: two payment tills, four never-updated IoT cameras, a back-office laptop with customer data, Maya's dual-use personal laptop, staff phones, a smart

speaker, a printer, and customer Wi-Fi sharing the same network as the till. WPA2 with a three-year-old chalkboard password, default router admin credentials, no monitoring, no logs, and uncertainty about even having a firewall.

- Professional network assessments follow a structure, here is a simplified version for one of them.
    - **Inventory First:** You cannot defend what you don't know exists. The first step is the inventory of what's on the network- every device, every service, every connection.
    - **Identify the risks**: With the inventory in hand, you assess each element against what you know: what's wrong, what could go wrong, what's exposed.
    - **Not every weakness is equally urgent**, and a plan that treats them all the same is a bad plan. Real assessments prioritise: what's the most dangerous, most likely, most consequential weakness, and what's merely untidy?
        - The payment system exposed to the customer Wi-Fi is a critical, urgent risk (real money, legal obligations, the most likely target). The smart speaker is a lower priority
        - Prioritisation is what makes a plan actionable for someone with limited time and budget- which every business has.
    - **Recommend specific fixes.** Vague advice ("improve security") is useless. Each risk gets a specific, concrete remediation: not "segment the network" but "put the payment tills on their own network segment, isolated from customer Wi-Fi and from the cameras."
    - **Acknowledge constraints.** Maya is not an enterprise. A plan that demands she hire a security team and buy a SIEM is worthless to her. The art of small-business security advice — which you've seen throughout the Maya thread — is recommending what's realistic at her scale: router features she already has, affordable services, habits that cost nothing.

- **Think like the attacker**: Maya's network offers a short, quiet path — crackable/chalkboard Wi-Fi password onto a flat network where any foothold reaches everything, a default-credentialed router, no monitoring to notice.
- **Think like the defender:** the highest-leverage fix is segmentation, then verification and access control, then monitoring assume-breach/limit-the-blast-radius turned into a priority list.
- You'll produce a **Network Defense Plan** for Maya: assessment summary, network inventory, prioritised risks (each tied to why it's dangerous), specific realistic fixes, a phased action plan (first / this week / this month — executable order beats overwhelming perfection), and how to verify success.

Linux

DAY- 46 Why Linux

- The goal is to navigate a system, read files, run commands, understand permission, and write sample scripts.
- Linux is an operating system, which manages the computer's hardware and runs its programs. This is the layer between the physical machine and the application that has been used.
- The key differentiator for Linux is its open source, meaning the underlying code is freely available for anyone to read, modify, and use- unlike Windows and macOS, which is closed.
- It comes in many varieties, called distributions or distros- ubuntu, Debian, Fedora, Kali and dozens more, they all are linux but packed for different purposes. Ubuntu is beginner friendly, Kali is built for security testing, other target servers, or tiny devices, or specific needs.
- You already use Linux constantly: most internet servers and all Android phones run it.

- We focus on the command line because that's where the professional power is, not because Linux requires it.
- Why does command line linux skills is required for cybersecurity:
    - **The servers you defend and attack run on Linux**: The majority of the world's web servers, cloud infrastructure, and backend systems run Linux. You can't secure or assess what you can't navigate, so understanding Linux is understanding the actual terrain of the work
    - **The security tools run on Linux**: Wireshark, the IDS tools like Snort and suricata, the analysis utilities- a huge proportion of it runs on Linux.Kali Linux exists specifically as a distribution pre-loaded with security tools.
    - **The command line is faster, scriptable, and remote-friendly.**The command line lets you do things precisely, combine small tools into powerful operations, and critically automate them with scripts. It also works over remote connections. A SOC analyst or system's administrator working on a remote server has only the command line, and that's normal.
    - **Transparency suits security.** Because Linux is open source, you can see exactly what it's doing — every file, every process, every configuration is inspectable. For security work, where understanding precisely what a system is doing is the whole job, this transparency is invaluable.
- Why the command line, when the graphical interfaces are easier, why does serious technical work still use the command line.The command line is also called the terminal, the shell , or CLI(Command line interface)- is a way of controlling a computer by typing in commands rather than clicking.
    - **Precision and Power**: A graphical interface can only offer the buttons its designers built. The command line gives you direct access to the full capability of the system — anything the computer can do, you can instruct it to do, with exactness.
        - Renaming these three hundred files according to this pattern is a nightmare for graphical interface and one line on the command line.

- **Combination**: Command-line tools are small and focused, but they can be chained together — the output of one feeding into the next — to build operations far more powerful than any single tool.
- **Automation**. Anything you can type, you can save into a script and run again automatically. Repetitive work that would take hours of clicking becomes a script you run in seconds.
    - For security work — checking many systems, processing many logs, responding to many alerts — this automation is transformative.
- **Speed and remoteness**. Once you're comfortable, the command line is simply faster for many tasks, and it works over the thin remote connections used to reach distant servers, where there's no graphical option at all.

Day- 47 Setting up Linux:

- Today you set up a real, isolated Linux environment on your own computer — a safe sandbox where nothing you do can harm your actual system
- There are several ways to get linux running, and understanding the options help you pick the right one:
    - **Windows Subsystem for Linux (WSL)**: WSL is a feature built into modern Windows that runs a real Linux environment directly inside Windows, with a single command to install.
    - **Virtual machines**. A virtual machine (VM) is a complete computer simulated in software, running inside your real computer. UTM or VirtualBox for macs, creates a virtual computer, and you install linux into it as if it were a real machine. The linux inside it is completely isolated- its a separate simulated computer, so its extremely safe.
        - The tradeoff is that it uses a lot of computer resources.
    - **The cloud and online options.** You can also rent a Linux server from a cloud provider, or use free in-browser Linux terminals for quick practice.

These are useful in specific situations — a cloud server is real-world relevant, and browser terminals need no setup at all — but for this phase.

    - **Dual-boot or full install**. The most committed option is installing Linux directly on your computer, either alongside your existing operating system (dual-boot) or replacing it. This gives the best performance and the most authentic experience, but it's the most involved to set up and carries more risk if done wrong.
- You'll install Ubuntu, one of the world's most popular Linux distributions: beginner-friendly, hugely widely used (so skills transfer to real systems), strongly community-supported (your problems are already solved online), and free.
- A fresh Linux prompt shows your username, computer name, and location, ending in a dollar sign with a blinking cursor — the dollar sign means it's ready for a command, and the tilde means you're in your home directory.
- After Installing the Linux run these commands to check if its working or not:
    - `whoami` — prints your Linux username. Confirms who you are in the system.
    - `pwd` — stands for "print working directory", shows where you currently are in the filesystem (you'll be in your home directory).
    - `uname -a` — prints information about the system, confirming you really are on Linux.
    - `echo Hello Linux` — prints back whatever you tell it. A gentle proof that the system does what you type.

Day- 48 File System

- A filesystem is simply how an operating system **organises and stores files.**Linux follows a well-defined structure, the same on nearly every system,
- The core idea is that Linux organises everything into a single **tree of directories**. There's one starting point at the very top, and everything else branches downward

from it. Files live inside **directorie**s, directories live inside other directories, and the whole thing forms one connected **hierarchy**.

- The top of the linux tree is called root directory, and its symbol is forward slash: /. Everything on the entire system lives somewhere beneath the root.
    - When you see a path starting with /, it's starting from the very top of the tree.
- Root means two different things in Linux, and it's worth separating them now so they don't tangle later. **The root directory** (/) is the top of the filesystem- a location. **The root user** is the all-powerful administrator account- a person.
- From the root a tree branches into directories, and from those into more directories, down to the individual files. A **path describes a location in this tree** by listing the directories you pass through to reach it, separated by slashes.
    - /home/maya/documents means: start at the root, go into home, then maya, and then documents.
- The tilde (~) is shorthand for your home directory, your personal space where you start.
- pwd prints the full path from root to where you are now — readable as a route.
- There are the set of directories under the root of linux system, it is the same in every Linux system, here are some:
    - **/home**: holds the personal directories of regular users- your home directory lives here, at /home/your-username. This is your space, where your files go.
    - **/etc:** holds system configuration files — the settings for the system and its software. When you need to configure something on Linux, the answer very often lives in /etc. For security work this directory matters enormously, because configuration is where much security lives.
    - **/var:** holds variable data — files that change as the system runs. Critically for you, this is where logs live, in /var/log. On real linux these logs are files under /var/log. This is where an investigator would go to examine the system.

    - **/usr:** holds user programs and their supporting files — much of the installed software. Most of the commands you run live somewhere under here.
    - **/bin and /sbin:** hold essential programs — the actual command-line tools. When a command is run on the system, the program that runs lives in a directory like this. /bin for general commands, and /sbin for system-administration ones.
    - **/tmp:** holds temporary files, cleared regularly. It's worth knowing for security because, being world-writable and temporary, it's commonplace for both legitimate scratch work and the attacker to drop files.
    - **/root:** is the home directory for the root user, not to get confused with itself. The administrator's personal space.
- There are two ways to describe a location in the tree:
    - **An absolute path** starts from the root and describes the complete route from the very top of the tree. It always begins with /, because it always begins at root.
        - /home/maya/documents/report.txt is an absolute path.
        - it works from anywhere, because it specifies the full route from the top
    - **A relative path** starts from where you currently are and describes the route from there. It does not begin with /. If you're currently in /home/maya and you refer to documents/report.txt, that relative.
        - It means the documents directory is here and report.txt inside it.
    - The two special relative path symbols are a single dot (.) means "the current directory" — right here. Two dots (..) means "the parent directory" — one level up the tree, toward the root.
        - If you are in /home/maya/documents and you refer to .. you mean /home/maya. These double dots move up the tree.

- Linux's defining principle: nearly **everything is treated as a file** — not just documents, but hardware devices (under /dev), running processes (under /proc), and live system information, all presented through the same unified file interface.
- This means one consistent set of file-handling skills works on almost the entire system: **learn to read a file** and you can **inspect hardware, processes, configuration, and logs** alike.
- For security it's a gift — investigating a Linux system is largely knowing which files to read (configuration in /etc, logs in /var/log, processes in /proc).

Day-49 Navigation

- Navigation is the connective tissue of all command-line work — almost every task begins with getting to the right place and seeing what's there.
- The essential commands are a very few: pwd(where am I), ls (what's here), cd(move there).
    - **pwd (where am I)**: This stands for **print working directory** and it prints the full absolute path from the root to the **current location**.
    - **ls (what's here)**: This stands for **list** and it **shows the content** of a directory- the files and sub directories that it contains. ls is who you see what;s around you.
    - **cd (how do I get there)**: This stands for change directory, and it's how you move. You give it a destination, and it moves you there. cd documents moves you into the subdirectory of documents.
- The rhythm is: cd to move, ls to look, pwd to confirm.
- Moving around the tree with cd:
    - **Moving down (into subdirectories)**: uses the directory name, usually as a relative path. If ls shows you a directory called projects in the current location, cd project moves you into it. You can go several levels at once:
        - cd projects/website/image moves you down by three levels.

    - **Moving up (towards the root)**: uses the double-dot. (cd ..) moves you up one level, to the parent directory. cd (../..) moves you up by two levels.
        - (cd ../documents) means go up one level then into documents.
    - **Jumping anywhere**: uses an absolute path, starting from the root. (cd /etc) takes you straight to the configuration directory no matter where you currently are, as the absolute path is location independent.
    - **Two shortcuts you'll use constantly**:
        - (cd ~) or just (cd) takes you home, to your home directory.
        - (cd -) takes you back to the directory you were just in, like a back button.
- A few features turn slow, error-prone typing into fast and fluent movement.
    - Tab completion is the most important: start typing a name, press Tab, and the system completes it — saving typing and preventing typos (press Tab twice to see multiple matches).
    - ls options reveal more: ls (-l) for a detailed listing (permissions, owner, size, date — central) (ls -a) for hidden files (dot-prefixed names, which matter for security as config and hidden-malware both use them). (ls -la) combines both of the previous commands.
    - clear (or Ctrl+L) cleans the screen, up-arrow recalls previous commands.
- "ls" is itself a security tool: simply listing and looking reveals unexpected files, hidden files (ls -a), recent modifications in places nothing should change (ls -l), and strange permissions — the investigator's eye built on the navigator's habits.

DAY- 50 Navigate a Linux System

- The strategy for exploring an unknown system. Investigators and administrators use exactly this approach, scaled up, when they first sit down at a system they don't know.
    - **Start at the top and get the lay of the land:** at the root always begin with ls / to see the major directories- the standard structure. This tells you the

system follows the normal layout and reminds you where the important areas are: home for users, etc for configuration, var for logs and changing data. Five seconds here frames everything that follows.

- **Go where the interesting things are**: Rather than wandering randomly, head for the directories that matter for your purpose.
  - Investigation- The logs in /var/log
  - Configuration- /etc
  - Temporary files- /tmp
- **Look carefully when you arrive**: At every location use ls to see what's there, and ls -la to see everything including hidden files and details. This is the most crucial part in investigation, a file that shouldn't be there, a hidden file, a recent modification date.
- **Drill down where something interests you**: When a directory contains something worth examining, cd into it and look again. Follow the structure downwards towards whatever you're after, ls-ing at each level.

Day-51 Permissions

- The clustered letter and dashes like "rwxr-xr–" before each line when the command "ls -l" is ran, these are the permissions, they control who is allowed to do what to every file and directory on the system, and understanding them is foundational to linux system.
- The crucial question answered is: who can read it, who can change it, and who can run it. When a system is used by multiple people and it is facing the internet, this is very crucial to protect from an attack.
- Permissions are the least privilege made converter at file level: every file states who needs what access. Too loose permissions are where the security problem begins, so setting them is a genuine hands-on skill.
- The three permissions and the three groups.

    - **The three permissions** are read, write and execute, written as the letters r,w, and x. **Read(r)** means you can view the contents-read a file, or list a directory's contents. **Write(w)** means you can change it-edit, or delete a file, or add and remove things in a directory. Execute (x) means you can run it- run the files as a program, or (for a directory) enter with cd.
        - For a directory, execute means the right to go into it.
    - **The three categories** are users, group, and other. **Users(u)** is the owner of the file, usually whoever created it. **Group(g)** is a group of users who share access to every file belonging to a group, and members of that group set the group permissions. **Other(o)** is everyone else, every other user on the system which on an internet facing system means every one on it.
- Each file records read/write/execute separately for user, group, and other — nine settings that precisely state who can do what.
- The permission in ls -l are written as a string of ten characters:
    - -rwxr-xr--, read it left to right.
    - The first character is a dash(-) or d it means the file is a regular file
    - The next three (rwx) are the user, it means they can read, write and execute it.
    - The next three (r-w) are for the group: read, no write, and execute.
    - The last three: (r--) are the other permissions: read only.
    - A dash in any position means that permission is absent.
- The pattern is the same ten characters, in the same order: type, the user permissions, group permissions, and others.
- The ls -l commands also shows you the owner and the group the permissions refer to.
- Two commands that can change the permissions are chmod (changes the permissions themselves) , and chown (changes who owns the file).
    - **Chmod ("change mode")** sets a file permission. There are two two ways to specify them:

        - **The symbolic**, which almost reads like english: **chmod u+x**, adds the execute permission to the user; **chmod o-r** removes the read permission for others.You name the category (u,g,o), whether you're adding or removing (+ or -), and the permissions are (r,w,x).
        - **The numeric way**: each permission has a number: read(4), write(2), execute(1). You add them in the read+write+execute: 4+2+1=7. read+execute is 4+1=5; read only is 4. You give three digits, one each for user, group, and others. **Chmod 755** means: users (7)(rwx), group(5)(r-x), and others(5)(r-x), this is a common setting for a **programming file**, and for a **regular file**: **chmod 644** means user 6 (rw-), group 4 (r--), other 4 (r--) — a very common setting for regular files: owner can read and write, everyone else can only read. **chmod 600** means user 6 (rw-), group and other 0 (nothing) — owner-only access, used for **private files** like keys.
    - **chwon (change owner)** changes which users own the file, and can also change the group. chown maya report.txt makes maya the owner of report.txt. Since the owner gets user permissions, this usually needs admin privilege.
- misconfigured (over-permissive) permissions are a leading cause of breaches — world-readable password files, world-writable privileged scripts, readable private keys — so finding and fixing them is high-value hardening
- Permission enforces the idea of least privilege at the finest grain: grant exactly what's needed, no more.
- The execute permission is a particular lever — malware must be executable to run, so removing execute neutralises a suspicious file without deleting the evidence.

DAY- 52 Users and sudo

- Permissions are the rules; users and groups are who the rules apply to — together they are Linux access control.
- Linux is multi-user to its core, keeping identities separated and bounded, which is central to how it stays secure. The most important idea is to differentiate between the root user, user, and disciplined practice of borrowing root's power only when needed via sudo.
- Every account on a linux system is a user, identified by username, and on the back of the system it is a number.
    - **The ordinary users**: They can work on their own home directories, run programs, and do every day things, they are deliberately limited. They cannot change system-wide settings, install software for everyone, or touch other users' files, because their permissions do not allow it.
    - The **root user** or **super user- all powerful administrator**: Root can do anything: read, modify or delete any files regardless of its permissions, change any system settings, install new software, create and delete other users. Root is not bound to any permission rules.
- Working as root is dangerous because nothing contains a mistake or a compromise — so the core discipline is: work as a limited user almost always, borrow root's power only when needed. That's the least privilege applied to your own account.
- **sudo ("superuser do")** runs a single command with root's power, just for that command, then drops straight back to your ordinary limited level.
- You prefix a privileged command with sudo, the system confirms with your password, runs that one thing as root, and you're immediately limited again — power borrowed in small, deliberate, confirmed bursts.
- It’s far safer than logging in as root and staying there: most work is contained at a limited level, and sudo-plus-password friction makes you pause before doing something powerful.

- Sedo also logs who did what with elevated power. Who is allowed to sudo is a security-significant decision- it's effectively the key to root.
- The middle layer that sits in between the user and everyone else is a **group**: is a collection of users who share access, and groups are how linux manages permissions for many people without setting them person by person.
- Create a group, add users, set files' group permission once, and all members get access through membership; managing the team means just adding and removing people.
- For security, group membership IS access: people quietly accumulate more privilege than intended through groups (the sudo group being the sharpest example – it means root access), so auditing who's in powerful groups is real security work.
- The basic picture: every file has an owner and a group plus permissions for owner/group/other; every person is a user in one or more groups with limited privileges; linux checks identity against a file's owner and group to decide which permission apply and sudo is the controlled, audited way to exceed ordinary limits.
- For an investigator it's a checklist who are the users, who can sudo, who's in powerful groups, are sensitive files owned and permissioned right- and a compromise often shows here in the access control.

DAY-53 Reading Files

- Reading files is most of what linux security investigation is, because everything is almost a file- logs, system state, configurations.
- The challenge is the size, some files might be thousands of lines, with the help of the right tool you can see what you need to without drowning.
- The two basic ways to read a file are to dump the whole thing or to page through it
    - **cat (concatenate):** cat "filename" - prints the entire content of the file straight to the screen. It is perfect for small files, as it can show the whole thing.

        - cat is not a viable option for larger files, as there will be thousands of lines in the file.
    - **less** : less “filename” opens the file in a pager- a viewer that shows one screen after the other and lets you move up through your own pace.
        - You scroll up and down with the arrow keys or page up/down
        - Jump to the start or end, search within the file by typing a slash followed by a word, which jumps you to the match.
        - You quit less by pressing q.
        - It loads a page gently rather than dumping all at once, so it is the best option for larger files.
- Often you don't want the whole file- you want just the beginning or just the end. Two tools give you exactly that, and they’re more useful than they sound.
    - **head** : shows the start of the file – head “filename”, prints the first ten lines by default, and you can ask for different numbers also.
        - This answers what’s at the top of the file- useful for seeing the headers, checking the type of the file, or sampling the first few entries without opening the whole thing.
    - **tail** : shows the end of the file, the last ten lines by default– tail “filename”.
        - This is very useful for logs, as they grow by adding new entries at the bottom.
        - tail with the -f option ("follow") shows the end of a file and then keeps showing new lines as they're added, live. This is invaluable for watching what a system is doing right now, an administrator troubleshooting a live problem, or a security analyst watching authentication attempts arrive as someone tries to log in.
        - Ctrl+c to quit the live tailing log.
- **grep** searches a file and shows only the lines containing what you're looking for.grep "failed" auth.log pulls every failed login from a vast log in one command
- A few options make grep far more useful.

    - **grep -i** makes the search case-insensitive, so it matches "Error", "error", and "ERROR" alike.
    - **grep -n** shows the line number of each match, so you know where in the file it is.
    - **grep -r** searches recursively through all files in a directory and its subdirectories, so you can search a whole tree at once.
    - **grep -v** inverts the search, showing lines that do not match, useful for filtering out noise you want to ignore.
    - **grep -c** counts the matches instead of showing them, answering "how many?
- For security it's indispensable — investigation is so often searching for specific things in large files, and grep is the signal extracted from the noise.

DAY- 54 Searching the Filesystem

- Today's tools find the files themselves- locating a file by what you know about it (name, type ,when it was changed) anywhere in the system.
- For security this is the investigation itself: hunting for compromised systems means searching for files that shouldn't be there- something an attacker dropped, a file modified in suspicious time, wrong permissions, a hidden file in an odd place.
- The main tool is **find,** and it is genuinely powerful- it searches the file system tree for file matching criteria you specify, and the range of criteria is large.
- The basic shape is: find, then where to start searching, then what to match. You give it a starting directory, and it searches that directory and everything beneath it.
    - **Finding by name**: find /home -name "report.txt", the search happens under the home directory for a file named report.txt.
    - **Finding by patterns**: find /home -name "*.log" , find every file ending in .log under /home directory. (* asterisk means anything here)
        - "find /" - searches the entire system under the root, slow and thorough.

        - “find .” - searches just from where you are, fast and focused.
    - **Finding by type**: “-type f” finds the regular files, “-type d” searches for directories. So find / -type d -name “config” finds for directories called config anywhere
    - **Finding by time**: “-mtime” finds files by when they were last modified. find / -mtime -1 finds everything modified on the last day.
    - **Finding by size**: “-size” find files above or below a size, used to find unexpectedly large files or empty ones.
    - **Finding by permission**: “-perm” finds files with specific permissions. find / -perm -4000 finds a particular dangerous permission type that attackers exploit.
    - **find by owner**: “-user” finds files owned by a particular user. find / -user maya finds everything maya owns.
- The fastest way is using **locate,** as find walks the actual filesystem everytime checking each file against your criteria.
- locate consults a pre-built index — a database of all the filenames on the system, built and updated periodically in the background.it's a lookup rather than a search, it's extremely fast, even across the whole system.
- There two limitations for locate, though it is faster than find:
    - First, it only finds by name — it can't do “find” criteria like modification time, size, permissions, or owner.
    - Second, and importantly, its index can be out of date: because the database updates periodically rather than constantly.a file created since the last update won't be in the index yet, and a recently-deleted file might still appear.
- That staleness has a security implication: an attacker's very recent files might not be in the index, so for recent activity, find (live filesystem) is more reliable.
- Use locate for everyday "where's that file by name?"; use find for rich criteria, live state, and security investigation.

- find is the investigator's hunting tool, its criteria matching the questions:
    - "what changed recently?" (by modification time — surfaces what an intrusion touched)
    - "what's in the places attackers like?" (find /tmp — executables in temporary directories are suspicious)
    - "What are dangerous permissions?" (a system-wide permission audit in one command),
    - "what does this suspicious account own?" (by owner — maps a malicious account's footprint)
- Finding files is pattern-matching against suspicion: form a hypothesis about what an attacker did and test it across the whole system instantly.

DAY- 55 Pipes and Redirection

- The pipe, which chains small tools so the output of one feeds the next.
- Linux's design is small, sharp, single-purpose tools plus a way to connect them — so a few simple tools combine into an enormous range of powerful operations, far more than could be built as features.
- The pipe (a single vertical bar) takes the output of the command on its left and feeds it as input to the command on its right — so a tool works on another tool's results instead of only on a file.
- Worked example: grep "failed" auth.log | wc -l hands grep's failed-login lines straight to the line-counter wc, giving a single number — how many failed logins — that neither tool could produce alone.
- Pipelines can chain several tools, reading left to right as a sequence of transformations: each stage simple, the chain powerful.
- Redirection sends a command's output to a file instead of the screen (or another command).
- The greater-than sign writes output into a file, replacing its contents (command > results.txt). The double greater-than appends to the end, keeping what's there

(command >> log.txt) — one arrow replaces, two arrows add (confusing them can lose data).

- Together with the pipe, this completes where output can go: to the screen by default, to another command via a pipe, or to a file via redirection.
- For investigation, redirection preserves evidence — capture a search's results or a list of suspicious files directly into a record for your report.
- The skill is building pipelines for real questions: start with the question, break it into simple transformations, each a tool. "How many times did this IP appear?" = grep then wc -l.
- "What unique errors?" = grep then sort then uniq. The recurring pipeline tools: wc (counts; wc -l for lines), sort (orders), uniq (removes adjacent duplicates, after sort), plus head/tail (take first/last few)
- Search, sort, deduplicate, count, take the top — an enormous range of questions becomes answerable.
- Pipelines distill huge logs into the precise answers, and it’s how you’ll extract findings.

DAY- 56 Investigating a suspicious File

- The first Linux capstone: investigate a suspicious file with the whole toolkit (navigate, permissions, read, search, pipes) to determine what it is and whether it's dangerous — the separate skills becoming one capability.
- The framing principle, a genuine field discipline: investigate safely — examine the file without running it, because running an unknown file is how you get compromised, while reading and examining it is how you understand it safely.
- A suspicious-file investigation is a sequence of questions each with a tool.
    - What kind of file is it- the file command reads the real contents and reports the true type, because a name can lie eg: invoice.pdf could be a program
    - What are its properties? – ls -l; shows permissions , owners, size, and modification time, each a clue.

    - What's inside: cat/less/head/tail read text and scripts directly; strings pull readable text out of binary (revealing addresses, commands, paths).strings is a key tool for examining binaries safely, because it shows you meaningful content without running anything.
    - What's notable inside? gerp scans the content for red flags (web addresses, suspicious commands, references to sensitive files). file , ls-l, string/cat, grep– that sequence builds a complete picture without running the file.
- A script is the perfect file to learn investigation on because it's readable: every line is a command it would run, so reading it reveals its full intentions safely.
- Red flags: downloading and running something from the internet (classic malware), modifying or weakening system files and permissions, creating user accounts, touching logs (covering tracks), seeking elevated privilege, or being obfuscated (hiding behavior, itself suspicious).
- This is why reading beats running: you learn full intent without the script ever executing, and removing execution lets you keep it as evidence safely.
- Investigation is not complete until you reach an honest verdict. The three valid conclusions are:
    - bening examined, does something legitimate — here's what
    - malicious: here's the specific dangerous evidence, named precisely.
    - suspicious/uncertain : here's what concerning and what I couldn't resolve.
- Evidence-based reasoning makes a verdict defensible: not "this looks scary" but "malicious because it downloads and runs a file from an unfamiliar address with elevated privilege — a classic dropper pattern.

DAY- 57 Processes

- The running programs are called processes, and "what's running on these systems?" is one of the most important questions in all of security.
- A suspicious file can only do harm if it runs - and when it runs, it becomes a process. Malware that has compromised a system is, at that moment, a running

process: consuming resources, making network connections, doing whatever it was built for.

- A file investigation asks "what could this do?"; a process investigation asks "what is happening right now?" Understanding processes is also fundamental literacy: the OS's main job is managing them, and on top sits the security skill of spotting the process that shouldn't be there.
- When you launch a program the system loads it and starts it running as a process, while the system runs many processes you never directly started. At any moment, a typical computer has a great many processes running, most of them the system quietly doing its work.
- Every process has a few key properties worth knowing, because they're what you look at when investigating each has a:
    - **process ID, or PID**: A unique number identifying that specific running process, which you use to refer to it.
    - **Owner**: the user the process runs as, which matters in security, as a process running as root has the root's power, so a malicious process running as root is far more dangerous than one running as an ordinary user.
    - **Consumes resource**s: processor time(CPU) and memory- and how much can be a clue: a process using a surprising amount of CPU might be doing something that it shouldn't.
    - **A parent**: the process that started it, forming a family tree of processes, which can reveal how something came to be running.
- As in linux nearly everything is a file, running processes appear as files too, under the /proc directory.
- Each process has a directory under /proc named by its PID, containing files that describe everything about it — what program it is, what it's doing, what files it has open, what it's connected to.
- This means the same file-reading skills you've built can inspect running processes, because processes are presented as files.

- The two essential tools show you the running processes, and between them they answer what's running in two complementary ways: a snapshot, and a live view:
    - ps (ps aux) gives a snapshot- a list of processes at the moment you run it. The most common form, ps aux. List all processes and the system with useful detail: The owner, the PID, how many resources each is using, and the command that's running.
    - top (modern version htop): gives a live view- a continuously updating display of processes, refreshing every couple of seconds, typically sorted with the most resources-hungry at the top.
    - Where ps is a photosnap top is a live video feed.
- Several things makes a process suspicious:
    - A process consuming a lot of CPU or memory for no clear reason can be malware doing its work in the background.
        - Cryptomining is a classic example, using a machine's resources to mine crypto for the attacker.
    - A process running as a root, that you don't recognise is concerning because it has full power.
    - A process with strange or random looking name, or a name that is slightly misspelled
    - A process running from an unusual location like /tmp, is this a red flag
    - A process making network connections that you cannot explain.
- The investigation is simple, you spot everything that's suspicious and you look into it. List with ps aux, scan for patterns, investigate what catches your eye (including via /proc)
- The kill command stops a process by its PID- halting active malware while you preserve the file evidence.
- A file investigation examines potential (what could happen); a process investigation examines the actual (what's happening now).

- Live activity can reveal what the disk doesn't, because some malware leaves little on disk while running actively in memory, so "what's running?" can catch a threat that files alone would miss.
- Processes connect to what's next: a suspicious process makes network connections, and leaves log entries.

DAY- 58 The Network form the Terminal

- When a machine is compromised, two questions dominate: what's running and what it is talking to.
- The networking part which was displayed in wireshark now you can see from inside of the terminal.
- Seeing what a machine is connected to, from the machine itself, with no wireshark needed is a core investigating capability.
- The central tool that shows you the network connection a machine has- what it's connected to, and what it's listening for.
- The modern command is ss (short for socket statistics). The older, classic one is netstat, which you may still see everywhere and which does much the same job.They show the live network state of a machine
    - First, established connections: active conversations the machine is currently having with other machines, each showing the local address and port and the remote address and port. This answers what is the machine connected to right now?
    - Second, listening ports: services on the machine waiting for incoming connections, each sitting on a port ready to accept. This answers what machine is being offered on the network?
- The ss common form also shows which process owns each port, tying a connection back to its process.
- In terms of security, an unrecognized listening port can be backdoor; an unexplained connection can be malware exfiltrating or beaconing.

- Two more terminal tools rounds out the network toolkit- one for testing whether you can reach something, one for looking up names- and both are everyday utilities:
    - ping : tests whether another machine is reachable across the network, and roughly how fast. You give it an address or a name, and it sends small test messages and reports whether they come back and how long they take
        - It answers the simple question like can I reach this and is the connection still alive.
    - dig : looks up DNS information- it's a terminal tool for the DNS. You give it a domain name, and it queries DNS and shows you the answer: the IP address that the name points to, and more details besides.
        - it queries DNS and shows what the name resolves to and more; how you verify from the terminal what a name actually points to, which matters because a falsified answer misdirects you.
- The loop: look at connections with ss, find one to an external address that doesn't fit, then trace it to the process responsible (every connection belongs to a process),
- The signs mirror suspicious traffic seen locally: a connection to an unfamiliar external IP, a listening port no legitimate service should have (a backdoor), repeated connections at regular intervals (the beacon signature), or a process with no reason to touch the network that's connecting abroad.
- A process running (what's running now). The network connection it makes (what's it talking to now), and the logs(what has happened over time) - three views of the same underlying activity that an investigator waves together.

DAY-59 Reading Logs

- A system keeps a record of what has happened over time- a written memory of events, stretching back hours, days, or longer. That memory is logs, and reading them is the third and final view of the investigation live system.

- Logs are where the investigation finds a story: failed logins before a break-in, the moment an account was created, what ran and when. Reading logs lest you tell the story of what happened.
- Things that are in /var/log:
    - The authentication logs (auth.log): It records authentication events logins and logouts, failed login attempts, uses of sudo, and other security-significant access events.
    - The system log: usually syslog or messages, is the general purpose record of system activity- services starting and stopping, hardware events, a broad stream of what the system and its software are doing. It's noisier and more varied than the authentication log, but this where you would look to see what a component has been doing.
- Individual applications and services also keep their own logs, often in their own subdirectories under /var/log
    - A web server logs the requests it received
    - A database logs its activity
- The modern way to keep logs in linux is through "Journal", read with a command called "journalctl". The journal stores logs in a structured way instead of plain text files. It lets you filter by time, by service, by priority.
- Reading logs with tools that are familiar:
    - tail - shows a log's most recent activity
    - follow - mode watches at the new entries live on authentication logs
    - less - pages through long log calmly
    - grep - pulls relevant lines from a huge log, such as failed login attempts, usernames, time.
- pipelines find their natural home here — counting failed logins, finding unique attempting addresses, ranking sources- distilling thousands of lines of code into a handful of facts that matter.

- Recognise the patterns: a password-guessing attack is a burst of failed logins in the authentication log; a success after the failures is the break-in moment, recorded; account creation, sudo escalation, and services starting/stopping all leave traces — and the traces in order are the story
- Timestamps are why logs matter: they let you order events in time (the timeline)
- With logs the investigation triad is complete, you can ask and answer the full set of questions: what's running now? “the process” what has happened over the time, and in what sequence.

DAY-60 Package Managers

- Linux has something more like the app store model, but more powerful and more central to how the whole system works: the package manager.
- The single most effective security is keeping software up to date, because most real world attacks exploited the known vulnerability in out up-to-date software- flaws already found and fixed but never applied.
- The package manager is the tool that does that updating, so learning it is learning the mechanism of security’s closest thing to a golden rule.
- The **package manager** is a tool that installs, updates, and removes software on a linux system, handling the whole job in a controlled, central way.
- The common ones are apt (ubuntu/Debian), yum or dnf (Fedora/Red Hat). The key idea is a repository: a trusted, curated collection of software maintained by the distribution, so installing draws from a vetted source rather than a random website- a major security advantage over downloading installers.
- The package manager automatically handles dependencies, supporting software a program needs.
- Core commands: apt update (refresh the catalogue), apt upgrade (apply updates the security-critical one) , apt install (add software by name) , apt remove (uninstall), apt search (find software). And any change the system software requires sudo.

- Keeping **software updated is the single most effective security practice** available.
- Software has flaws; when one is found, the makers fix it in an update — but until you apply it, your system still has the flaw, and once it's public and fixed, attackers actively hunt unpatched systems.
- The window between a fix's release and your application is a known, exploitable weakness. Most successful real-world attacks exploit known vulnerabilities in un-updated software — the fix existed, it just wasn't applied.
- So every **apt upgrade** is a security action, closing holes attackers hunt. Updates can occasionally break things (why production systems test first), but the risk of not updating far outweighs it.
- Software hygiene- current software, only trusted sources, remove what you don’t need - is foundational, and updates are its core.
- The shadow side of software management: installing from untrusted sources, a common compromise route. The safe default is your distributor’s curated repository (vetted, packaged, installed with apt).
- But Linux lets you install from anywhere — downloading packages, adding third-party repositories, running install scripts from the web, compiling from source — each stepping further from the vetted source and carrying more risk.
- A malicious install script is a classic attack ("just paste this to install our tool"), so the same "don't run untrusted things" discipline applies — installing software is running code that can change your system.
- The principle is trust and provenance: where did this come from, do I trust the source? Prefer the repository; step outside it only consciously, from official sources, with caution.

DAY-61 Bash Script.

- A text file of commands that runs them in sequence, top to bottom, whenever you run the file. Everything that you can type can be in script and it does for you in the same way every single time.
- This matters because real work is repetitive: any task done more than once can be captured in a script, and run with one command.
- A few things can turn an ordinary text file into a script, and understanding them demystifies the whole thing. There are really just three pieces:
    - **The first line: the shebang** (#!). A script usually starts with this in the first line, and then followed by the path to the program that should run the script. The shebang means “run this file using bash”
    - **Second: the commands:** one per line, usually, executed in order from top to bottom. echo: print a message, date: show the time, ls: list, anything. The script runs them in sequence top to bottom. Lines starting with # are the comments and it is a good habit to write these comments.
    - **Third: A script must be executable** to run, a file can only be run as a program if it has execute permission. A freshly written script does not have the permission so you add the permission using ‘ chmod +x “filename' ’.
- A variable is a named palace to store a value, so you can use it by name rather than repeating it. In a script, you might store today’s date and filename or a count in a variable, then refer to it wherever needed.
- Structure: scripts can make decisions if and loop, so they’re not limited to a fixed sequence but can branch and respond- exactly what a security script checking several things and reporting problems.
- As a defensive tool, scripting is everywhere: a script can gather a system's state for investigation, run routine checks across many systems, process logs, or monitor and alert — because doing everything by hand doesn't scale.

- But scripts are also common forms of attack: The day 56 suspicious file was a malicious script, and attackers use scripts heavily because automation serves them too.
- Writing scripts deepens your ability to read malicious ones- they're built from the same pieces they are just scripting, and whichever side you're on determines how you use it.

DAY- 62 Cron Jobs

- Making things happen on their own, on a secluded, with no one present. This is one of most useful administration capabilities- and one of the most important for security- because the same mechanism that runs your useful tasks automatically is exactly how attackers make malicious code keep running long after they've gone.
- On Linux the classic tool is **cron**, which runs scheduled tasks (cron jobs) at specified times, faithfully, in the background.
- The list of scheduled jobs is called **corntab** ("corn table"). Each user has their own, plus the system has its own.
- Each corntab line is one job with two parts: **a schedule** (when) and **a command** (what)- at this time run this. The command often being the script that one wrote
- The schedule has **five fields** : minute, hour, day of the month, month, day of week. (*) asterisk means every for a field.
- Scheduled tasks live in two places: each user's **personal crontab** and the **system-wide cron locations** (under /etc) — and both are places an attacker might hide a task and an investigator must check.
- Corn is a classic attacker's persistence mechanism. After compromising a system, an attacker wants to not lose access- across reboots, killed processes, or a closed entry point.
- They add a corn job running their malicious code on a schedule, so it runs again and again forever: kill the process and corn restarts it.

- For the investigation, checking scheduled tasks (both user crontabs and the system's corn location) is essential if you clean the active malware but miss the cron job that re-installs it, the attacker is back in minutes.
- Most cron jobs are benign and valuable: scheduled backups, log processing, automated security checks, routine cleanup. Using cron well: know what's scheduled, and run jobs with least privilege, not more than needed.
- A subtle real risk: if a cron job runs a script as a root but the script is writable by an ordinary user, anyone who can edit the script controls what runs as root. Corn used well is a faithful servant; used carelessly, a standing risk.

DAY- 63 SSH (Secure Shell)

- In real security work it almost never is: the systems you defend and administer are servers elsewhere, with no screen or keyboard.
- SSH is the answer — the secure way to connect to and control a remote machine across the network, giving you a command line on a distant machine as if you were sitting at it
- SSH is the standard near universal way to reach remote linux systems, assumed in all systems and security work- and, as a critical doorway, both an essential tool and a major attacker target.
- SSH gives you a command line on a remote machine, securely it encrypts everything between you and the remote machine, so anyone intercepting the networking traffic sees only scrambled data.
- It works client-to-server model, the remote machine runs SSH server listening on port 22, and you cannot with an SSH client.
- The command is ssh followed by username-at-address; you authenticate and get a shell on the remote system where every command runs there not locally.
- SSH authentication has two methods:
    - **A password**: simple but guessable, and internet-facing servers face constant automated password-guessing

    - **SSH keys**: stronger — a pair of mathematically-linked keys, a private key kept secret on your machine and a public key placed on servers you access; the public key verifies you hold the private one without it ever leaving your machine, so no password crosses the network and there's nothing to guess.
- This converges with the permissions: a private key is among your most sensitive files, so SSH insists it has owner only permission and refuses a key others can read; the most sensitive file gets the tightest permission, enforced by the tool.
- Because SSH grants full command-line access, it's one of the most attacked things on the internet — get a server's SSH and you have the server.
- Any internet-facing SSH server is hit within minutes by relentless automated guessing- failed logins filling the auth logs.
- So a weak SSH password on an exposed server nearly guarantees eventual compromise, so use keys not passwords, disable direct root login, limit who can connect, sometimes change the port, and monitor auth logs. It’s a microcosm of defence: a necessary service is also a risk, so you don’t remove it- you harden it.

DAY- 64 Hardening Linux

- **Hardening** is the practice of making a system as **resistant to attack** as possible- deliberately reducing the ways it can be compromised, before any attack comes.
- hardening is what you do beforehand to make going wrong far less likely — the difference between investigating a break-in and fitting good locks first.
- A hardened system isn't invulnerable, but it's dramatically harder to compromise and closes the easy paths most attacks rely on.
- **Attack surface** is the sum of all the ways a system could possibly be attacked: every service it runs, every account it has, every piece of software installed, every open port, every place an attacker could try to get in or exploit.
- The larger the attack surface, greater the chances of attacks, and vice versa. Hardening is fundamentally reducing attack surface, and this one idea unifies nearly every hardening action: turn off unneeded services, remove unused

software, delete accounts that shouldn't exist and limit those that remain, close to ports, nothing needed- each removes a piece of surface an attacker could target.

- Its minimalism-as-security principle, least privilege., and software hygiene are both attack surface reduction. The mindset should be, “does this need to be here, and if so, how can it be made safer?”
- The core hardening actions are to keep software updated, apply least privilege everywhere, remove what you don’t need; secure remote access, monitor and log and use fiewall.
- Two ideas complete hardening as a mature discipline.
    - Defence in depth: don’t rely on a single defence, layer several so that if one fails the other still protects you, harden SSH AND apply least privilege (so a breach is limited) AND monitor logs (so you notice) AND keep backups (so you can recover); each layer backstops the others, building resilience from imperfect parts
    - And the honest limit: hardening reduces risk, it doesn't eliminate it — there's no perfectly secure system, so the goal isn't invulnerability but making compromise as difficult, costly, and noticeable as possible.

DAY- 65 Linux Forensics

- Investigating a compromised system to reconstruct what happened- the real work of digital forensics and incident response, in miniature but genuine. The question that matters; is it compromised, what did the attacker do, how did they get in, what did they touch, are they maintaining access?
- Then write it up as an evidence backed report someone could act on.
- A real investigation is a methodical sweep through a checklist:
    - What’s running that shouldn’t be?- processes
    - What is it talking to that it shouldn’t? - connection
    - What does history show?- log
    - What accounts shouldn’t exist?- accounts

    - What's scheduled that shouldn't be?- tasks
    - What files are suspicious?- files
- The investigation's real output is the story — what happened, in order.now built for real: each finding has a time (log timestamps, file modification times, account creation, process start), and ordered by time they become a narrative of the intrusion.

Cybersecurity

DAY-66 The CIA Traid

- CIA (confidentiality, Integrity and Availability) are the three fundamental goals of information security, and almost every security measure serves one or more.
- **Confidentiality:** keeping information secret from those who shouldn't see it (passwords, encryption, file permission, preventing eaves dropping) - a breach is a leak or interception.
- **Integrity:** Keeping information accurate and unaltered (protecting files from modifications, verifying downloads weren't tempered with) - breach is tampered or altered data, and silently wrong data can be worse than stolen data.
- **Availability:** Keeping systems and information accessible to authorized users when needed (uptime, backups, resisting denial-of-service; ransomware is an availability attack)- security keeps good actors able to work, not only bad actors out.
- The CIA triad retroactively names all the security measures File permissions protect confidentiality and integrity; encryption and SSH the same; backups protect availability and integrity; updates all three; monitoring detects breaches of any.
- The attacks map on as violations: eavesdropping and theft break confidentiality, tampering and malware break integrity, denial-of-service and ransomware break availability.

- Every attack breaks one or more of the three; every defence protects them — the common language underneath all of security.
- It's the foundation of every security decision: professionals ask of any risk, control, incident which property is threatened, which they're protecting and the impact to each.
- The clearest tension is confidentiality versus availability: the most confidential system is one no one can access (useless), the most available is one everyone can reach (insecure) — real security lives between, and a system so secure no one can use it has failed, because availability is part of security.
- Other trade-offs: strong security adds friction that can push users to insecure workarounds; heavy integrity checking can cost availability
- Context decides the weighting — availability paramount for hospital life-support, confidentiality for secrets, integrity for bank records.There's no universal answer, only the right balance — contextual judgement, which real work prizes above memorised facts.
- Attacks are classified as which property they violate; risk management is deciding how much to protect each against threats with limited resources.
- It also gives you the professional's way of thinking and speaking — not "this attack is scary" but "this is a confidentiality breach" or "an availability attack" — the industry's precise shared language that signals real understanding.
- Think of a diary with a lock. Confidentiality is the lock keeping strangers out. Integrity is knowing no one has scribbled in or altered your entries. Availability is being able to find the key and open the diary when you want to write.

DAY- 67 Threats, Vulnerabilities, Risks

- Threat, vulnerability, and risk are three words of how organisations decide what to protect and spend on: assessing what to fix means reasoning about what the threats are, where you're vulnerable, and what the risk is. Confusing the words confuses the thinking.

- Threat is what could harm you; a vulnerability is the weakness that lets it.
- **A threat** is a potential cause of **harm**- attackers, but also fire, flood, hardware failure, a careless employee- the source of harm, existing independently of you and largely outside of your control.
- **A vulnerability** is a **weakness a** threat could exploit — an unpatched flaw, a weak password, an open port, a misconfigured permission, an untrained employee — something about YOU, your weak spot, and largely WITHIN your control.
- The key insight: harm happens when a threat meets a vulnerability. A threat with no matching vulnerability does nothing; a vulnerability with no threat sits harmless. Hardening works by removing your side of the combination.
- Risk is the potential for harm when a threat exploits a vulnerability — the combination of the two, weighed by likelihood and impact.
- High risk = likely AND damaging; low = improbable OR harmless.
- Risk is the word that drives decisions while threat and vulnerability are inputs: you can't fix everything, so you prioritise by risk — fix the likely, high-impact things first.
- There are four fundamental responses to any risk — the complete menu of risk management:
    - Reduce it : patch, add controls, harden to lower likelihood or impact- most security work.
    - Accept it: consciously decide it’s small enough to live with because fixing costs more than the harm-legitimate and necessary but with caution.
    - Transfer it: shift the impact elsewhere, classically cyber-insurance.
    - Avoid it: eliminate it by not doing the risky thing, reducing the attack surface.
- Choose by the risk’s size and each option’s cost. Mature security isn’t eliminating all risk- it’s deliberate, informed choices about each.
- The frame: faced with any situation, separate it — what are the threats (who/what could harm), where are the vulnerabilities (what weaknesses could they exploit),

what's the risk (likelihood times impact), and what do we do (reduce/accept/transfer/avoid)?

DAY- 68 Encryption Without the Math

- Encryption is one of the most misunderstood security tools- almost no professional implements encryption; they understand what it provides and uses it correctly, which is what security work requires.
- It matters because it is the primary way confidentiality is protected; nearly every secure system relies on it for messages, banking, passwords, stored data, connections.
- Encryption is scrambling information so only someone with the right key can unscramble it.
- The vocabulary: plaintext (the original readable information), ciphertext (the scrambled unreadable version), encryption (plaintext to ciphertext) , and decryption is the reverse of encryption.
- Modern algorithms are public — attackers know exactly how the scrambling works — and security comes entirely from the key being secret. A good system is secure even when the attacker knows the algorithm.
- Encryption protects confidentiality as anyone with the KEY cannot read it, and it also comes under integrity, as there is no tampering with data.
- Two standard contexts: encryption in transit protects data moving across a network (SSH, HTTPS, the browser padlock) against eavesdropping; encryption at rest protects stored data (files, disks, backups, phones) against theft — a stolen encrypted laptop reveals nothing.
- Encryption's central practical challenge is key management — since all security rests on the key, everything depends on keeping it secret from attackers while getting it to legitimate parties.

- Encryption's central practical challenge is key management — since all security rests on the key, everything depends on keeping it secret from attackers while getting it to legitimate parties.
- This is genuinely very hard as how do two people share a key without an eavesdropper intercepting it?- asymmetric encryption will explain this
- Key management — generating, distributing, storing, protecting, and retiring keys — is often where encryption succeeds or fails in the real world.
- There are few more limitations for encryption: it protects data not endpoints: a compromised computer reads your decrypted data. If the key leaks: an encrypted connection to a vulnerable server still reaches the vulnerable server, this could be used for ransomware also.
- encryption scrambles information with a key so only the key-holder reads it; protects confidentiality (primarily) and supports integrity; applies in transit and at rest; rests on the secret key not the public method; its hard problem is key management; and it has clear limits (endpoints, leaked keys).

DAY-69 Symmetric and Asymmetric Encryption

- Symmetric encryption uses the same single key to both encrypt and decrypt — both sides use the identical key; like an everyday lock where one key both locks and unlocks
- Its major strength is speed: it's computationally efficient and can encrypt and decrypt large amounts of data quickly, making it the workhorse for bulk data — whole disks, large files, the stream of a network connection.
- the key-sharing problem — both parties need the same key, and getting it to the other securely over an untrusted network is the hard, circular problem
- Asymmetric encryption uses a pair of mathematically-linked keys — public and private — that do different things: what one locks, only the other unlocks.
- You keep the private key secret forever; you give out the public key freely. Anything encrypted with your public key can only be decrypted with your private

key — so anyone can encrypt to you using your freely-available public key, and only you can decrypt, because only you hold the private key.

- This dissolves the key-sharing problem: the shared thing (public key) needn't be secret, the secret thing (private key) never travels — so you can communicate securely with a stranger.
- This is exactly like the SSH key pair, and it powers the secure web, encrypted web, encrypted email and digital signatures.
- Asymmetric solves key-sharing but is slow (impractical for bulk data); symmetric is fast and perfect for bulk data but can't solve key-sharing alone.
- Use asymmetric encryption to securely share a symmetric key, then use that shared key with fast symmetric encryption for the actual data, this is the model behind nearly every secure connection- when you browser connects securely, the two sides use asymmetric techniques to agree to fast symmetric encryption for the conversation.

DAY-70 Hashing

- It is one-way math that can never be reversed, where encryption is reversible.
- It does two jobs for which being one-way is the whole point:verifying information hasn't changed (integrity) and protecting passwords.
- **A hash function** takes any input — a word, a file, a whole database — and produces a fixed-length fingerprint (the hash or digest).
- Defining properties:
    - deterministic (same input always gives the same hash, enabling verification),
    - fixed-length (any input size gives the same hash size),
    - one-way (easy to compute the hash from the input, infeasible to go backward — the defining, deliberate property)
    - collision-resistant (two different inputs almost never produce the same hash, so a hash reliably identifies its input).

- The only limitation: change the input even one character and the hash changes completely — making hashing a sensitive tamper detector.
- Unlike encryption, a hash using no key has no way back- a different tool, not a worse one.
- The first main use of hashing is verifying integrity: have the original's rehash, re hash later, compare- match means unchanged, difference means altered.
- Downloading software: the makers publish the genuine file's hash (a checksum/SHA-256 this is how it is displayed mostly), and the only limitation catches even a single altered byte.
- File integrity monitoring detects altered system files, forensic investigators hash evidence to prove it's unaltered, backups are hash-verified.
- Hashing's second great use protects passwords. The problem: a site needs to check your password but storing actual passwords is dangerous.The solution: store the hash of your password, not the password; at login, hash what you typed and compare the stored hash — verified without ever storing the password.
- Nuance: attackers crack hashes by guessing (hashing common passwords and comparing), so weak passwords are still vulnerable and strong ones still matter; salting(adding random data to the password) and slow hashing(makes it computationally expensive for each guess) strengthen protection.
- The clean distinction: encryption is two-way and about secrecy (uses a key, reversible by the key-holder, protects confidentiality); hashing is one-way and about verification (no key, irreversible, protects integrity and passwords).

DAY-71 The Kill Chain

- Real attacks are campaigns with distinct phases over days to months: research, prepare tools, find a way in, get in, establish a foothold, set up remote control, act on the goal.
- Each stage is a separate chance for a defender to detect and stop it. The kill chain names these stages.

- The best known kill chain has seven stages, and they trace an attack from the attacker's first research to their final goal:
    - **Reconnaissance:** The attacker researches the target.
    - **Weaponization:** They build the attack tool, combining an exploit with a payload.
    - **Delivery:** Getting the weapon to the target (phishing email, malicious link)- often targeting the humans.
    - **Exploitation:** The weapon triggers, exploiting a vulnerability to run code- the food in the door.
    - **Installation:** establishing lasting presence/persistence (rouge accounts, corn jobs).
    - **Command and Control:** The malware phones homes for remote control.
    - **Action on objectives:** achieving the actual goal- stealing, encrypting, spying.
- Because the attack is a chain of stages, you can break it at ANY stage, and breaking it anywhere stops the attack- the attacker must succeed at every stage, the defender only needs to stop them at one.
- Block delivery (spam filter), prevent exploitation (patch the vulnerability), detect installation (spot persistence), block C2(cut beaconing), catch actions on objectives (stop the theft).
- This is the best reason why defence in depth works: layered defences across stages mean the attacker must beat all of them. Since no single defence is perfect we just need to break the chain somewhere.
- The kill chain is a practical thinking tool organising three core activities.
    - **Detection:** It tells defenders what to watch for at each stage- reconnaissance (scanning, probing), delivery (phishing), exploitation(code execution), installation(persistence), C2(beaconing), actions on objectives(data leaving), so monitoring covers many stages and an attack mussed at one is caught at another.

    - **Response:** This stage gauges severity- a blocked phishing email is routine, active C2 is serious, data being stolen is a crisis- exactly how responders triage, and it shows what's already happened and what's next.
    - **Analysis:** Reconstructing the attacker's path through stages tells the complete story and reveals where defences failed.

DAY - 72 MITRE ATT&CK

- MITRE ATT&CK zooms in: a framework cataloguing the specific techniques attackers really use at every stage, one of the most important and widely-used resources in the industry.
- ATT&CK has hundreds of documented techniques. What makes it extraordinary: it's not theoretical but a knowledge base built from real-world observations of how actual attackers behave, organised into a comprehensive public catalogue maintained by the non-profit MITRE
- The security community's collective knowledge of how attackers operate, written down.
- ATT&CK is organised around tactics (the why — an attacker's goal at a stage) and techniques (the how — a specific method to achieve it).
- Tactics read like a refined kill chain: initial access, execution, persistence, privilege escalation, defence evasion, credential access, discovery, lateral movement, collection, command and control, exfiltration, and impact — each a category of attacker intent.
- Under each tactic sit many techniques, concrete documented methods: under persistence are creating a scheduled task and creating a new account; under defence evasion are ways to hide.
- The technique is the move; the tactic is the purpose. This two-level structure organises the overwhelming variety of attacker behaviour into a navigable map — and gives you the language professionals use to describe attacks.

- ATT&CT is a working tool shaping the real defence:
    - **Detection Coverage**: go through it technique by technique asking "would we detect this?" — turning "are we secure?" into a concrete audit that reveals blind spots to close; teams build coverage maps
    - **Common language**: standard names and identifiers give the whole industry a shared vocabulary, so threat reports, tools, and teams describe attacker behaviour precisely and check their defences against named techniques.
    - **Threat intelligence and emulation**: ATT&CK documents which techniques specific real attacker groups use (so you defend against the groups likely to target you), and red teams (offensive testers) use it to emulate real techniques while blue teams (defenders) check they'd catch them — structured, meaningful testing against the techniques real attackers actually use.
- The kill chain is high-level strategy — the broad stages every attack moves through, simple and memorable, great for the big picture(where is the attack, where can we break it). ATT&CK is detailed tactics and techniques — the specific methods within those stages, comprehensive and granular, great for specifics(exactly how might they do it, can we detect it?)
- The kill chain says the attack reached "installation"; ATT&CK names the specific persistence techniques to look for. Using both: the kill chain to reason about overall flow and where to focus, ATT&CK to get specific and build detections.
- MITRE ATT&CK matters directly for a security career — it's everywhere in the industry: security tools are built around it (organising alerts by technique), threat reports use it.
- Take away the shape, not a memorised catalogue: what it is (a real-world knowledge base of attacker techniques), how it's structured (tactics as goals, techniques as methods), and how it's used (detection coverage, common language, threat intel, red/blue team testing)

DAY - 73 Map a Real Breach

- what analysts do: gather the public reporting on a breach, then map it to a framework to understand the attacker's path start to finish — how they got in, what they did at each stage, where defences failed — turning a confusing news story into a clear structured understanding
- The analytical work is assigning each narrative part to its kill-chain stage. Research (website, employees, email format) = reconnaissance. Crafting the phishing email with the malicious attachment = weaponization. Sending it to finance staff = delivery.
- The attachment exploiting outdated software = exploitation (the foot in the door). The backdoor and hidden scheduled task = installation/persistence. The machine connecting out to the attacker's server = command and control. Copying the database out = actions on objectives (the goal)
- Laid out as ordered stages, the narrative becomes a structured map of the attacker's path — and opens the key question: where could it have been broken?
- The second, more valuable half of the analysis is asking at each stage "where could this have been stopped?" — producing actionable insight and showing you think like a defender.
- At delivery: email filtering or a trained employee catches the phishing. At exploitation: patched software (Day 60) makes the exploit fail At installation: persistence monitoring (Day 62) detects the backdoor. At command and control: beaconing detection (Day 58) catches the outbound connection. At actions on objectives: monitoring for large data transfers catches the theft.
- The power: the attack offered many chances to stop it, at nearly every stage — it succeeded only because defenders missed every one.
- a real attack and analysed it like a professional — mapped it to a framework, traced the attacker's path through the stages, and identified where the chain could have broken. That's the core skill of attack analysis, central to SOC analysts, incident responders, and threat analysts.

DAY- 74 Threat Actors

- Behind every attack is someone who chose to attack for their own reasons, and their identity and motivation shape everything — what they target, what techniques they use, how long they persist, how you defend.
- Not all the attackers are the same: a teenager defacing a site for fun, a criminal gang after ransomware profit, and a nation-state stealing secrets are radically different adversaries with different goals, skills and practice.
- The main threat actor types, distinguished by motivation and capability:
    - **Cybercriminals:** The most common, motivated by money(ransomware, fraud, stealing/selling data), often organised like businesses, broadly and opportunistically targeting easy victims ; most attacks an ordinary organisation faces.
    - **Nation-state actors** — governments, the most capable and best-resourced, motivated by espionage and sabotage; patient and sophisticated (called Advanced Persistent Threats / APTs), pursuing specific targets for months without giving up.
    - **Hacktivists** — motivated by ideology/cause not money, targeting organisations they object to (defacement, leaks, disruption).
    - **Insiders** — people with legitimate access (employees, contractors) misusing it from grievance, profit, or being tricked; dangerous because they're already inside the perimeter.
    - **Script kiddies** — unskilled attackers using others' tools for mischief or notoriety; low-capability but numerous, why even uninteresting targets get swept by automated attacks.
- The capability spectrum: threat actors range from numerous low-skill script kiddies and opportunistic criminals running automated attacks, up to elite nation-state teams with years of patience — but the vast majority of attacks are low-to-medium sophistication, because that's most of the volume.

- So basic security hygiene (hardening, patching, good passwords, awareness) defeats the large majority of attacks.
- A realistic view: most individuals and ordinary organisations are overwhelmingly targeted by cybercriminals and automated attacks, not nation-states (which concentrate on high-value targets), so defending well against common opportunistic threats is the right priority — and solid fundamentals defeat exactly those.
- An attack is technical steps chosen by a specific adversary pursuing a specific goal — understanding both mechanism and motive is understanding attacks fully; when analysing an incident you can ask not just what did they do? But who was this and what did they want?.

DAY- 75 Common Vulnerabilities

- the famous named categories — injection, cross-site scripting (XSS), remote code execution (RCE) — explained conceptually.
- A unifying idea makes them click: most share one root cause — a system trusting input or data it shouldn't, treating untrusted data as a command to run, code to execute, or content to display, when it should have treated it as mere input to be checked first.
- The most famous vulnerability category is **injection,** best known as **SQL injection**. The concept: applications take user input (a search box, a login form) and use it to build a command or query sent to another system like a database.
- The vulnerability arises when the application mixes the user's input directly into that command without separating the two — a clever attacker crafts input so part of it is interpreted not as data but as part of the command itself, "injecting" their own instructions.
- SQL injection applies this to database queries: crafted input alters the query, making the database reveal data it shouldn't, bypass a login, or destroy data — often the cause when "attackers stole the whole database through the website.

- It's the trusting-untrusted-input root cause; the defence is to keep user input always as data, never letting it become command.
- Cross-site scripting (XSS) is injection aimed at web pages and their viewers: a site takes input from one user and displays it to others without proper handling, so an attacker's malicious code runs in other users' browsers (stealing sessions, impersonating, redirecting).
- Where SQL injection attacks the database, XSS attacks the users — same root mistake, untrusted input treated as trusted content
- Remote code execution (RCE) is the most severe: an attacker runs their own code on a system remotely, near the worst case because they can then do anything the system can — the holy grail for attackers, frequently the kill-chain exploitation step, and the one that triggers emergency patch rushes.
- These three (injection, XSS, RCE) are among the most-referenced categories; understanding them and the trusting-untrusted-input theme gives real literacy.
- They persist because software is enormous, complex, and human-built under pressure: a large application has countless places handling input, and one spot where a developer forgot to separate data from command is enough for a vulnerability.
- For you as a defender: you needn't write secure code, but you must recognise these categories, understand why apps get breached, and appreciate the defences (secure coding, testing, patching, never trusting input).

DAY- 76 Malware

- Malware means malicious software — any software written to do harm. This is further categorised into families, based on what it does and how it spreads.
- How does it spread?- distinguishes three classic terms.
    - A virus attaches to a legitimate file or program and spreads when that host is shared and run — it can't spread alone, needing a human to open or run the infected host (parasitic, like a biological virus needing a host cell)

    - A worm spreads by itself, automatically, across networks, without a host or human action — exploiting vulnerabilities to jump machine to machine and multiply at astonishing speed; this autonomy makes worms cause network-wide disasters in minutes, the most devastating fast outbreaks in history
    - A trojan (Trojan horse) is malware disguised as something legitimate and desirable, tricking the user into installing it themselves through deception (the "just paste this to install" trick, Day 60; the disguised invoice, Day 73).
    - Viruses need human actions, worms self-spread, trojan deceives.
- What does it do? gives us the family names that describe malware purposes.
    - Ransomware encrypts the victim's data and demands payment for the key, weaponising encryption and attacking availability by denying you your own data — it has paralysed hospitals and companies.
    - Spyware secretly gathers information (passwords, secrets), attacking confidentiality; a keylogger records every keystroke, and spyware's defining feature is stealth — staying hidden to keep stealing
    - A botnet enslaves many computers under remote control for spam or denial-of-service attacks.
    - A rootkit hides deeply to conceal its and other malware's presence.
    - Adware bombards with unwanted ads
- These names describe purpose, independent of how the malware arrived.Real-world malware is messier than a tidy list.
- First, it's usually a blend — a single specimen might spread like a worm, disguise like a trojan, hide like a rootkit, steal like spyware, and deploy ransomware; the two-question lens (how it spreads, what it does) beats a single label because real malware has answers to both, often several
- Second, the delivery-versus-payload distinction clarifies it: a delivery mechanism (how it gets on and spreads) plus a payload (what it does), mapping onto the kill

chain's delivery/installation stages and actions-on-objectives — analysts say "a trojan delivery installed a ransomware payload."

- Third, malware connects everything: it's the suspicious process, the network connection, the persistence you'd detect, and defending against it uses patching, not running untrusted programs, backups, and layered defences.
- You can now name and distinguish the malware families precisely: "ransomware" means an availability attack via malicious encryption demanding payment; "a worm spread across the network" means automatic propagation exploiting vulnerabilities without anyone clicking; "spyware" means stealthy information theft.

DAY-77 Defence in depth

- Defence, beginning with its most important principle: defence in depth — never rely on a single layer; layer many, so if one fails, others still protect you.
- No single defence is perfect — firewalls get misconfigured, passwords stolen, employees fooled, patches missed. Stake everything on one and the day it fails you're exposed. So build many imperfect layers, each catching what others miss, so an attacker must defeat all of them.
- The goal isn't one perfect wall (impossible) but depth — many layers whose weaknesses don't overlap, and the three metaphors make defence in depth vivid.
    - **The castle:** layers of moat, outer wall, inner wall, keep — breaching one just brings the attacker to the next, never to victory.
    - **The onion:** peel one layer, another's beneath, all the way to the valuable centre.
    - **The Swiss cheese** model is the most sophisticated and explains WHY imperfect layers work: each layer is a slice full of holes (no layer is perfect), but stacked, the holes don't line up — for an attack to get all the way through, it must pass a hole in every slice, all aligned, far less likely than slipping through one.

- Defensive depth layers in several dimensions.
    - By location, outside-in (the onion): physical, network (firewalls/segmentation), host (hardening), application (against vulnerabilities), and data security (encryption) — each a barrier, so a breached network still leaves hosts protected and a fallen host still leaves encrypted data unreadable.
    - By function, layers in time: prevention (stop attacks), detection (notice what gets past — monitoring/logs), response (act on it) — you need all three because prevention sometimes fails, and without detection an attacker who gets in operates unseen.
    - By people, process, and technology: security isn't only technical — a brilliant tool undone by an untrained employee clicking a link shows why all three matter. Richer layering means no single failure is catastrophic.
- Defence in depth operationalises the kill chain: since an attack must traverse every stage while you need to stop it at only one, you place defensive layers at multiple stages.
- The two ideas reinforce: the attacker must pass a defence at every stage to succeed; you need one defence anywhere to work. It's the Swiss cheese model in attack-lifecycle form.
- This is why detection layers and "assume breach" matter: prevention sometimes fails, so build detection and response deep inside to catch attackers who get past the front door — the same thinking as zero trust.
- The question is never "which single defence?" but "do we have enough layers that we're protected even when some fail?" It also gives the professional's mindset: don't expect any defence to be perfect, don't panic when one fails or chase an impenetrable system — build depth, assume some layers fail, ensure others catch what gets through.

DAY-78 Vulnerability Management

- Vulnerabilities are everywhere and keeping software patched is the top defence, but there's a practical problem: an enormous number exist, with thousands of new ones published yearly across the hundreds or thousands of software pieces an organisation runs.
- **Vulnerability management**: the systematic process to find, assess, prioritise, and fix vulnerabilities, plus the global CVE system that lets the world track them consistently, solves the above problem.
- Since you cannot fix everything at once, so prioritise by risk, fixing the most dangerous first.
- CVE stands for **Common Vulnerabilities and Exposures**, and it's a global system that gives every publicly known vulnerability a unique identifier, so that everyone — researchers, vendors, defenders, tools — can refer to the same vulnerability unambiguously.
- An ID looks like CVE-2024-12345 and identifies one specific vulnerability in one product forever — so when a vendor patches it, a tool reports it, and an article discusses it, all unambiguously mean the same weakness.
- The value is the same as any common language without unique IDs, people described the same vulnerability inconsistently and couldn’t coordinate with them, scanners, vendors, and defenders.
- CVSS (Common Vulnerability Scoring System) scores each vulnerability 0 to 10 (higher is worse), reflecting how easy it is to exploit and how much damage it could do, grouped into bands: critical, high, medium, low — telling you at a glance how much to worry.
- The CVSS score is NOT the whole story of YOUR risk — it's general severity, but your actual risk depends on context (do you run it, is it exposed, is it valuable?). A critical CVE in software you don't use is no risk; a medium one in your internet-facing database may outrank a critical one on an isolated test box. Real risk is severity weighed by your context.

- The lifecycle of vulnerability management:
    - **Discover:** find what vulnerabilities exist in your environment, largely via vulnerability scanners that check your systems against the known-vulnerability catalogue and report which CVEs affect which systems
    - **Assess and prioritise:** judge how much each matters using CVSS severity AND your context, producing a prioritised list (the heart of the discipline, since you can't fix everything).
    - **Remediate:** fix the prioritised ones, usually by patching, sometimes by configuration changes, workarounds, or accepting low risk.
    - Verify: re-scan to confirm the fix worked. Then repeat — endlessly, because new vulnerabilities appear constantly and environments change
- It's never "done"; it's perpetual maintenance, much of what security roles do daily, and where a huge fraction of breaches are quietly prevented.
- Also understand the crucial insight that severity is not the same as your risk — that prioritising requires weighing a vulnerability's general severity against your specific context.
- The central skill is prioritisation — deciding what to fix first when you can't fix everything — where risk thinking becomes concrete: high severity plus relevant plus exposed plus valuable equals fix first.

DAY - 79 Patching

- Keeping software patched is the top security practice and the main remediation, and it sounds trivially simple — vulnerability found, vendor releases a fix, you install it.
- Yet breaches keep happening through known, already-patched vulnerabilities whose fixes weren't applied.
- **A patch** is an update from a software maker fixing problems including security vulnerabilities; applying it closes the weakness so attackers can't exploit it.

- It sounds simple in a laptop’s context just by clicking on update, but difficulty appears at scale and in complex environments.
- The gap between patch exists and patch applied lives most of the breaches as through scaling it faces genuine, difficult friction, which is why patching your systems is most-repeated yet most-unfollowed advice in security.
- The reasons patching is hard at scale are real and worth understanding, because they explain the chronic gap:
    - **Patches can break things**:— a patch changes software, and changes can break a depended-on feature, conflict with other software, or cause an outage; this fear is the biggest reason for caution, so organisations test patches first, leaving the vulnerability open during testing (patch fast and risk the breakage or test and stay vulnerable longer).
    - **Patches cause downtime** — many require a restart, and taking critical always-on systems (hospitals, factories, banks) offline is disruptive and costly.
    - **Scale and inventory** — thousands of systems and countless software make patching everything an enormous logistical task, and you can't patch what you don't know you have.
    - **Legacy and unpatchable systems** — unsupported old software (no patches made), or specialised equipment that can't be updated — are persistent headaches with no applicable fix.
- The patch window is the race between a vulnerability becoming known and getting it patched before attackers exploit it.
- When a vulnerability is disclosed and a patch released, a clock starts: defenders rush to apply it, attackers rush to exploit unpatched systems — and once it's public, attackers hunt for the unpatched.
- The window between a patch's release and your applying it is known, exploitable danger; the faster you patch, the smaller it is. The race is often brutally fast —

attackers can exploit within hours, sometimes faster than organisations can test and deploy.

- A zero-day is a vulnerability exploited before any patch exists (the vendor had "zero days") — the nightmare case, since you can't patch your way out. But most attacks exploit KNOWN vulnerabilities with available patches that weren’t applied- the common, preventable problem.
- The core discipline is test, then deploy — but quickly: test a patch on a few systems to catch breakage, then roll out once proven, keeping the patch window small (not skipping testing, not testing so slowly you stay vulnerable for weeks).
- Prioritisation applies as in vulnerability management: critical security patches for actively-exploited vulnerabilities on exposed systems get fast tracked, while routine patches follow scheduled patch windows.
- When you genuinely can't patch (legacy/unpatchable systems), you apply compensating controls — defence in depth to the rescue: isolate the system on a separate network segment, add monitoring, restrict access — lowering the risk without fixing the vulnerability itself.
- The encouraging bottom line: patching the known vulnerabilities that cause most breaches is achievable and the highest-value work most organisations can do.

DAY- 80 Triage Vulnerabilities

- The central skill running through vulnerability handling is prioritisation — deciding what to fix first when you can't fix everything so triage a realistic list of vulnerabilities into a defensible fix-first order using severity and context,
- Vulnerability triage has more vulnerabilities than you can fix at once, so assess which pose the greatest risk and fix those first, the skill here is not fixing but deciding the order, where the real judgement lives: get it right and you reduce the most risk with limited time.
- Triage judges each vulnerability's real risk to your specific environment — severity weighed by context

    - Severity: is the starting point, not the whole story
    - Exposure is often the most important: can an attacker actually reach it, an internal facing vulnerability is far more urgent than the same one isolated internally. Exposure drives likelihood.
    - Value/impact: what does the system do, what's lost if compromised? (customer database outranks a disposable test machine- the impact half of risk.
    - Active exploitation: is it being attacked in the wild now? Dramatically more urgent- the patch window is already running.
    - Patch availability: is there a ready fix, or is it unpatchable or needing compensating controls.
- Real risk = severity weighed by exposure, value, active exploitation, and patch availability.
- Two things that justifies a triage are first, placement: the value isn't the ranking but the reasoning: for each vulnerability, say why it's where it is. Second, be honest about trade-offs: you're consciously accepting risk on lower-priority items, some may need compensating controls not patches, and the order shifts as circumstances change.
- Be honest about trade-offs: you're consciously accepting risk on lower-priority items, some may need compensating controls not patches, and the order shifts as circumstances change.

Day-81 Incident Response

- Incident Response is a structured process teams follow when an attack gets through as no defence is perfect.
- The core insight: responding well requires a process, not panic. A serious incident is high-pressure (data leaving, systems down, minutes matter), and frantic reaction causes mistakes — destroying evidence, missing part of the attack, tipping off the attacker, causing more damage.

- The first three incident response phases get from readiness through detection to stopping the spread.
    - **Preparation** (before any incident): the plan, roles, tools, training, and rehearsal done in advance — the best response executes a calm plan rather than inventing one mid-crisis; unglamorous but foundational (the same wisdom as hardening and assume-breach).
    - **Identification/detection**: recognising an incident is happening (an alert fires, monitoring flags something, a user reports it) and confirming it's real and assessing its scope — where your investigation skills and monitoring come in.
    - **Containment:** Stopping the incident spreading or causing more damage- isolating affected systems, blocking access, limiting harm. The crucial tension: contain fast but preserve evidence- don’t wipe a machine and destroy how it happened.
    - **Eradication** removes the threat completely — malware, the exploited vulnerability, created accounts, and crucially every persistence mechanism, because missing even one means the attacker returns and you're breached again (fully evicting them requires finding every foothold).
    - **Recovery** restores systems to normal from clean backups (why backups matter so much), cautiously confirming the threat is gone before reconnecting, restoring the availability the incident disrupted.
    - **Lessons Learned** — the most-skipped, most-valuable phase — turns the painful experience into improvement: what happened, how did they get in, what do we change? The kill-chain analysis lives here, revealing where defences failed. It feeds back into preparation, closing the loop so each incident makes the organisation stronger.
- Speed versus evidence: act fast to limit damage, but acting too fast destroys the evidence needed to understand the incident and fully eradicate (wipe a server and

you stop the attacker but lose how they got in — and might miss a persistence mechanism); responders isolate rather than wipe, capturing state first

- Containment versus availability: isolating compromised systems stops the attack but takes them offline, trading availability for security — weighed against operational cost, sometimes partial containment over total shutdown.
- Eradicate fully versus restore quickly: the pressure to get back to normal pushes against thorough eradication, but restoring into a still-compromised state means you're breached again. These tensions are why preparation and a clear process matter — they keep fast trade-offs sound.

DAY-82 OSNIT On Yourself

- The term to know is OSNIT- open-source intelligence, which means gathering information about a target from publicly available sources.
- Open-source here doesn't mean software; it means openly available information- anything anyone can find without breaking in: social media, public websites, search engines, public records, leaked data that's circulating, and more.
- Done on a target as an attack's first step, OSNIT is exactly the kill chain's reconnaissance stage.
- It matters two ways:it completes your understanding of reconnaissance by showing how much is learnable from public sources before any technical attack, and doing it on yourself reveals your actual exposure so you can reduce and manage it — turning the attacker's own technique into self-defence.
- The primary use is targeting and personalising attacks, especially phishing: by researching you (employer, colleagues, interests, recent events, services used), an attacker crafts a far more believable lure — spear-phishing, built on OSINT.
- OSINT also enables password resets (security question answers like pet's name are found through reconnaissance), and is also helpful to plan a border attack.
- The main categories of findable information give you a checklist for examining your own footprint.

    - **Social media** is usually the richest source — your profiles and others' reveal where you live and work, relationships, interests, routines, photos showing location and habits; even if yours are locked down, you appear in others' public posts.
    - **Data breaches** are major and overlooked: countless services have been breached and the leaked data (emails, passwords, details) circulates and is searchable — there's a good chance your email appears in known breaches, and a leaked reused password is directly exploitable (you can and should check).
    - **Public records** and search results surface old accounts, posts, profiles, and records by simply searching your name, email, and usernames. Metadata is the subtle one — location data in photos, hidden details in documents.
- OSINT on yourself is meant to drive action, not alarm. reduce what you can: lock down social media and stop oversharing location/routine/security-question answers;
- Check whether your email is in known breaches and stop using any leaked passwords (never reuse passwords — the breach makes the danger concrete); close old unused accounts; mind photo/document metadata.

DAY- 83 AI as a Threat

- How AI is being used to make attacks more convincing, faster, and more widespread. It fundamentally does not create an attack, it amplifies it by making them better- more convincing, faster to produce, and easier to scale.
- AI raises the bar in specific, understandable ways, but the fundamentals remain the foundation of defence — clear-eyed realism, neither dismissing nor fearing it.
- The first and most significant attacker use of AI is making attacks more convincing, hitting hardest where the human is the target — phishing and social engineering.

- Classic phishing calls (bad grammar, awkward phrasing, clumsy wording) were reliable partly because producing polished, fluent, personalised text was hard. AI generates fluent natural text effortlessly at scale, so the bad-writing tell is weakening.
- AI also makes personalisation easy at scale, sharpening spear-phishing auto-generating a tailored, convincing message per target at previously impossible volume — fluent writing plus easy personalisation means more convincing, targeted phishing, faster and in greater volume.
- AI further enables convincing impersonation via generated media (deepfakes — fake voice, images, video). The defence implication: signals based on quality and polish are weakening, so verification must rely on separate trusted channels and scepticism of urgency, not on whether something looks or sounds right.
- AI rapidly gathers, sorts, and summarises public information, assembling a profile far faster than manual work and across many targets — so the kill chain's first stage becomes faster and more scalable, feeding the personalised attacks.
- Greater scale through automation/generation: producing many phishing variations, generating content, assisting with code — the same attacks but more of them, more cheaply, reaching more targets. (A hundred-times cheaper attack is a bigger threat even if each is the same.)
- Lower skill bar: AI assists attacks that once needed expertise, raising what unskilled attackers (the script kiddies) can do — though with real limits (it doesn't make a novice elite; sophisticated attacks still need expertise).
- AI-written phishing is still phishing, defended by verifying through trusted channels, scepticism of urgency, not clicking unexpected links, and multi-factor authentication
- AI-accelerated reconnaissance feeds attacks that patching, hardening, and defence in depth still address. The mature takeaway: adapt where AI changes things (rely less on weakened signals like writing quality, more on robust verification), and

double down on fundamentals (they matter more than ever). You’re not starting over applying what you know to a sharper version of the same game.

DAY- 84 AI as a Defender

- Defenders (security teams, SOC analysts, incident responders) use AI too, amplifying it, making familiar tasks faster, more thorough, and more effective — so your defensive knowledge isn't obsolete either.
- Defenders genuinely need this because they face a crushing volume problem: monitoring enormous data far beyond human hand-examination, finding rare attack signs in the flood, while keeping up with a rising tide of attacks — exactly where AI helps most.
- The single most valuable thing AI does for defenders is find threats hidden in overwhelming data — the needle in the haystack — directly addressing the volume problem.
- Through the investigation work it shows that across a whole organisation means millions of entries daily no human can read, so attacks hide in the flood.AI examines that volume far faster, looking for patterns and anomalies and surfacing the suspicious few for human investigation — it doesn't tire or skip data.
- The key method is anomaly detection: AI learns the organisation’s normal activity, then flags deviations automatically at massive scale.
- This is transforming the SOC (security operations centre): AI does the first-pass sifting so analysts focus where it counts, helping defenders keep up with a volume of threats that has outgrown what humans can manage alone.
- AI helps defenders across their work. It accelerates investigation and analysis: rapidly pulling together related information, summarising large data, helping explain what suspicious code or activity does.
- The breach-analysis and investigation skills are accelerated, not replaced; the analyst keeps the judgement while AI handles time-consuming gathering, which matters most during an incident where speed is critical.

- It helps with response and routine work: partly automating clear-cut actions (isolating an obviously compromised system), and sifting the flood of mostly-harmless alerts so analysts aren't buried — addressing real alert fatigue and burnout.
- AI assists the human defender, it doesn't replace them. AI makes mistakes (false positives, missed threats) needing human interpretation; it lacks full context (the business, what an alert really means); and critical decisions (is this a real incident, how serious, what to do, acceptable risk) require human judgement and accountability AI lacks.
- AI in security: attackers use it to amplify attacks (more convincing, faster, scalable, accessible), defenders use it to amplify defences (handling volume, finding threats faster, accelerating analysis and response)

DAY-85 Compliance

- Real-world security happens inside a web of rules, laws, and standards — compliance — one of the biggest forces shaping security work, far bigger than beginners expect.
- **Compliance** means meeting the rules you're required to follow: the laws, regulations, and standards applying to your organisation, which differ by what you do and where (handling personal data means data-protection laws; a hospital, health data rules; taking payments, payment-card standards.
- Compliance exists for genuine reasons,**Protecting people:** organisations hold vast personal, health, and financial data, and people are harmed when it's mishandled (identity theft, fraud, exposure) — but left alone, some organisations would underinvest, because protection costs them while breach-harm falls on individuals; regulations correct this by making protection a legal obligation.
- **Establishing a baseline:** without required standards, security would vary wildly and you'd have no assurance any organisation protects your data adequately —

regulations set a minimum bar everyone subject to them must meet, raising the floor.

- **Accountability and trust:** compliance demands you can demonstrate protection (documentation, audits, evidence) and face penalties (fines) if you don't — the threat of penalties is a large part of why organisations take security seriously, and it lets customers and regulators trust that a compliant organisation meets the standard.
- There are many specific regulations and standards, and they vary by country and industry, but they fall into a few understandable categories based on what they protect. Here are the main kind, with well-known examples:
    - **Personal-data** privacy laws protect individuals' personal information generally — most famously GDPR (the EU's data-protection law), strict on collecting/using/storing/protecting personal data with big breach penalties, influencing laws worldwide; the reason for privacy notices and consent requests everywhere.
    - **Sector-specific r**egulations protect especially sensitive data in particular industries — health (HIPAA in the US) and financial (SOX) information — because the stakes are high.
    - **Payment and industry standards** tie to activities — PCI-DSS (the Payment Card Industry Data Security Standard), mandatory for handling card payments (an industry standard, not a government law, but enforced)
    - **Security frameworks** (ISO 27001, national cybersecurity frameworks) are structured best-practice sets organisations adopt to organise and demonstrate security.
- Compliance and security overlap but are not the same — grasping the distinction is mature thinking. The key insight: compliance doesn't guarantee security, and security doesn't guarantee compliance.
- An organisation can be fully compliant (passing every audit) yet insecure, because compliance is a minimum, often generic baseline — box-ticking to pass without

addressing your actual threats still gets breached ("we passed the audit" isn't "we're secure").Conversely, strong security can fail compliance on a technicality (missing documentation).

- Compliance is a floor, not a ceiling. But don't swing to cynicism: compliance genuinely raises a baseline many organisations wouldn't achieve otherwise.
- Many roles exist wholly or partly to achieve and maintain compliance (implementing required controls, passing audits, documenting that requirements are met), and "we need to be compliant with X"
- Practically: knowing the major categories (privacy laws, sector-specific rules, payment standards, frameworks) helps you understand any organisation's regulatory context and security priorities;

DAY- 86 Risk Management

- Risk management sits above everything you've learned and gives it purpose — identifying what could go wrong, deciding how much each risk matters, and choosing what to do.
- The foundational truth: security doesn't exist for its own sake; it exists to manage risk, and every control, defence, and compliance requirement is ultimately a way of reducing risk to an acceptable level.
- The goal isn't to "be secure" by stopping all attacks (impossible, and not the actual goal) but to manage risk to an acceptable level given limited resources — not zero risk but acceptable risk, reducing the most significant risks enough, at justifiable cost, that what remains is tolerable.
- Organisational risk management is a structured, repeating cycle, covering all risks.
    - **Identify risks**: work out what's worth protecting (valuable assets) and what threatens them, using threat-modelling thinking— you can't manage risks you haven't identified, and it's ongoing as new risks emerge

    - **Assess risks**: judge how significant each is using likelihood times impact, ranking them so major risks get attention over minor ones — the same prioritisation as the triage, applied organisation-wide.
    - **Respond to risks:** decide what to do about each (the heart of it).
    - **Monitor and review**: risk management is continuous because the landscape constantly changes, so organisations cycle back through identification and assessment over time.
- There are exactly four ways to respond to a risk, and choosing wisely among them is much of risk management.
    - **Mitigate (reduce):** lower the risk's likelihood or impact — most security work (patching, hardening, defence in depth, training), the default for significant risks you can act on, but one option among four.
    - **Accept:** consciously live with it because it's low enough or reducing it costs more than it's worth — a deliberate, documented, owned decision, not negligence.
    - **Transfer:** shift the risk or its financial impact elsewhere — classically cyber insurance, or outsourcing to a specialist.
    - **Avoid**: eliminate the risk by not doing the risky thing — discontinuing an activity, not collecting data you don't need (data you don't hold can't be breached).
- Security is fundamentally a business decision under constraints, balancing risk against cost and competing priorities. Resources are limited and security competes with everything else (product, operations, growth) — every pound on security is a pound not spent elsewhere, so it must be justified: does the risk it reduces warrant the cost?
- You can't "do all the security" — beyond a point, more costs more than the risk it removes is worth; risk management spends limited resources where they reduce the most risk per pound.

- And how much risk to accept — the risk appetite — is a leadership/business decision, varying rationally by organisation (a bank accepts less than a startup); the security team identifies, assesses, and recommends, but leadership decides.

DAY-87 The Job Landscape.

- The largest part of the field, and where you'd likely begin, is defensive security — the blue team: protecting organisations by monitoring for threats, detecting and responding to attacks, keeping systems secure. Most security jobs are defensive and most newcomers start here.
- The classic entry-level role is the SOC analyst (security operations centre) — monitoring systems for attack signs, investigating alerts, triaging serious from noise, escalating or responding (examining logs, spotting the unusual, triaging by risk, recognising techniques, understanding incidents).
- Offensive security (the red team) is what people picture as "hacking" but is a smaller share than defence: attacking systems deliberately and legally to find weaknesses first.
- The classic role is the penetration tester — with permission, breaking into systems to find vulnerabilities, then reporting them (the kill chain and vulnerabilities of Days 71, 75, applied offensively but ethically); related are red teamers and vulnerability researchers.
- Governance, risk, and compliance is larger than beginners expect: managing security at the organisational/policy level — meeting compliance, managing risk, policies, audits, advising leadership — suiting organisational and communication strengths, and a genuine entry point for analytical rather than deeply technical people.

DAY- 88 Tools You'll see

- The categories of tools security professionals use, so the working environment of real roles feels familiar rather than foreign.

- The crucial point to hold from the start: tools are interchangeable; the thinking is what transfers.
- A **SIEM** (security information and event management) is the SOC's central tool: it collects logs from across an organisation, centralises them, and lets analysts search, correlate, and alert — the workbench for the haystack-of-logs problem.
- **EDR** (endpoint detection and response) tools install on individual computers (laptops, servers) and continuously watch their activity — processes, connections, persistence, suspicious behaviour — letting responders isolate machines remotely;
- SIEM centralises across everything, EDR is the deep watch on each endpoint IDS/IPS (intrusion detection/prevention) systems watch network traffic for attack signs at boundaries.Network monitoring catches the beaconing at scale.
- AI assistance is increasingly built into modern SIEM/EDR.
- Prevention tools enforce defences: firewalls control network traffic; secure email gateways filter phishing; identity and access management (IAM) tools, including multi-factor auth as a baseline, control who accesses what; endpoint protection blocks malicious software.
- Investigation tools support deeper analysis: packet analysers like Wireshark examine traffic; forensic tools examine compromised systems while preserving evidence; sandboxes safely execute suspicious files to see what they do; threat intelligence platforms aggregate techniques and campaigns (where ATT&CK lives in practice).
- Management tools support the discipline over time: vulnerability scanners report which CVEs affect what; patch management coordinates deployment ;GRC platforms support managing risk and tracking compliance.

DAY-89 Hiring Managers

- The most important thing to grasp from the start: hiring managers hire for signals, not credentials. They're trying to predict from limited information whether you'll be capable of learning and doing the job — looking for evidence you genuinely

understand the work, have shown initiative beyond claiming interest, and can communicate what you know clearly.

- None of this requires a particular degree, exhaustive certifications, or years of prior experience for entry-level. What it needs is demonstrable foundation, demonstrable interest, and the ability to talk about both.
- Hiring managers look for four main signals in entry-level candidates.
    - Demonstrable foundation — evidence you understand how security works conceptually (kill chain, ATT&CK, defence in depth, risk-based prioritisation), not just claims of interest; the substance this whole course built, exactly what employers listen for in "tell me about something you've learned."
    - Demonstrable work — evidence you've done things, not just learned them, where a portfolio of actual work matters enormously because it shifts you from "I claim to know X" to "look what I produced"; your nine artifacts (audit, phishing analysis, threat model, network investigation, defence plan, Linux investigation, forensics, kill chain analysis, vulnerability triage) are tangible evidence many other candidates lack.
    - Ability to communicate — explaining what you understand and what you've done clearly, in your own words; the surest sign of real comprehension is being able to teach it.
    - The human side — curiosity, integrity, teachability — is a way of being signalled through how you present yourself, mattering as much as anything technical for entry-level.
- Presenting what you've built matters as much as building it. Lead with specifics, not generalities — "I've built a portfolio of nine projects, for example mapping a real breach to the kill chain and triaging vulnerabilities by real risk" is concrete and memorable; "I'm interested in security" is forgettable. Concrete examples beat vague claims everywhere (CV, LinkedIn, interviews).

- Explain your thinking, not just what you did: the most impressive thing about a project isn't the artifact but how you reasoned (why severity isn't real risk; why the attacker must win every stage and defenders need only one) — this elevates the portfolio from "things I made" to "evidence of how I think.
- Be honest about what you don't know — hiring managers spot inflation instantly, and "I haven't used that tool but I understand the category and would pick it up quickly" beats pretending; honesty about what you don't know, combined with confidence in what you do, signals the integrity and teachability they want, especially for entry-level.
- Four common mistakes derail entry-level applicants, all avoidable.
    - Focusing on certifications and tools you don't have instead of the foundation you do — hiring managers train you on tools; they hire for the thinking, so lead with concepts and frameworks.
    - Vague enthusiasm without substance — "I'm passionate about cybersecurity" tells the manager nothing; concrete evidence of what your interest has produced (the 90-day foundation, nine projects, the specific ones that grabbed you and why) is compelling.
    - Imposter syndrome paralysis — feeling so underqualified you don't apply, when the requirements list is a wish list and hiring managers know entry candidates don't match every item; apply confidently to roles even without ticking every box.
    - Not networking — entry-level roles are often found through being known, and the LinkedIn posting you've been doing is exactly this engagement (more important than people expect). The better approach is visibility while applying, not silent applications alone.

DAY-90 Wrap-Up

- You have a complete conceptual foundation plus nine portfolio artifacts showing what you can do with it. Today you assemble it all into one complete portfolio — the 10th and final artifact — and the course closes.
- A scattered set of artifacts is much less than the sum of its parts; assembled into a coherent portfolio with cover narrative, summaries, and a personal statement, the same artifacts become something you hand to a recruiter, link from LinkedIn, and share in an interview.
- Before assembly, name what you have clearly. A complete conceptual foundation — the digital world, networks, Linux, and the whole security landscape (CIA triad, kill chain, ATT&CK, threat actors, vulnerabilities, malware, defence in depth, vulnerability management, patching, incident response, AI, compliance, risk management, jobs, tools); you can read security articles, job descriptions, threat reports, and incident messages and understand them — foundational literacy that isn't trivial.
- A way of thinking trained — you reason like someone taught to think about security: mapping attacks to the kill chain, prioritising by real risk over raw severity, asking "what could someone do with this?", layering defences, approaching security as risk management not impossible perfection — what employers actually want, far less common than assumed.
- And nine portfolio artifacts: Personal Security Audit (Day 3), Phishing Analysis (15), Threat Model (20), Wireshark Investigation (35), Maya Defence Plan (45), Linux Investigation (56), Linux Forensics (65), Kill Chain Breach Analysis (73), Vulnerability Triage Report (80) — concrete demonstrable work showing range across technical, investigative, analytical, architectural, forensic, attack-analysis, and risk-based decision-making.
- A portfolio isn't just a folder of PDFs; it has structure that helps a reader navigate and a story that makes it more than its parts. Yours has: a cover page introducing you and what you've built; a portfolio overview telling the story of the journey

- Statements to be used while presenting:
    - "I've completed a ninety-day cybersecurity foundation course and produced this portfolio of ten artifacts demonstrating my understanding across the field, from personal security audits and threat modelling to network investigation, Linux forensics, attack analysis, and vulnerability triage."
- Each Artifact should be selling the main idea in the project.
    - "Kill Chain Breach Analysis: a structured analysis of a real-world breach scenario mapped to all seven kill-chain stages, identifying the specific techniques and the defensive points where the chain could have been broken — demonstrating attack analysis and the application of security frameworks to a concrete case"
- Be honest and specific in the personal statement: "the moment vulnerability triage clicked was..." beats generic claims to passion, because specifics reveal you as a person — which is what an employer hires.
- Two things to carry forward: keep your portfolio updated as you do real work (new projects augment the foundation pieces over time), and keep the way of thinking sharp by using the frameworks daily on whatever security news or situations you encounter.